

Computer Organization and Design

Chapter 1

Exercise 1.1

List and describe three types of computers.

Solution

- Personal Computer: This class is made for personal use and it outcome with many kinds of workloads at reasonable performance price
- Super Computers: This class is made to outcome with the highest performance possible on specific workloads.
- Embedded Computers: This class is made to outcome with the best Watt-Performance, it outcome with a specialized and easy workload.

Exercise 1.2

The seven great ideas in computer architecture are similar to ideas from other fields. Match the seven ideas from computer architecture, “Use Abstraction to Simplify Design,” “Make the Common Case Fast,” “Performance via Parallelism,” “Performance via Pipelining,” “Performance via Prediction” “Hierarchy of Memories,” and “Dependability via Redundancy” to the following ideas from other fields:

- A) Assembly lines in automobile manufacturing
- B) Suspension bridge cables
- C) Aircraft and marine navigation systems that incorporate wind information
- D) Express elevators in buildings
- E) Library reserve desk
- F) Increasing the gate area on a CMOS transistor to decrease its switching time
- G) Building self-driving cars whose control systems partially rely on existing sensor systems already installed into the base vehicle, such as lane departure systems and smart cruise control systems

Solution

- A) Performance via Pipelining
- B) Use Abstraction to Simplify Design
- C) Performance via Prediction
- D) Performance via Parallelism
- E) Hierarchy of Memories
- F) Make the Common Case Fast
- G) Dependability via Redundancy

Exercise 1.3

Describe the steps that transform a program written in a high-level language such as C into a representation that is directly executed by a computer processor.

Solution

First the compiler transform the high level language to assembly code, then this assembler code pass trough the assembler to transform to machine code.

Exercise 1.4

Assume a color display using 8 bits for each of the primary colors (red, green, blue) per pixel and a frame size of 1280×1024 .

- A) What is the minimum size in bytes of the frame buffer to store a frame?
 B) How long would it take, at a minimum, for the frame to be sent over a 100 Mbit/s network?

Solution

- A) A single frame need $1280 * 1024 * 3 = 3932160$ Bytes, that is almost 4 MB
 B) $\frac{3932160 * 8}{100 * 10^6} = 0.3145728$ seconds

Exercise 1.5

Consider the table below, which tracks several performance indicators for Intel desktop processors since 2010. The “Tech” column shows the minimum feature size of each processor’s fabrication process. Assume that die size has remained relatively constant, and the number of transistors comprised in each processor scales at $(1/t)^2$, where t = the minimum feature size.

For each performance indicator, calculate the average rate of improvement from 2010 to 2019 as well as the number of years required to double each at that corresponding rate.

Desktop processor	Year	Tech	Max. clock speed (GHz)	Integer IPC/ core	Cores	Max. DRAM Bandwidth (GB/s)	SP floating point (Gflop/s)	L3 cache (MiB)
Westmere i7-620	2010	32	3.33	4	2	17.1	107	4
Ivy Bridge i7-3770K	2013	22	3.90	6	4	25.6	250	8
Broadwell i7-6700K	2015	14	4.20	8	4	34.1	269	8
Kaby Lake i7-7700K	2017	14	4.50	8	4	38.4	288	8
Coffee Lake i7-9700K	2019	14	4.90	8	8	42.7	627	12
Imp./year		__%	__%	__%	__%	__%	__%	__%
Doubles every		__years	__years	__years	__years	__years	__years	__years

Solution

Subject	Improvement per Year	Doubles every
Tech	25.39%	7.875 Years
Max. Clock speed	16.34%	12.23 Years
Integer IPC / core	22.22%	9 Years
Cores	44.44%	4.5 Years
Max. DRAM Bandwidth	27.74%	7.2 Years
SP floating point	65.10%	3.07 Years
L3 Cache	33.33%	6 Years

Exercise 1.6

Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

- A) Which processor has the highest performance expressed in instructions per second?

- B) If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
- C) We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

Solution

- A) If we want to find the number of instructions per second we need to divide the number of instructions over the time needed by the CPU to complete the task so the formula would be

$$\frac{\text{Instructions count}}{\text{CPU time}} = \frac{\text{Clock rate}}{\text{CPI}}$$

$$IPS_{P1} = \frac{3 * 10^9 \frac{\text{Cycles}}{\text{Second}}}{1.5 \frac{\text{Cycles}}{\text{Instructions}}} = 2 * 10^9 \frac{\text{Instructions}}{\text{Second}}$$

$$IPS_{P2} = \frac{2.5 * 10^9 \frac{\text{Cycles}}{\text{Second}}}{1 \frac{\text{Cycles}}{\text{Instructions}}} = 2.5 * 10^9 \frac{\text{Instructions}}{\text{Second}}$$

$$IPS_{P3} = \frac{4 * 10^9 \frac{\text{Cycles}}{\text{Second}}}{2.2 \frac{\text{Cycles}}{\text{Instructions}}} = 1.81 * 10^9 \frac{\text{Instructions}}{\text{Second}}$$

So the winner is P2

- B) We can get the number of instructions using the instructions per second of before

*Instruction count = Instructions per second * seconds*

$$IC_{P1} = (10 \text{ sec}) * (2 * 10^9 \text{ instr/sec}) = 20 * 10^{10} \text{ instructions}$$

$$IC_{P2} = (10 \text{ sec}) * (2.5 * 10^9 \text{ instr/sec}) = 25 * 10^{10} \text{ instructions}$$

$$IC_{P3} = (10 \text{ sec}) * (1.81 * 10^9 \text{ instr/sec}) = 18.1 * 10^{10} \text{ instructions}$$

The number of cycles needed are

*Number of cycles = Instructions count * cycles per instructions*

$$NC_{P1} = (20 * 10^{10} \text{ instr}) * (1.5 \text{ cycl/instr}) = 3 * 10^{11} \text{ cycles}$$

$$NC_{P2} = (25 * 10^{10} \text{ instr}) * (1.0 \text{ cycl/instr}) = 2.5 * 10^{11} \text{ cycles}$$

$$NC_{P3} = (18.1 * 10^{10} \text{ instr}) * (2.2 \text{ cycl/instr}) = 4.0 * 10^{11} \text{ cycles}$$

$$C) \frac{ET_{New}}{ET_{Old}} = \frac{Instr * (1.2 * CPI) * 1 / CR_{New}}{Instr * CPI * 1 / CR_{Old}} = 0.7$$

$$CR_{New} = 1.2 * CR_{Old} / 0.7$$

Exercise 1.7

Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.

Given a program with a dynamic instruction count of 1.0E6 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D, which is faster: P1 or P2?

- A) What is the global CPI for each implementation?
- B) Find the clock cycles required in both cases.

Solution

- A) First we need to calculate the total time of the CPU to do the calculations for every class

$$CPUT_{P1} = \frac{(0.1*1+0.2*2+0.5*3+0.2*3)*10^6 \text{ instr/cycle}}{2.5*10^9 \text{ Hz}} = 1.04*10^{-3} \text{ sec}$$

$$CPUT_{P2} = \frac{(0.1*2+0.2*2+0.5*2+0.2*2)*10^6 \text{ instr/cycle}}{3.0*10^9 \text{ Hz}} = 0.66*10^{-3} \text{ sec}$$

Obviously P2 is faster than P1

$$GCPI_{P1} = (0.1*1+0.2*2+0.5*3+0.2*3) = 2.6$$

$$GCPI_{P2} = (0.1*2+0.2*2+0.5*2+0.2*2) = 2.0$$

B) Clock cycles

$$CC_{P1} = 2.6*10^6$$

$$CC_{P2} = 2.0*10^6$$

Exercise 1.8

Compilers can have a profound impact on the performance of an application. Assume that for a program, compiler A results in a dynamic instruction count of 1.0E9 and has an execution time of 1.1 s, while compiler B results in a dynamic instruction count of 1.2E9 and an execution time of 1.5 s.

- Find the average CPI for each program given that the processor has a clock cycle time of 1 ns.
- Assume the compiled programs run on two different processors. If the execution times on the two processors are the same, how much faster is the clock of the processor running compiler A's code versus the clock of the processor running compiler B's code?
- A new compiler is developed that uses only 6.0E8 instructions and has an average CPI of 1.1. What is the speedup of using this new compiler versus using compiler A or B on the original processor?

Solution

$$A) \text{ CPI} = \frac{\text{CPU time}}{\text{Instruction count} * \text{Clock cycle time}}$$

$$CPI_A = \frac{1.1 \text{ sec}}{1*10^9 \text{ instr} * 1*10^{-9} \text{ sec}} = 1.1$$

$$CPI_B = \frac{1.5 \text{ sec}}{1.2*10^9 \text{ instr} * 1*10^{-9} \text{ sec}} = 1.25$$

$$B) \frac{CR_B}{CR_A} = \frac{1.2*10^9 \text{ instr} * 1.25}{1*10^9 \text{ instr} * 1.1} = 1.36$$

$$C) \frac{ET_A}{ET_C} = \frac{(1.0*10^9 \text{ instr}) * 1.1 * (1*10^{-9} \text{ sec})}{(6.0*10^8 \text{ instr}) * 1.1 * (1*10^{-9} \text{ sec})} = 1.66$$

$$\frac{ET_A}{ET_C} = \frac{(1.2*10^9 \text{ instr}) * 1.25 * (1*10^{-9} \text{ sec})}{(6.0*10^8 \text{ instr}) * 1.1 * (1*10^{-9} \text{ sec})} = 2.5$$

Exercise 1.9

The Pentium 4 Prescott processor, released in 2004, had a clock rate of 3.6 GHz and voltage of 1.25 V. Assume that, on average, it consumed 10 W of static power and 90 W of dynamic power. The Core i5 Ivy Bridge, released in 2012, has a clock rate of 3.4 GHz and voltage of 0.9 V. Assume that, on average, it consumed 30 W of static power and 40 W of dynamic power.

- For each processor find the average capacitive loads.
- Find the percentage of the total dissipated power comprised by static power and the ratio of static power to dynamic power for each technology.
- If the total dissipated power is to be reduced by 10%, how much should the voltage be reduced to maintain the same leakage current? Note: power is defined as the product of voltage and current.

Solution

$$1) \text{ Capacitive Load} = \frac{\text{Power}}{1/2 * \text{Frequency} * \text{Voltage}^2}$$

$$CL_{Pentium} = \frac{90 \text{ W}}{1/2 * 3.6 \text{ GHz} * (1.25 \text{ V})^2} = 32 * 10^{-9} \text{ F}$$

$$CL_{Core} = \frac{40 \text{ W}}{1/2 * 3.4 \text{ GHz} * (0.9 \text{ V})^2} = 29.04 * 10^{-9} \text{ F}$$

$$2) \text{ Total Power} = \frac{\text{Static Power}}{\text{Static Power} + \text{Dynamic Power}}$$

$$TP_{Pentium} = 10 \text{ W} / (10 \text{ W} + 90 \text{ W}) = 0.1 = 10 \%$$

$$TP_{Core} = 30 \text{ W} / (30 \text{ W} + 40 \text{ W}) = 0.4285 = 42.85 \%$$

$$3) \text{ Leak Current} = \frac{\text{Power}}{\text{Voltage}}$$

$$LC_{Pentium} = \frac{10 \text{ W}}{1.25 \text{ V}} = 8 \text{ A}$$

$$LC_{Core} = \frac{30 \text{ W}}{0.9 \text{ V}} = 33.3 \text{ A}$$

$$\text{Total Power} = (1/2 * \text{Frequency} * \text{Capacitive Load} * \text{Voltage}^2) + (\text{Leak Current} * \text{Voltage})$$

$$0.9 * 100 \text{ W} = ((1/2 * 3.6 \text{ GHz} * 32 \text{ nF} * V^2) + 8 \text{ A} * V)$$

$$0 = 57.6 V^2 + 8 V - 90$$

$$V = 1.1824 \text{ V}$$

$$\text{Total Power} = (1/2 * \text{Frequency} * \text{Capacitive Load} * \text{Voltage}^2) + (\text{Leak Current} * \text{Voltage})$$

$$0.9 * 70 \text{ W} = ((1/2 * 3.4 \text{ GHz} * 29.04 \text{ nF} * V^2) + 33.3 \text{ A} * V)$$

$$0 = 49.36 V^2 + 33.3 V - 63$$

$$V = 0.8417 \text{ V}$$

Exercise 1.10

Assume for arithmetic, load/store, and branch instructions, a processor has CPIs of 1, 12, and 5, respectively. Also assume that on a single processor a program requires the execution of 2.56E9 arithmetic instructions, 1.28E9 load/store instructions, and 256 million branch instructions. Assume that each processor has a 2 GHz clock frequency.

Assume that, as the program is parallelized to run over multiple cores, the number of arithmetic and load/store instructions per processor is divided by 0.7 × p (where p is the number of processors) but the number of branch instructions per processor remains the same.

- Find the total execution time for this program on 1, 2, 4, and 8 processors, and show the relative speedup of the 2, 4, and 8 processors result relative to the single processor result.
- If the CPI of the arithmetic instructions was doubled, what would the impact be on the execution time of the program on 1, 2, 4, or 8 processors?
- To what should the CPI of load/store instructions be reduced in order for a single processor to match the performance of four processors using the original CPI values?

Solution

$$1) ET_{1P} = \frac{(2.56 + 10^9 \text{ instr}) * 1 + (1.28 * 10^9 \text{ instr}) * 12 + (0.256 * 10^9 \text{ instr}) * 5}{(2 * 10^9) \text{ Hz}} = 9.6 \text{ sec}$$

$$ET_{2P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 1}{0.7 * 2} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 2} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 7.04 \text{ sec}$$

$$ET_{4P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 1}{0.7 * 4} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 4} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 3.84 \text{ sec}$$

$$ET_{8P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 1}{0.7 * 8} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 8} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 2.24 \text{ sec}$$

$$\text{Speedup}_{2P} = 9.6 / 7.04 = 1.36$$

$$\text{Speedup}_{4P} = 9.6 / 3.84 = 2.5$$

$$\text{Speedup}_{8P} = 9.6 / 2.24 = 4.28$$

$$2) \quad ET_{1P} = \frac{(2.56 + 10^9 \text{ instr}) * 2 + (1.28 * 10^9 \text{ instr}) * 12 + (0.256 * 10^9 \text{ instr}) * 5}{(2 * 10^9) \text{ Hz}} = 10.88 \text{ sec}$$

$$ET_{2P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 2}{0.7 * 2} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 2} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 7.95 \text{ sec}$$

$$ET_{4P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 2}{0.7 * 4} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 4} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 4.29 \text{ sec}$$

$$ET_{8P} = \frac{\frac{(2.56 + 10^9 \text{ ins}) * 2}{0.7 * 8} + \frac{(1.28 * 10^9 \text{ ins}) * 12}{0.7 * 8} + (0.256 * 10^9 \text{ ins}) * 5}{(2 * 10^9) \text{ Hz}} = 2.46 \text{ sec}$$

$$ET_{4P} = \frac{(2.56 + 10^9 \text{ instr}) * 1 + (1.28 * 10^9 \text{ instr}) * CPI_{LW} + (0.256 * 10^9 \text{ instr}) * 5}{(2 * 10^9) \text{ Hz}} = 3.84 \text{ sec}$$

$$3) \quad CPI_{LW} = \frac{(3.84 \text{ sec} * 2 * 10^9 \text{ Hz}) - (0.256 * 10^9 \text{ instr}) * 5 - (2.56 + 10^9 \text{ instr}) * 1}{(1.28 * 10^9 \text{ instr})} = 3$$

Exercise 1.11

Assume a 15 cm diameter wafer has a cost of 12, contains 84 dies, and has 0.020 defects/cm². Assume a 20 cm diameter wafer has a cost of 15, contains 100 dies, and has 0.031 defects/cm².

- 1) Find the yield for both wafers.
- 2) Find the cost per die for both wafers.
- 3) If the number of dies per wafer is increased by 10% and the defects per area unit increases by 15%, find the die area and yield.
- 4) Assume a fabrication process improves the yield from 0.92 to 0.95. Find the defects per area unit for each version of the technology given a die area of 200 mm².

Solution

$$1) \quad \text{Die Area} = \frac{\text{Wafer Area}}{\text{Dies per Wafer}}$$

$$\text{Yield} = \frac{1}{(1 + \text{Defects per Area} * \text{Die Area} / 2)^2}$$

I will assume that N = 2

$$DA_{W1} = \frac{\pi * (15 \text{ cm} / 2)^2}{84 \text{ dies}} = 2.1037 \text{ cm}^2$$

$$Y_{W1} = \frac{1}{(1 + 0.020 \text{ defect} / \text{cm}^2 * 1 / 2 * 2.1037 \text{ cm}^2)^2} = 0.9592$$

$$DA_{w2} = \frac{\pi * (20 \text{ cm}/2)^2}{100 \text{ dies}} = 3.1415 \text{ cm}^2$$

$$Y_{w2} = \frac{1}{(1 + 0.031 \text{ defect/cm}^2 * 1/2 * 3.1415 \text{ cm}^2)^2} = 0.9092$$

$$2) \text{ Cost per Die} = \frac{\text{Cost per Wafer}}{\text{Dies per Wafer} * \text{Yield}}$$

$$CD_{w1} = \frac{12}{84 * 0.9592} = 0.1489$$

$$CD_{w2} = \frac{15}{100 * 0.9092} = 0.1649$$

$$3) \text{ } DA_{w1} = \frac{\pi * (15 \text{ cm}/2)^2}{1.1 * 84 \text{ dies}} = 1.9124 \text{ cm}^2$$

$$Y_{w1} = \frac{1}{(1 + 1.15 * 0.020 \text{ defect/cm}^2 * 1/2 * 1.9124 \text{ cm}^2)^2} = 0.9574$$

$$DA_{w2} = \frac{\pi * (20 \text{ cm}/2)^2}{1.1 * 100 \text{ dies}} = 2.8559 \text{ cm}^2$$

$$Y_{w2} = \frac{1}{(1 + 1.15 * 0.031 \text{ defect/cm}^2 * 1/2 * 2.8559 \text{ cm}^2)^2} = 0.9054$$

$$4) \text{ Defects per Area} = \frac{1/\sqrt[2]{\text{Yield}} - 1}{\text{Die Area}/2}$$

$$\text{Defects per Area} = \frac{1/\sqrt[2]{0.92} - 1}{2 \text{ cm}^2/2} = 0.0425 \text{ defects/cm}^2$$

$$\text{Defects per Area} = \frac{1/\sqrt[2]{0.95} - 1}{2 \text{ cm}^2/2} = 0.0259 \text{ defects/cm}^2$$

Exercise 1.12

The results of the SPEC CPU2006 bzip2 benchmark running on an AMD Barcelona has an instruction count of 2.389E12, an execution time of 750 s, and a reference time of 9650 s.

- 1) Find the CPI if the clock cycle time is 0.333 ns.
- 2) Find the SPECratio.
- 3) Find the increase in CPU time if the number of instructions of the benchmark is increased by 10% without affecting the CPI.
- 4) Find the increase in CPU time if the number of instructions of the benchmark is increased by 10% and the CPI is increased by 5%.
- 5) Find the change in the SPECratio for this change.
- 6) Suppose that we are developing a new version of the AMD Barcelona processor with a 4 GHz clock rate. We have added some additional instructions to the instruction set in such a way that the number of instructions has been reduced by 15%. The execution time is reduced to 700 s and the new SPECratio is 13.7. Find the new CPI.
- 7) This CPI value is larger than obtained in 1.12.1 as the clock rate was increased from 3 GHz to 4 GHz. Determine whether the increase in the CPI is similar to that of the clock rate. If they are dissimilar, why?
- 8) By how much has the CPU time been reduced?
- 9) For a second benchmark, libquantum, assume an execution time of 960 ns, CPI of 1.61, and clock rate of 3 GHz. If the execution time is reduced by an additional 10% without affecting the CPI and with a clock rate of 4 GHz, determine the number of instructions.
- 10) Determine the clock rate required to give a further 10% reduction in CPU time while maintaining the number of instructions and with the CPI unchanged.
- 11) Determine the clock rate if the CPI is reduced by 15% and the CPU time by 20% while the number of instructions is unchanged.

Solution

- 1) $CPI = \frac{750\text{ s}}{2.389 * 10^{12} * 0.333 * 10^{-9}\text{ s}} = 0.9427$
- 2) $SPECratio = \frac{Reference\ Time}{Execution\ Time}$
 $SPECratio = \frac{9650\text{ s}}{750\text{ s}} = 12.8666$
- 3) $\frac{ET_{New}}{ET_{Old}} = \frac{1.1 * 2.389 * 10^{12}\text{ instr} * 0.9427\text{ cyc/instr} * 0.333 * 10^{-9}\text{ s}}{750\text{ s}} = 1.1$
- 4) $\frac{ET_{New}}{ET_{Old}} = \frac{1.1 * 2.389 * 10^{12}\text{ instr} * (1.05 * 0.9427\text{ cyc/instr}) * 0.333 * 10^{-9}\text{ s}}{750\text{ s}} = 1.155$
- 5) $SPECratio = \frac{9650\text{ s}}{1.155 * 750\text{ s}} = 11.1399$
- 6) $CPI = \frac{700\text{ s} * 4 * 10^9\text{ Hz}}{0.85 * (2.389 * 10^{12}\text{ instr})} = 1.3788$
- 7) When we increase the clock rate we improve by $4/3 = 1.3333$. But CPI increase $1.3788/0.9427 = 1.4626$. Both clock rate and CPI are relative closer but is better to prioritize CPI
- 8) $\frac{ET_{Old}}{ET_{New}} = \frac{750\text{ s}}{700\text{ s}} = 1.0714$
- 9) $IC = \frac{0.9 * 960\text{ sec} * (4 * 10^9\text{ Hz})}{1.61\text{ cycles/instr}} = 2146.58 * 10^9\text{ Instr}$
- 10) $CR = \frac{2146.58 * 10^9\text{ Instr} * 1.61\text{ cycles/instr}}{0.8 * 960\text{ sec}} = 4.5\text{ GHz}$
- 11) $CR = \frac{2146.58 * 10^9\text{ Instr} * 0.85 * 1.61\text{ cycles/instr}}{0.8 * 960\text{ sec}} = 3.825\text{ GHz}$

Exercise 1.13

Section 1.11 cites as a pitfall the utilization of a subset of the performance equation as a performance metric. To illustrate this, consider the following two processors. P1 has a clock rate of 4 GHz, average CPI of 0.9, and requires the execution of 5.0E9 instructions. P2 has a clock rate of 3 GHz, an average CPI of 0.75, and requires the execution of 1.0E9 instructions.

- 1) One usual fallacy is to consider the computer with the largest clock rate as having the highest performance. Check if this is true for P1 and P2.
- 2) Another fallacy is to consider that the processor executing the largest number of instructions will need a larger CPU time. Considering that processor P1 is executing a sequence of 1.0E9 instructions and that the CPI of processors P1 and P2 do not change, determine the number of instructions that P2 can execute in the same time that P1 needs to execute 1.0E9 instructions.
- 3) A common fallacy is to use MIPS (millions of instructions per second) to compare the performance of two different processors, and consider that the processor with the largest MIPS has the largest performance. Check if this is true for P1 and P2.
- 4) Another common performance figure is MFLOPS (millions of floating-point operations per second), defined as $MFLOPS = \text{No. FP operations} / (\text{execution time} * 1E6)$ but this figure has the same problems as MIPS. Assume that 40% of the instructions executed on both P1 and P2 are floating-point instructions. Find the MFLOPS figures for the processors.

Solution

- 1) $ET_{P1} = \frac{0.9\text{ cycles/instr} * (5 * 10^9\text{ instr})}{4 * 10^9\text{ Hz}} = 1.125\text{ sec}$

$$ET_{P2} = \frac{0.75 \text{ cycles/instr} * (1 * 10^9 \text{ instr})}{3 * 10^9 \text{ Hz}} = 0.25 \text{ sec}$$

P2 is faster than P1

$$2) \quad ET_{P1} = \frac{0.9 \text{ cycles/instr} * (1 * 10^9 \text{ instr})}{4 * 10^9 \text{ Hz}} = 0.225 \text{ sec}$$

$$IC_{P2} = \frac{0.225 \text{ sec} * (3 * 10^9 \text{ Hz})}{0.75 \text{ cycles/instr}} = 0.9 * 10^9 \text{ instr}$$

$$3) \quad MIPS_{P1} = \frac{(5 * 10^9 \text{ instr})}{1.125 \text{ sec} * 10^6} = 4.44 * 10^3$$

$$MIPS_{P2} = \frac{(1 * 10^9 \text{ instr})}{0.25 \text{ sec} * 10^6} = 4 * 10^3$$

$$4) \quad ET_{P1} = \frac{0.9 \text{ cycles/instr} * 0.4 * (5 * 10^9 \text{ instr})}{4 * 10^9 \text{ Hz}} = 0.45 \text{ sec}$$

$$ET_{P2} = \frac{0.75 \text{ cycles/instr} * 0.4 * (1 * 10^9 \text{ instr})}{3 * 10^9 \text{ Hz}} = 0.1 \text{ sec}$$

$$MFLOPS_{P1} = \frac{0.4 * (5 * 10^9 \text{ instr})}{0.45 \text{ sec} * 10^6} = 4.44 * 10^3$$

$$MFLOPS_{P2} = \frac{0.4 * (1 * 10^9 \text{ instr})}{0.1 \text{ sec} * 10^6} = 4 * 10^3$$

Exercise 1.14

Another pitfall cited in Section 1.11 is expecting to improve the overall performance of a computer by improving only one aspect of the computer. Consider a computer running a program that requires 250 s, with 70 s spent executing FP instructions, 85 s executed L/S instructions, and 40 s spent executing branch instructions.

- 1) By how much is the total time reduced if the time for FP operations is reduced by 20%?
- 2) By how much is the time for INT operations reduced if the total time is reduced by 20%?
- 3) Can the total time can be reduced by 20% by reducing only the time for branch instructions?

Solution

$$1) \quad RTAI = 250 \text{ s} - \left(\frac{70 \text{ s}}{1/0.8} + (250 \text{ s} - 70 \text{ s}) \right) = 14 \text{ s}$$

$$2) \quad \text{I will assume the rest of seconds are INT operations } 250\text{s} - 70\text{s} - 85\text{s} - 40\text{s} = 55\text{s}$$

$$3) \quad AOI = \frac{55 \text{ s}}{0.8 * 250 \text{ s} - (250 - 55 \text{ s})} = 11$$

$$4) \quad \text{No it can't. Because a 20\% reduction is 200s and we can only reduce 40s}$$

Exercise 1.15

Assume a program requires the execution of 50×10^6 FP instructions, 110×10^6 INT instructions, 80×10^6 L/S instructions, and 16×10^6 branch instructions. The CPI for each type of instruction is 1, 1, 4, and 2, respectively. Assume that the processor has a 2 GHz clock rate.

- 1) By how much must we improve the CPI of FP instructions if we want the program to run two times faster?
- 2) By how much must we improve the CPI of L/S instructions if we want the program to run two times faster?
- 3) By how much is the execution time of the program improved if the CPI of INT and FP instructions is reduced by 40% and the CPI of L/S and Branch is reduced by 30%?

Solution

$$1) ET_{Total} = \frac{((1*50)+(1*110)+(4*80)+(2*16))*10^6}{2*10^9} = 256*10^{-3} sec$$

$$ET_{FP} = \frac{1 cyc / instr 50*10^6 instr}{2*10^9} = 25*10^{-3} sec$$

$$0.5*256*10^{-3} s = (25*10^{-3} s) / n + (256-25)*10^{-3} s$$

$$n = \frac{(25*10^{-3} s)}{(0.5*256*10^{-3} s) - (256-25)*10^{-3} s} = -0.2427$$

$$CPI_{FP} = 1 / -0.2427 * 1 cyc / instr = -4.12 cyc / instr$$

No possible improvement can be done to achieve 2x performance

$$2) ET_{L/S} = \frac{4 cyc / instr 80*10^6 instr}{2*10^9} = 160*10^{-3} sec$$

$$0.5*256*10^{-3} s = (160*10^{-3} s) / n + (256-160)*10^{-3} s$$

$$n = \frac{(160*10^{-3} s)}{(0.5*256*10^{-3} s) - (256-160)*10^{-3} s} = 5$$

$$CPI_{L/S} = 1/5 * 1 cyc / instr = 0.2 cyc / instr$$

$$3) ET_{New Total} = \frac{((0.6*1*50)+(0.6*1*110)+(0.7*4*80)+(0.7*2*16))*10^6}{2*10^9} = 171.2*10^{-3} sec$$

$$\frac{ET_{Old}}{ET_{New}} = \frac{256 s * 10^{-3}}{171.2 s * 10^{-3}} = 1.4953$$

Exercise 1.16

When a program is adapted to run on multiple processors in a multiprocessor system, the execution time on each processor is comprised of computing time and the overhead time required for locked critical sections and/or to send data from one processor to another.

Assume a program requires $t = 100$ s of execution time on one processor. When run p processors, each processor requires t/p s, as well as an additional 4 s of overhead, irrespective of the number of processors. Compute the per-processor execution time for 2, 4, 8, 16, 32, 64, and 128 processors. For each case, list the corresponding speedup relative to a single processor and the ratio between actual speedup versus ideal speedup (speedup if there was no overhead).

Solution

Number of cores	Execution time	Speedup
1	100s	100s/100s=1
2	100s/2+4s=54s	100s/54s=1.8518
4	100s/4+4s=29s	100s/29s=3.4482
8	100s/8+4s=16.5s	100s/16.5s=6.0606
16	100s/16+4s=10.25s	100s/10.25s=9.756
32	100s/32+4s=7.125s	100s/7.125s=14.035
64	100s/64+4s=5.5625s	100s/5.5625s=17.9775
128	100s/128+4s=4.78125s	100s/4.78125s=20.915

Chapter 2

Exercise 2.1

For the following C statement, write the corresponding RISC-V assembly code. Assume that the C variables f, g, and h, have already been placed in registers x5, x6, and x7 respectively. Use a minimal number of RISC-V assembly instructions.

```
f = g + (h - 5);
```

Solution

```
addi    x7, x7, -5
add     x5, x6, x7
```

Exercise 2.2

Write a single C statement that corresponds to the two RISC-V assembly instructions below.

```
add     f, g, h
add     f, i, f
```

Solution

```
f = i + g + h;
```

Exercise 2.3

For the following C statement, write the corresponding RISC-V assembly code. Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively.

```
B[8] = A[i-j];
```

Solution

```
4 Bytes = sizeof(int)
sub     x30, x28, x29 // x30 = i-j
slli    x30, x30, 2   // x30 = 4*(i-j)
add     x30, x30, x10 // x30 = &A[]+4*(i-j)
lw      x30, 0(x30)   // x30 = A[i-j]
sw      x30, 32(x11)  // B[8] = A[i-j]
```

Exercise 2.4

For the RISC-V assembly instructions below, what is the corresponding C statement? Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively.

```
slli    x30, x5, 2    // x30 = f*4
add     x30, x10, x30 // x30 = &A[f]
slli    x31, x6, 2    // x31 = g*4
add     x31, x11, x31 // x31 = &B[g]
lw      x5, 0(x30)    // f = A[f]

addi    x12, x30, 8
lw      x30, 0(x12)
add     x30, x30, x5
lw      x30, 0(31)
```

Solution

```

addi    x12, x30, 4    // x12 = &A[f]+4 = &A[f+1]
lw      x30, 0(x12)    // x30 = A[f+1]
add     x30, x30, x5    // x30 = A[f+1]+A[f]
lw      x30, 0(x31)    // B[g] = A[f+1]+A[f]

```

Exercise 2.5

Show how the value 0xabcdef12 would be arranged in memory of a little-endian and a big-endian machine. Assume the data are stored starting at address 0 and that the word size is 4 bytes.

Solution

Little endian:

```

0C    08    04    00
AB    CD    EF    12

```

Big Endian

```

0C    08    04    00
12    EF    CD    AB

```

Exercise 2.6

Translate 0xabcdef12 into decimal.

Solution

$$10 \cdot 16^7 + 11 \cdot 16^6 + 12 \cdot 16^5 + 13 \cdot 16^4 + 14 \cdot 16^3 + 15 \cdot 16^2 + 1 \cdot 16 + 2 = 2882400018$$

Exercise 2.7

Translate the following C code to RISC-V. Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively. Assume that the elements of the arrays A and B are 4-byte words:

```
B[8] = A[i] + A[j];
```

Solution

```

slli    x30, x29, 2    // x30 = j*4
add     x30, x10, x30   // x30 = &A[] + j*4 = &A[j]
lw      x30, 0(x30)     // x30 = A[j]
slli    x31, x28, 2    // x31 = i*4
add     x31, x10, x31   // x31 = &A[] + i*4 = &A[i]
lw      x31, 0(x31)     // x31 = A[i]
add     x30, x31, x30   // x30 = A[i] + A[j]
sw      x30, 32(x11)    // B[8] = A[i] + A[j]

```

Exercise 2.8

Translate the following RISC-V code to C. Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively.

```

addi    x30, x10, 4
addi    x31, x10, 0
sw      x31, 0(x30)
lw      x30, 0(x30)
add     x5, x30, x31

```

Solution

```

A[1] = &A[0];
f = A[1] + &A[0];    // f = 2*&A[0]

```

Exercise 2.9

For each RISC-V instruction in Exercise 2.8, show the value of the opcode (op), source register (rs1), and destination register (rd) fields. For the I-type instructions, show the value of the immediate field, and for the R-type instructions, show the value of the second source register (rs2). For non U- and UJ-type instructions, show the funct3 field, and for R-type and S-type instructions, also show the funct7 field.

Solution

Imm _{11:0}	rs1	funct3	rd	op	I-Type
000000001000_01010_000_11110_0010011				addi	x30, x10, 8

Imm _{11:0}	rs1	funct3	rd	op	I-Type
000000000000_01010_000_11111_0010011				addi	x31, x10, 0

Imm _{11:5}	rs2	rs1	funct3	Imm _{4:0}	op	S-Type
0000000_11111_11110_010_00000_0100011					sw	x31, 0(x30)

Imm _{11:0}	rs1	funct3	rd	op	I-Type
000000000000_11110_010_11110_0000011				lw	x30, 0(x30)

funct7	rs2	rs1	funct3	rd	op	R-Type
0000000_11111_11110_000_00101_0110011					add	x5, x30, x31

Exercise 2.10

Assume that registers x5 and x6 hold the values 0x8000000000000000 and 0xD000000000000000, respectively.

- What is the value of x30 for the following assembly code?
add x30, x5, x6
- Is the result in x30 the desired result, or has there been overflow?
- For the contents of registers x5 and x6 as specified above, what is the value of x30 for the following assembly code?
sub x30, x5, x6
- Is the result in x30 the desired result, or has there been overflow?
- For the contents of registers x5 and x6 as specified above, what is the value of x30 for the following assembly code?
add x30, x5, x6
add x30, x30, x5
- Is the result in x30 the desired result, or has there been overflow?

Solution

- 0x5000000000000000
- There were an overflow
- 0xB000000000000000
- Yes is the desired outcome
- 0xD000000000000000
- No, it had an overflow

Exercise 2.11

Assume that x5 holds the value 128.

- For the instruction add x30, x5, x6, what is the range(s) of values for x6 that would result in overflow?
- For the instruction sub x30, x5, x6, what is the range(s) of values for x6 that would result in overflow?
- For the instruction sub x30, x6, x5, what is the range(s) of values for x6 that would result in overflow?

Solution

1. If we assume that the length of the registers are 32-bits and because obey twos complement format then we take the twos complement of 0x80 and its 0xFFFFF80 and we remove the sign bit so it start to overflow from 0x7FFFFFF80 to 0x7FFFFFFF
2. Because we know that when we add 0x7FFFFFF80 do the sign flip then we take twos complement of that number which is 0x80000080 so the range of overflow is 0x8000007F to 0x80000000
3. Now we need to know the range of number which will overflow when add 0xFFFFF80, the range is from 0x80 to the most positive number 0x7FFFFFFF

Exercise 2.12

Provide the instruction type and assembly language instruction for the following binary value:

0000_0000_0001_0000_1000_0000_1011_0011

Solution

0000000_00001_00001_000_00001_0110011

First we check the opcode=0110011 and we know is a R-Type instruction. We follow with rd=00001 equivalent to x1, then funct3=000 so is and addition or subtraction, then rs1=00001, that means x1, then rs2= 00001 also means x1, and finally funct7=0000000 so is an addition.

add x1, x1, x1

Exercise 2.13

Provide the instruction type and hexadecimal representation of the following instruction:

sw x5, 32(x30)

Solution

0000001_00101_11110_010_00000_0100011	Format as S-Type instruction
0000_0010_0101_1111_0010_0000_0010_0011	Format bin to hex
0x025F2023	Hex format

Exercise 2.14

Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following RISC-V fields:

opcode=0x33, funct3=0x0, funct7=0x20, rs2=5, rs1=7, rd=6

Solution

The opcode indicate an R-Type instruction. The funct3 indicate add or sub. Funct7 indicate a subtraction.

sub x6, x7, x5

Exercise 2.15

Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following RISC-V fields:

opcode=0x3, funct3=0x2, rs1=27, rd=3, imm=0x4

Solution

The opcode indicate a load instruction (I-Type). Funct3 indicate a lw instruction.

lw x3, 4(x27)

Exercise 2.16

Assume that we would like to expand the RISC-V register file to 128 registers and expand the instruction set to contain four times as many instructions.

1. How would this affect the size of each of the bit fields in the R-type instructions?

2. How would this affect the size of each of the bit fields in the I-type instructions?
3. How could each of the two proposed changes decrease the size of a RISC-V assembly program? On the other hand, how could the proposed change increase the size of a RISC-V assembly program?

Solution

1. For 128 register we will need to expand from 5 bits to 7 bits so if we had rs1, rs2 and rd inside an R-type then we need 6 extra bits. For instructions can be tricky because it can be encoded to funct7 but if they are individual instructions with their own opcode and taking in count that RV32I had up to 8 different opcodes if we multiply by 4 we would need 2 bits more or nothing if share the same opcode.
2. Same story for I-type, we need 4 extra bits for rs1 and rd. For instructions we could need up to 2 extra bits but probably won't need because it can share many operations codification inside funct3.
3. It could decrease it because with more focused and variable extensive usage of variables we could create smaller programs. However it can increase and by a lot for context switching

Exercise 2.17

Assume the following register contents:

x5 = 0x00000000AAAAAAA, x6 = 0x1234567812345678

1. For the register values shown above, what is the value of x7 for the following sequence of instructions?

```
slli    x7, x5, 4
or      x7, x7, x6
```
2. For the register values shown above, what is the value of x7 for the following sequence of instructions?

```
slli    x7, x6, 4
```
3. For the register values shown above, what is the value of x7 for the following sequence of instructions?

```
srli    x7, x5, 3
andi    x7, x7, 0xFEF
```

Solution

1. Value of x7=0x1234FEFABABE5678

```
slli    x7, x5, 4      // x7=0x0000AAAAAAA0000
or      x7, x7, x6      // x7=0x1234FEFABABE5678
```
2. Value of x7=0x5678123456780000

```
slli    x7, x6, 4      // x7=0x5678123456780000
```
3. Value of x7=0x000000000000AAA

```
srli    x7, x5, 3      // x7=0x000000000000AAAA
andi    x7, x7, 0xFEF // x7=0x000000000000AAA
```

Exercise 2.18

Find the shortest sequence of RISC-V instructions that extracts bits 16 down to 11 from register x5 and uses the value of this field to replace bits 31 down to 26 in register x6 without changing the other bits of registers x5 or x6. (Be sure to test your code using x5 = 0 and x6 = 0xffffffff. Doing so may reveal a common oversight.)

Solution

```
srli    x27, x5, 11      // shift 16:11 to 5:0
andi    x27, x27, 0x3F    // turn 31:6 on 0s and keeping 5:0
slli    x27, x27, 26      // shift 5:0 to 31:26
lui     x28, 0xFC000      // ones complement of 0x03FFF
xori    x28, x28, 0xFFF    // 0xFC000000 XOR 0xFFFFFFFF = 0x03FFFFFF
and     x6, x6, x28      // turn 31:26 to 0s and keeping 25:0
or      x6, x6, x27      // Mix x27 31:26 bits with x28 25:0 bits
```

Exercise 2.19

Provide a minimal set of RISC-V instructions that may be used to implement the following pseudoinstruction:

```
not x5, x6    // bit-wise invert
```

Solution

```
xori    x5, x6, -1    // invert bits XOR with 0xFFFFFFFF
```

Exercise 2.20

For the following C statement, write a minimal sequence of RISC-V assembly instructions that performs the identical operation. Assume $x6 = A$, and $x17$ is the base address of C .

```
A = C[0] << 4;
```

Solution

```
lw      x6, 0(x17)
slli    x6, x6, 4
```

Exercise 2.21

Assume $x5$ holds the value $0x0000000001010000$. What is the value of $x6$ after the following instructions?

```
bge     x5, x0, ELSE
jal      x0, DONE
```

ELSE:

```
ori      x6, x0, 2
```

DONE:

Solution

$x6=2$

Exercise 2.22

Suppose the program counter (PC) is set to $0x20000000$.

1. What range of addresses can be reached using the RISC-V jump-and-link (jal) instruction? (In other words, what is the set of possible values for the PC after the jump instruction executes?)
2. What range of addresses can be reached using the RISC-V branch if equal (beq) instruction? (In other words, what is the set of possible values for the PC after the branch instruction executes?)

Solution

1. From $0x1FF00000$ to $0x200FFFFE$
2. From $0x1FFFF000$ to $0x20000FFE$

Exercise 2.23

Consider a proposed new instruction named `rpt`. This instruction combines a loop's condition check and counter decrement into a single instruction. For example `rpt x29`, loop would do the following:

```
if (x29 > 0) {
    x29 = x29 - 1;
    goto loop
}
```

1. If this instruction were to be added to the RISC-V instruction set, what is the most appropriate instruction format?
2. What is the shortest sequence of RISC-V instructions that performs the same operation?

Solution

1. This is a complex instruction and it won't fit well in any instruction format without modify the architecture workflow. The best approximation is an B type. This because is completely necessary to not increment PC value and perform a comparison.
2. The following:

```

LOOP:
    bge    x0, x29, DONE
    addi   x29, x20, -1
    jal    x0, LOOP
DONE:

```

Exercise 2.24

Consider the following RISC-V loop:

```

LOOP:
    beq    x6, x0, DONE
    addi   x6, x6, -1
    addi   x5, x5, 2
    jal    x0, LOOP

```

DONE:

1. Assume that the register x6 is initialized to the value 10. What is the final value in register x5 assuming the x5 is initially zero?
2. For the loop above, write the equivalent C code. Assume that the registers x5 and x6 are integers acc and i, respectively.
3. For the loop written in RISC-V assembly above, assume that the register x6 is initialized to the value N. How many RISC-V instructions are executed?
4. For the loop written in RISC-V assembly above, replace the instruction “beq x6, x0, DONE” with the instruction “blt x6, x0, DONE” and write the equivalent C code.

Solution

1. x5 = 20
2. Equivalent code:


```

i=10;
acc=0;
while(i!=0) {
    acc = acc + 2;
    i = i - 1;
}

```
3. 4*N+1
4. Replaced Line:


```

i=10;
acc=0;
while(i>=0) {
    acc = acc + 2;
    i = i - 1;
}

```

Exercise 2.25

Translate the following C code to RISC-V assembly code. Use a minimum number of instructions. Assume that the values of a, b, i, and j are in registers x5, x6, x7, and x29, respectively. Also, assume that register x10 holds the base address of the array D.

```

for(i=0; i<a; i++)
    for(j=0; j<b; j++)
        D[4*j] = i + j;

```

Solution

```

    add    x7, x0, x0        // i = 0
LOOP1:

```

```

    bge    x7, x5, DONE1    // if (i<a)
    add    x29, x0, x0      // j = 0

LOOP2:
    bge    x29, x6, DONE2    // if (j<b)

    add    x30, x7, x29      // x30 = i+j
    slli   x31, x29, 4       // x31 = 4*(4*j)
    add    x31, x31, x10     // x31 = &D[0] + 16 * j
    sw     x30, 0(x31)       // D[4*j] = i + j

    addi   x29, x29, 1       // j++
    jal    x0, LOOP2

DONE2:
    addi   x7, x7, 1         // i++
    jal    x0, LOOP1
DONE1:

```

Exercise 2.26

How many RISC-V instructions does it take to implement the C code from Exercise 2.25? If the variables *a* and *b* are initialized to 10 and 1 and all elements of *D* are initially 0, what is the total number of RISC-V instructions executed to complete the loop?

Solution

It takes 122 instructions. We need to start counting with the principal instruction “ $D[4*j] = i + j$ ” which repeats $4*(a*b)$ times. The code of the second loop takes $(3*b+1)*a$ instructions and the code of the first loop takes $4*a+1$ instructions. This makes $4*(10*1)+(3*1+1)*10+4*10+1+1=122$ instructions to execute.

Exercise 2.27

Translate the following loop into C. Assume that the C-level integer *i* is held in register *x6*, *x5* holds the C-level integer called *result*, and *x10* holds the base address of the integer *MemArray*.

```

    addi   x6, x0, 0
    addi   x29, x0, 100
LOOP:
    lw     x7, 0(x10)
    add    x5, x5, x7
    addi   x10, x10, 4
    addi   x6, x6, 1
    blt    x6, x29, LOOP

```

Solution

```

int i = 0;
int result, *MemArray;
do {
    result = result + *MemArray;
    MemArray = MemArray + 1;
    i++;
} while(i<100);

```

Exercise 2.28

Rewrite the loop from Exercise 2.27 to reduce the number of RISC-V instructions executed. Hint: Notice that variable `i` is used only for loop control.

Solution

```
    addi    x6, x0, 100
LOOP:
    lw      x7, 0(x10)
    add     x5, x5, x7
    addi    x10, x10, 4
    addi    x6, x6, -1
    bge     x6, x0, LOOP
```

Exercise 2.29

Implement the following C code in RISC-V assembly. Hint: Remember that the stack pointer must remain aligned on a multiple of 16.

```
int fib(int n) {
    if (n==0)
        return 0;
    else if (n==1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```

Solution

```
FIB:                                // int fib(int n)
    addi    x2, x2, -16              // save to stack
    sw      x1, 0(x2)
    sw      x18, 4(x2)
    sw      x19, 8(x2)

    addi    x28, x0, 0
    bne     x10, x28, IFELSE         // if(n==0)
    addi    x10, x0, 0               // return 0
    jal     x0, DONE

IFELSE:
    addi    x28, x0, 1
    bne     x10, x28, ELSE           // if(n==1)
    addi    x10, x0, 1               // return 1
    jal     x0, DONE

ELSE:
    add     x18, x0, x10              // x18 = n
    addi    x10, x18, -1
    jal     x1, FIB                  // fib(n-1)
    add     x19, x0, x10
    addi    x10, x18, -2
    jal     x1, FIB                  // fib(n-2)
    add     x10, x19, x10             // x10 = fib(n-1) + fib(n-2)

DONE:
    lw      x1, 0(x2)
```

```

lw    x18, 4(x2)
lw    x19, 8(x2)
addi  x2, x2, 16
jalr  x0, 0(x1)           // return fib(n)

```

Exercise 2.30

For each function call in Exercise 2.29, show the contents of the stack after the function call is made. Assume the stack pointer is originally at address 0x7ffffffc, and follow the register conventions as specified in Figure 2.11.

Solution

This is the initial state

```
0x7ffffffc: 0x0    <-x2<-x8
```

Now we will suppose that x1=0x2000 (caller PC address), x18=0x20, x19=0x30 and we pass x10=0x2

```
0x7ffffffc: 0x0    <-x8
```

```
0x7ffffff8: 0x0
```

```
0x7ffffff4: 0x30
```

```
0x7ffffff0: 0x20
```

```
0x7fffffec: 0x2000<-x2
```

Now let's suppose that our program run on 0x8000. The program will run first fib(1), because this instructions lay 14 instructions (32-bits) below then x1=0x8038, x18=0x2 and x19=0x30 (still) and x2 will subtract other 16 spaces.

```
0x7ffffffc: 0x0    <-x8
```

```
0x7ffffff8: 0x0
```

```
0x7ffffff4: 0x30
```

```
0x7ffffff0: 0x20
```

```
0x7fffffec: 0x2000
```

```
0x7fffffec: 0x0
```

```
0x7ffffe8: 0x0
```

```
0x7ffffe4: 0x30
```

```
0x7ffffe0: 0x2
```

```
0x7ffffdc: 0x8038<-x2
```

Then fib(1) will return the value of 1, that mean x10=0x1 and the stack will return to the original state. Finally the program calculate fib(0). So the stack will have the follow values x1=0x8044, x18=0x2 and x19=0x1

```
0x7ffffffc: 0x0    <-x8
```

```
0x7ffffff8: 0x0
```

```
0x7ffffff4: 0x30
```

```
0x7ffffff0: 0x20
```

```
0x7fffffec: 0x2000
```

```
0x7fffffec: 0x0
```

```
0x7ffffe8: 0x0
```

```
0x7ffffe4: 0x1
```

```
0x7ffffe0: 0x2
```

```
0x7ffffdc: 0x8044<-x2
```

Finally the program calculate fib(2)=fib(1)+fib(0) and it will restore the stack with x8 and x2 pointing to 0x7ffffffc

Exercise 2.31

Translate function f into RISC-V assembly language. Assume the function declaration for g is int g(int a, int b). The code for function f is as follows:

```

int f(int a, int b, int c, int d){
    return g(g(a,b), c+d);
}

```

```
}
```

Solution

F:

```
addi    x2, x2, -16
sw       x1, 0(x2)
sw       x18, 4(x2)

add      x18, x12, x13      // x18 = c+d
jal      x1, G              // x10 = g(a,b) x10 is already a and x11 is b
add      x11, x0, x18       // x11 = c+d
jal      x1, G              // x10 = g(g(a,b), c+d)

lw       x1, 0(x2)
lw       x18, 4(x2)
addi     x2, x2, 16
jalr     x0, 0(x1)          // return g(g(a,b), c+d)
```

Exercise 2.32

Can we use the tail-call optimization in this function? If no, explain why not. If yes, what is the difference in the number of executed instructions in f with and without the optimization?

Solution

If we do a tail-call optimization we can restore stack in the last function because it won't affect if you lose the stack of function f if you return the value of it so then we can save just one instruction.

F:

```
addi     x2, x2, -16
sw        x1, 0(x2)
sw        x18, 4(x2)

add      x18, x12, x13      // x18 = c+d
jal      x1, G              // x10 = g(a,b) x10 is already a and x11 is b
add      x11, x0, x18       // x11 = c+d

lw       x1, 0(x2)
lw       x18, 4(x2)
addi     x2, x2, 16
jal      x0, G              // return g(g(a,b), c+d)
```

Exercise 2.33

Right before your function f from Exercise 2.31 returns, what do we know about contents of registers x10-x14, x8, x1, and sp? Keep in mind that we know what the entire function f looks like, but for function g we only know its declaration.

Solution

The registers from x10 to x14 are the parameters a, b, c and d, x8 is the frame pointer that cannot know their exact value, x1 will have the return address of the program that calls f and sp cannot be known but we know its restricted by x8.

Exercise 2.34

Write a program in RISC-V assembly to convert an ASCII string containing a positive or negative integer decimal string to an integer. Your program should expect register x10 to hold the address of a null-terminated

string containing an optional “+” or “-” followed by some combination of the digits 0 through 9. Your program should compute the integer value equivalent to this string of digits, then place the number in register x10. If a non-digit character appears anywhere in the string, your program should stop with the value -1 in register x10. For example, if register x10 points to a sequence of three bytes 50_{ten}, 52_{ten}, 0_{ten} (the null-terminated string “24”), then when the program stops, register x10 should contain the value 24_{ten}. The RISC-V mul instruction takes two registers as input. There is no “mul” instruction. Thus, just store the constant 10 in a register.

Solution

```

STR_NUM:
    addi    x2, x2, -16           // Save to stack ra, s1 and s2 and s3
    sw      x1, 0(x2)
    sw      x9, 4(x2)
    sw      x18, 8(x2)
    sw      x19, 12(x2)

    addi    x5, x0, 0x30         // ASCII Numbers position
    addi    x6, x0, 0xA         // Ammount of numbers (10)
    addi    x19, x0, 0          // accum = 0
    lbu     x18, 0(x10)         // readChar = *ptrStr load first character
    addi    x9, x0, -1          // first negative to make positive as default if no sign is specified
    addi    x7, x0, 0x2D        // if (readChar == "-") sign = -1
    beq     x18, x5, NEXT
    addi    x9, x0, 1
    addi    x7, x0, 0x2B        // if (readChar == "+") sign = 1
    beq     x18, x5, NEXT
    jal     LOOP

NEXT:
    addi    x10, x10, 1         // ptrStr++

LOOP:
    lbu     x18, 0(x10)         // readChar = *ptrStr load first character
    beq     x18, x0, SUCCESS    // if (readChar != "\0")
    sub     x18, x18, x5        // x18 = string to number
    blt     x18, x0, ERROR
    bge     x18, x6, ERROR      // if (readChar >= 0 || readChar <= 9)
    mul     x19, x19, x6        // accum *= 10
    add     x19, x19, x18       // accum += readChar
    addi    x10, x10, 1         // ptrStr++
    jal     LOOP

ERROR:
    addi    x10, x0, -1         // return -1
    jal     END

SUCCESS:
    mul     x10, x19, x9        // return accum*sign

END:
    lw      x1, 0(x2)
    lw      x9, 4(x2)
    lw      x18, 8(x2)
    lw      x19, 12(x2)
    addi    x2, x2, 16          // Restore stack
    jalr    x0, 0(x1)

```

Exercise 2.35

Consider the following code:

```
lb    x6, 0(x7)
sw    x6, 8(x7)
```

Assume that the register x7 contains the address 0×10000000 and the data at address is $0 \times 1122334455667788$.

1. What value is stored in 0×10000008 on a big-endian machine?
2. What value is stored in 0×10000008 on a little-endian machine?

Solution

1. On big endian, the byte loaded will be $0x11$ and the sign will extend (positive so fill with 0s). Then when we store we store the value of $0x00000011$ at 0×10000008 resulting with $0 \times 1100000055667788$ in memory
2. On little endian, the byte loaded will be $0x88$ and the sign will extend (negative). Then the store will write $0xFFFFFFFF88$ to address 0×10000008 resulting with $0 \times 11223344FFFFFFFF88$ in memory

Exercise 2.36

Write the RISC-V assembly code that creates the 32-bit constant $0x12345678$ and stores that value to register x10.

Solution

```
lui    x9, 0x12345
addi   x10, 0x678    // no sign bit
or     x10, x10, x9
```

Exercise 2.37

Write the RISC-V assembly code to implement the following C code as an atomic “set max” operation using the lr.d/sc.d instructions. Here, the argument shvar contains the address of a shared variable which should be replaced by x if x is greater than the value it points to:

```
void setmax(int* shvar, int x) {
    // Begin critical section
    if (x > *shvar)
        *shvar = x;
    // End critical section}
}
```

Solution

setmax:

AGAIN:

```
lr.w    x5, (x10)
bge     x5, x11, DONE
sc.w    x6, x5, (x10)
bne     x6, x0, AGAIN
```

DONE:

```
jalr    x0, 0(x1)
```

Exercise 2.38

Using your code from Exercise 2.37 as an example, explain what happens when two processors begin to execute this critical section at the same time, assuming that each processor executes exactly one instruction per cycle.

Solution

What is expected. When the lr.w is executed this memory space is locked and any memory operation that other processor attempt to do is going to be useless. When sc.w executes it frees this memory location, however it return a status of the operation if it is 0 then it successfully wrote the value

Exercise 2.39

Assume for a given processor the CPI of arithmetic instructions is 1, the CPI of load/store instructions is 10, and the CPI of branch instructions is 3. Assume a program has the following instruction breakdowns: 500 million arithmetic instructions, 300 million load/store instructions, 100 million branch instructions.

1. Suppose that new, more powerful arithmetic instructions are added to the instruction set. On average, through the use of these more powerful arithmetic instructions, we can reduce the number of arithmetic instructions needed to execute a program by 25%, while increasing the clock cycle time by only 10%. Is this a good design choice? Why?
2. Suppose that we find a way to double the performance of arithmetic instructions. What is the overall speedup of our machine? What if we find a way to improve the performance of arithmetic instructions by 10 times?

Solution

$$1. \frac{ET_{BEF}}{ET_{AFT}} = \frac{(1*500*10^9 + 10*300*10^9 + 3*100*10^9)*CC}{(0.75*1*500*10^9 + 10*300*10^9 + 3*100*10^9)*1.1*CC} = \frac{3800*10^9 CC}{4042.5*10^9 CC} = 0.94$$

No. It is not a good design choice. In total this slowed clock cycle makes worse performance even with this instructions reductions

$$2. \frac{ET_{BEF}}{ET_{AFT}} = \frac{(1*500*10^9 + 10*300*10^9 + 3*100*10^9)*CC}{(0.5*1*500*10^9 + 10*300*10^9 + 3*100*10^9)*CC} = 1.0704$$

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{(1*500*10^9 + 10*300*10^9 + 3*100*10^9)*CC}{(0.1*1*500*10^9 + 10*300*10^9 + 3*100*10^9)*CC} = 1.1343$$

Exercise 2.40

Assume that for a given program 70% of the executed instructions are arithmetic, 10% are load/store, and 20% are branch.

1. Given this instruction mix and the assumption that an arithmetic instruction requires two cycles, a load/store instruction takes six cycles, and a branch instruction takes three cycles, find the average CPI.
2. For a 25% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?
3. For a 50% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?

Solution

$$1. CPI_{AVG} = 0.7*2 + 0.1*6 + 0.2*3 = 2.6$$

$$CPI_{IMPRVD} = 0.75*2.6 = 0.7*CPI_A + 0.1*6 + 0.2*3$$

$$2. CPI_A = \frac{0.75*2.6 - 0.1*6 - 0.2*3}{0.7} = 1.07142$$

$$CPI_{IMPRVD} = 0.75*2.6 = 0.7*CPI_A + 0.1*6 + 0.2*3$$

$$3. CPI_A = \frac{0.5*2.6 - 0.1*6 - 0.2*3}{0.7} = 0.14285$$

Exercise 2.41

Suppose the RISC-V ISA included a scaled offset addressing mode similar to the x86 one described in Section 2.19 (Figure 2.39). Describe how you would use scaled offset loads to further reduce the number of assembly instructions needed to carry out the function given in Exercise 2.4.

Solution

It can help to reduce shift instruction to align to the format type of the variable in memory and because of it we can treat any pointer the same no matter the type.

f, g, h, i, j, A, B => x5, x6, x7, x28, x29, x10, x11

```
lwr    x28, x5(x10), 2    // x28 = A[f]
addi   x5, x5, 1          // f++
lwr    x29, x5(x10), 2    // x29 = A[f+1]
add    x30, x28, x29      // x30 = A[f+1]+A[f]
lwr    x30, x6(x11), 2    // B[g] = A[f+1]+A[f]
```

Exercise 2.42

Suppose the RISC-V ISA included a scaled offset addressing mode similar to the x86 one described in Section 2.19 (Figure 2.39). Describe how you would use scaled offset loads to further reduce the number of assembly instructions needed to implement the C code given in Exercise 2.7

Solution

It can help because we can point to an array with an variable right there and this kind of instructions helps a lot to reduce instructions.

f, g, h, i, j, A, B => x5, x6, x7, x28, x29, x10, x11

```
lwr    x30, x28, (x10), 2  // x30 = A[i]
lwr    x31, x29, (x10), 2  // x30 = A[j]
add    x30, x31, x30       // x30 = A[i] + A[j]
sw     x30, 32(x11)        // B[8] = A[i] + A[j]
```

Chapter 3

Exercise 3.1

What is $5ED4 - 07A4$ when these values represent unsigned 16-bit hexadecimal numbers? The result should be written in hexadecimal. Show your work.

Solution

5ED4	0101 1110 1101 0100	0101 1110 1101 0100
- 07A4	0000 0111 1010 0100	1111 1000 0101 1100
		1 0101 0111 0011 0000 = 5730

Exercise 3.2

What is $5ED4 - 07A4$ when these values represent signed 16-bit hexadecimal numbers stored in sign-magnitude format? The result should be written in hexadecimal. Show your work.

Solution

5ED4	0 101 1110 1101 0100	0 101 1110 1101 0100
- 07A4	1 000 0111 1010 0100	0 111 1000 0101 1100
		0 101 0111 0011 0000 = 5730

Exercise 3.3

Convert 5ED4 into a binary number. What makes base 16 (hexadecimal) an attractive numbering system for representing values in computers?

Solution

Because hexadecimal is a power of two become really easy to transform to binary
5ED4 0101 1110 1101 0100

Exercise 3.4

What is $4365 - 3412$ when these values represent unsigned 12-bit octal numbers? The result should be written in octal. Show your work.

Solution

4365	100 011 110 101	100 011 110 101
- 3412	011 100 001 010	100 011 110 110
		1 000 111 101 011 = 0753

Exercise 3.5

What is $4365 - 3412$ when these values represent signed 12-bit octal numbers stored in sign-magnitude format? The result should be written in octal. Show your work.

Solution

4365	1 00 011 110 101
- 3412	1 11 100 001 010
	1 11 111 111 111 = 7777 (-3777)

Exercise 3.6

Assume 185 and 122 are unsigned 8-bit decimal integers. Calculate $185 - 122$. Is there overflow, underflow, or neither?

Solution

185	1011	1001	1011	1001
- 122	0111	1010	1000	0110
	1	0011	1111	= 63

Exercise 3.7

Assume 185 and 122 are signed 8-bit decimal integers stored in sign-magnitude format. Calculate 185+122. Is there overflow, underflow, or neither?

Solution

185	1	011	1001	0	100	0111	(-57)
+122	0	111	1010	0	111	1010	(122)
				0	100	0001	= 65

Exercise 3.8

Assume 185 and 122 are signed 8-bit decimal integers stored in sign-magnitude format. Calculate 185-122. Is there overflow, underflow, or neither?

Solution

185	1	011	1001	(-57)
- 122	1	111	1010	(-122)
	1	011	0011	= -179 (-51)

It underflow

Exercise 3.9

Assume 151 and 214 are signed 8-bit decimal integers stored in two's complement format. Calculate 151+214 using saturating arithmetic. The result should be written in decimal. Show your work.

Solution

151	1001	0111	(-105)
+214	1101	0110	(-42)
	1	0110	1101 = -147 (109)

It underflow

Exercise 3.10

Assume 151 and 214 are signed 8-bit decimal integers stored in two's complement format. Calculate 151 - 214 using saturating arithmetic. The result should be written in decimal. Show your work.

Solution

151	1001	0111	1001	0111	(-105)
- 214	1101	0110	0010	1010	(42)
			1100	0001	= -63

Exercise 3.11

Assume 151 and 214 are unsigned 8-bit integers. Calculate 151+214 using saturating arithmetic. The result should be written in decimal. Show your work.

Solution

151	1001	0111
+214	1101	0110
	1	0110 1101 = 109 (365)

Exercise 3.12

Using a table similar to that shown in Figure 3.6, calculate the product of the octal unsigned 6-bit integers 62 and 12 using the hardware described in Figure 3.3. You should show the contents of each register on each step.

Solution

Iteration	Step	Multiplier	Multiplicand	Product
1	Initial Value	001 01 <u>0</u>	000 000 110 010	000 000 000 000
2	No operation	001 01 <u>0</u>	000 000 110 010	000 000 000 000
	Shift Left Multiplicand	001 01 <u>0</u>	000 001 100 100	000 000 000 000
	Shift Right Multiplier	000 10<u>1</u>	000 001 100 100	000 000 000 000
3	Prod+=Multiplication	000 10 <u>1</u>	000 001 100 100	000 001 100 100
	Shift Left Multiplicand	000 10 <u>1</u>	000 011 001 000	000 001 100 100
	Shift Right Multiplier	000 01<u>0</u>	000 011 001 000	000 001 100 100
4	No operation	000 01 <u>0</u>	000 011 001 000	000 001 100 100
	Shift Left Multiplicand	000 01 <u>0</u>	000 110 010 000	000 001 100 100
	Shift Right Multiplier	000 00<u>1</u>	000 110 010 000	000 001 100 100
5	Prod+=Multiplication	000 00 <u>1</u>	000 110 010 000	000 111 110 100
	Shift Left Multiplicand	000 00 <u>1</u>	001 100 100 000	000 111 110 100
	Shift Right Multiplier	000 00<u>0</u>	001 100 100 000	000 111 110 100
6	No operation	000 00 <u>0</u>	001 100 100 000	000 111 110 100
	Shift Left Multiplicand	000 00 <u>0</u>	011 001 000 000	000 111 110 100
	Shift Right Multiplier	000 00<u>0</u>	011 001 000 000	000 111 110 100
7	No operation	000 00 <u>0</u>	011 001 000 000	000 111 110 100
	Shift Left Multiplicand	000 00 <u>0</u>	110 010 000 000	000 111 110 100
	Shift Right Multiplier	000 00<u>0</u>	110 010 000 000	000 111 110 100

Exercise 3.13

Using a table similar to that shown in Figure 3.6, calculate the product of the octal unsigned 6-bit integers 62 and 12 using the hardware described in Figure 3.5. You should show the contents of each register on each step.

Solution

Iteration	Step	Multiplicand	Product
1	Initial Value	110 010	000 000 001 01 <u>0</u>
2	No operation	110 010	000 000 001 010
	Shift Right Product	110 010	000 000 000 10 <u>1</u>
3	Prod+=Multiplicand<<6	110 010	110 010 000 101
	Shift Right Product	110 010	011 001 000 01 <u>0</u>
4	No operation	110 010	011 001 000 010

	Shift Right Product	110 010	001 100 100 00 <u>1</u>
5	Prod+=Multiplicand<<6	110 010	111 110 100 001
	Shift Right Product	110 010	011 111 010 00 <u>0</u>
6	No operation	110 010	011 111 010 000
	Shift Right Product	110 010	001 111 101 00 <u>0</u>
7	No operation	110 010	001 111 101 000
	Shift Right Product	110 010	000 111 110 100

Exercise 3.14

Calculate the time necessary to perform a multiply using the approach given in Figures 3.3 and 3.4 if an integer is 8 bits wide and each step of the operation takes four time units. Assume that in step 1a an addition is always performed—either the multiplicand will be added, or a zero will be. Also assume that the registers have already been initialized (you are just counting how long it takes to do the multiplication loop itself). If this is being done in hardware, the shifts of the multiplicand and multiplier can be done simultaneously. If this is being done in software, they will have to be done one after the other. Solve for each case.

Solution

$$SW_{time} = 4 * 8 * 4 tu = 160 tu$$

$$HW_{time} = 3 * 8 * 4 tu = 96 tu$$

Exercise 3.15

Calculate the time necessary to perform a multiply using the approach described in the text (31 adders stacked vertically) if an integer is 8 bits wide and an adder takes four time units.

Solution

$$Mult_{time} = 7 * 4 tu = 28 tu$$

Exercise 3.16

Calculate the time necessary to perform a multiply using the approach given in Figure 3.7 if an integer is 8 bits wide and an adder takes four time units.

Solution

$$Mult_{time} = \log_2(8) * 4 tu = 12 tu$$

Exercise 3.17

As discussed in the text, one possible performance enhancement is to do a shift and add instead of an actual multiplication. Since 9×6 , for example, can be written $(2 \times 2 \times 2 + 1) \times 6$, we can calculate 9×6 by shifting 6 to the left three times and then adding 6 to that result. Show the best way to calculate $0x33 \times 0x55$ using shifts and adds/subtracts. Assume both inputs are 8-bit unsigned integers.

Solution

$$55_{16} * 33_{16} = 55_{16} * (2^5 + 2^4 + 2^1 + 2^0) = (55_{16} \ll 5) + (55_{16} \ll 4) + (55_{16} \ll 1) + (55_{16})$$

$$55_{16} * 33_{16} = AA0_{16} + 550_{16} + AA_{16} + 55_{16} = 10EF_{16}$$

Exercise 3.18

Using a table similar to that shown in Figure 3.10, calculate 74 divided by 21 using the hardware described in Figure 3.8. You should show the contents of each register on each step. Assume both inputs are unsigned 6-bit octal integers.

Solution

Iteration	Step	Quotient	Divisor	Remainder
1	Initial Value	000 000	010 001 000 000	000 000 111 100
2	Rem-=Div	000 000	010 001 000 000	101 111 111 100
	Rem<0; +Div; Shift left Q; Q ₀ =0	000 000	010 001 000 000	000 000 111 100
	Shift Div Right	000 000	001 000 100 000	000 000 111 100
3	Rem-=Div	000 000	001 000 100 000	111 000 011 100
	Rem<0; +Div; Shift left Q; Q ₀ =0	000 000	001 000 100 000	000 000 111 100
	Shift Div Right	000 000	000 100 010 000	000 000 111 100
4	Rem-=Div	000 000	000 100 010 000	111 100 101 100
	Rem<0; +Div; Shift left Q; Q ₀ =0	000 000	000 100 010 000	000 000 111 100
	Shift Div Right	000 000	000 010 001 000	000 000 111 100
5	Rem-=Div	000 000	000 010 001 000	111 110 110 100
	Rem<0; +Div; Shift left Q; Q ₀ =0	000 000	000 010 001 000	000 000 111 100
	Shift Div Right	000 000	000 001 000 100	000 000 111 100
6	Rem-=Div	000 000	000 001 000 100	111 111 111 000
	Rem<0; +Div; Shift left Q; Q ₀ =0	000 000	000 001 000 100	000 000 111 100
	Shift Div Right	000 000	000 000 100 010	000 000 111 100
7	Rem-=Div	000 000	000 000 100 010	000 000 011 010
	Rem<0; +Div; Shift left Q; Q ₀ =1	000 001	000 000 100 010	000 000 011 010
	Shift Div Right	000 001	000 000 010 001	000 000 011 010
8	Rem-=Div	000 001	000 000 010 001	000 000 001 001
	Rem<0; +Div; Shift left Q; Q ₀ =1	000 011	000 000 010 001	000 000 001 001
	Shift Div Right	000 011	000 000 001 000	000 000 001 001

Exercise 3.19

Using a table similar to that shown in Figure 3.10, calculate 74 divided by 21 using the hardware described in Figure 3.11. You should show the contents of each register on each step. Assume A and B are unsigned 6-bit integers. This algorithm requires a slightly different approach than that shown in Figure 3.9. You will want to think hard about this, do an experiment or two, or else go to the web to figure out how to make this work correctly. (Hint: one possible solution involves using the fact that Figure 3.11 implies the remainder register can be shifted either direction.)

Solution

Iteration	Step	Divisor	Remainder/Quot
-----------	------	---------	----------------

1	Initial Value	010 001	000 000 111 100
2	Rem<<1	010 001	000 001 111 000
	Rem-=Div	010 001	110 000 111 000
	Rem<0; +Div; Q ₀ =0	010 001	000 001 111 00 <u>0</u>
3	Rem<<1	010 001	000 011 110 000
	Rem-=Div	010 001	110 010 110 000
	Rem<0; +Div; Q ₀ =0	010 001	000 011 110 00 <u>0</u>
4	Rem<<1	010 001	000 111 100 000
	Rem-=Div	010 001	110 110 100 000
	Rem<0; +Div; Q ₀ =0	010 001	000 111 100 00 <u>0</u>
5	Rem<<1	010 001	001 111 000 000
	Rem-=Div	010 001	111 110 000 000
	Rem<0; +Div; Q ₀ =0	010 001	001 111 000 00 <u>0</u>
6	Rem<<1	010 001	011 110 000 000
	Rem-=Div	010 001	001 101 000 000
	Rem>0; Q ₀ =1	010 001	001 101 000 00 <u>1</u>
7	Rem<<1	010 001	011 010 000 010
	Rem-=Div	010 001	001 001 000 010
	Rem>0; Q ₀ =1	010 001	001 001 000 01 <u>1</u>

Exercise 3.20

What decimal number does the bit pattern $0 \times 0C000000$ represent if it is a two's complement integer? An unsigned integer?

Solution

$$0C\ 000000_{16} = 12 * 16^6 = 201326592$$

Exercise 3.21

If the bit pattern $0 \times 0000006F$ is placed into the Instruction Register, what RISC-V instruction will be executed?

Solution

jal x0, 0

Exercise 3.22

What decimal number does the bit pattern $0x0C000000$ represent if it is a floating point number? Use the IEEE 754 standard.

Solution

0 00011000 000000000000000000000000

$Sign = +$

$Exponent = 24 - 127 = -103$

$Mantissa = 1.0$

$0 \times 0C\ 000000 = 1 * 2^{-103}$

Exercise 3.23

Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 single precision format.

Solution

$63.25_{10} = 111111.01_2$

$Sign = +$

$Exponent = 5 = 132 - 127$

$Mantissa = 1.1111101$

$63.25_{10} = 0\ 10000100\ 1111101...0_2 = 0 \times 427\ D\ 0000$

Exercise 3.24

Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 double precision format.

Solution

$63.25_{10} = 111111.01_2$

$Sign = +$

$Exponent = 5 = 1028 - 1023$

$Mantissa = 1.1111101$

$63.25_{10} = 0\ 10000000100\ 1111101...0_2 = 0 \times 404\ FA\ 000000000000$

Exercise 3.25

Write down the binary representation of the decimal number 63.25 assuming it was stored using the single precision IBM format (base 16, instead of base 2, with 7 bits of exponent).

Solution

$63.25_{10} = 3F.4 * 16^0 = 3.F\ 4 * 16^{-1}$

$Sign = +$

$Exponent = 1 = 64 - 63$

$Mantissa = 3.F\ 4$

$63.25_{10} = 0\ 1000000\ 001111110100...0_2 = 0 \times 403\ F\ 4000$

Exercise 3.26

Write down the binary bit pattern to represent -1.5625×10^{-1} assuming a format similar to that employed by the DEC PDP-8 (the leftmost 12 bits are the exponent stored as a two's complement number, and the rightmost 24 bits are the fraction stored as a two's complement number). No hidden 1 is used. Comment on how the range and accuracy of this 36-bit pattern compares to the single and double precision IEEE 754 standards.

Solution

$-0.15625 = -(1/8 + 1/32) = -(0.00101_2) = -1.01 * 2^{-3}$

$Exponent = -3 = 111111111101_2$

$Mantissa = -1.01 = 0.11...0$

$-1.5625 * 10^{-1} = 111111111101\ 0110...0 = 0 \times FFD\ 600000$

Exercise 3.27

IEEE 754-2008 contains a half precision that is only 16 bits wide. The leftmost bit is still the sign bit, the exponent is 5 bits wide and has a bias of 15, and the mantissa is 10 bits long. A hidden 1 is assumed. Write down the bit pattern to represent -1.5625×10^{-1} assuming a version of this format, which uses an excess-16 format to store the exponent. Comment on how the range and accuracy of this 16-bit floating point format compares to the single precision IEEE 754 standard.

Solution

$$-0.15625 = -0.00101 = -1.01 \times 2^{-3}$$

$$\text{Sign} = -$$

$$\text{Exponent} = -3 = 12 - 15$$

$$\text{Mantissa} = 1.01$$

$$-1.5625 \times 10^{-1} = 1 \ 01100 \ 01...0 = 0xB100$$

Exercise 3.28

The Hewlett-Packard 2114, 2115, and 2116 used a format with the leftmost 16 bits being the fraction stored in two's complement format, followed by another 16-bit field which had the leftmost 8 bits as an extension of the fraction (making the fraction 24 bits long), and the rightmost 8 bits representing the exponent. However, in an interesting twist, the exponent was stored in sign-magnitude format with the sign bit on the far right! Write down the bit pattern to represent -1.5625×10^{-1} assuming this format. No hidden 1 is used. Comment on how the range and accuracy of this 32-bit pattern compares to the single precision IEEE 754 standard.

Solution

With this description we know that mantissa is denoted in two's complement however in order to work then the first bit need to has a value of -1. Then the mantissa can represent numbers between a number close to 1 and -1 ($1 > N \geq -1$). So then the exponent has to compensate this and actually has 2 bits before the point. The exponent can represent numbers between 127 and -127 ($127 \geq N \geq -127$).

$$-0.15625 = -(2^{-3} + 2^{-5}) = -1.01 \times 2^{-3}$$

$$\text{Mantissa} = -101_2 = 1011_2$$

$$\text{Exponent} = -2$$

$$\text{Sign}_{\text{exponent}} = 1$$

$$(-1 + 2^{-2} + 2^{-3}) \times 2^{-2} = -0.15625$$

$$-0.15625 = 1011 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0101 = B0000005$$

Exercise 3.29

Calculate the sum of 2.6125×10^1 and $4.150390625 \times 10^{-1}$ by hand, assuming A and B are stored in the 16-bit half precision described in Exercise 3.27. Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps.

Solution

$$2.6125 \times 10^1 = 2^4 + 2^3 + 2^1 + 2^{-3} = 11010.001_2 = 1.1010001 \times 2^4$$

$$4.150390625 \times 10^{-1} = 2^{-2} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-10} = 0.0110101001_2$$

$$= 1.10101001 \times 2^{-2}$$

$$4.150390625 \times 10^{-1} = 2^{-2} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-10} = 0.0110101001$$

$$= 1.10101001 \times 2^{-2}$$

	M	GR	S
+	1.1010001000	00	
	0.0000011010	10	01
	<u>1.1010100010</u>	10	01

$$1.1010100011 \times 2^4 = 26.546875$$

↳ 0.10011.1010100011

↳ 0x4EA3

Exercise 3.30

Calculate the product of -8.0546875×10^0 and $-1.79931640625 \times 10^{-1}$ by hand, assuming A and B are stored in the 16-bit half-precision format described in Exercise 3.27. Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps; however, as is done in the example in the text, you can do the multiplication in human-readable format instead of using the techniques described in Exercises 3.12 through 3.14. Indicate if there is overflow or underflow. Write your answer in both the 16-bit floating-point format described in Exercise 3.27 and also as a decimal number. How accurate is your result? How does it compare to the number you get if you do the multiplication on a calculator?

Solution

$$-8.0546875 \times 10^0 = -(2^3 + 2^{-5} + 2^{-6} + 2^{-7}) = -1000.0000111$$

$$= -1.0000000111 \times 2^3$$

$$-1.77931640625 \times 10^{-1} = -(2^{-3} + 2^{-5} + 2^{-6} + 2^{-7} + 2^{-12})$$

$$= -0.001011100001 = 1.011100001 \times 2^{-3}$$

$$\begin{array}{r}
 -1.0000000111 \\
 \times -1.0111000010 \\
 \hline
 000000000000 \\
 10000000111 \\
 000000000000 \\
 000000000000 \\
 000000000000 \\
 000000000000 \\
 100000000111 \\
 100000000111 \\
 100000000111 \\
 000000000000 \\
 100000000111 \\
 \hline
 +1.0111001100000100110
 \end{array}$$

$$1.0111001100 \times 2^0 = 1.44921875$$

$$\hookrightarrow 0.01111 \cdot 0111001100$$

↳ Ox 3DAA

Exercise 3.31

Calculate by hand 8.625×10^1 divided by -4.875×10^0 . Show all the steps necessary to achieve your answer. Assume there is a guard, a round bit, and a sticky bit, and use them if necessary. Write the final answer in both the 16-bit floating-point format described in Exercise 3.27 and in decimal and compare the decimal result to that which you get if you use a calculator.

Solution

$$8.625 \times 10^1 = 2^6 + 2^4 + 2^2 + 2^1 + 2^{-2} = 1010110.01 = 1.01011001 \times 2^6$$

$$-4.875 \times 10^0 = -(2^2 + 2^{-1} + 2^{-2} + 2^{-3}) = -100.111 = -1.00111 \times 2^2$$

	M	GR	S
	00001.00011011000100111010		
-1.00111	1.01011001		
	1.00111		
	0001000010		
	100111		
	0110110		
	100111		
	00111100		
	100111		
	101010		
	100111		
	0000110000		
	100111		
	001001000		
	100111		
	1000010		
	100111		
	00110110		
	100111		
	00111100		
	100111		
	10101		

$-1.0001101100 \times 2^4 = -17.6875$
 $\hookrightarrow 1.100110001101100$
 $\hookrightarrow 0xCC6C$

Exercise 3.32

Calculate $(3.984375 \times 10^{-1} + 3.4375 \times 10^{-1}) + 1.771 \times 10^3$ by hand, assuming each of the values is stored in the 16-bit half-precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating-point format and in decimal.

Solution

$$3.984375 \times 10^{-1} = 2^{-2} + 2^{-3} + 2^{-6} + 2^{-7} = 0.0110011 = 1.10011 \times 2^{-2}$$

$$3.4375 \times 10^{-1} = 2^{-2} + 2^{-4} + 2^{-5} = 0.01011 = 1.011 \times 2^{-2}$$

$$1.771 \times 10^3 = 2^{10} + 2^9 + 2^7 + 2^6 + 2^5 + 2^3 + 2^1 + 2^0 = 1101101011 = 1.101101011 \times 2^{10}$$

	M	GR	
1.1001100000	00	00	
+ 1.0110000000	00	00	
10.1111100000	00	00	
1.011111			$\times 2^{-1}$

	M	GR	S
1.1011101011	00	000000	
+ 0.0000000000	10	111111	
1.1011101011	10	111111	
1.1011101100			$\times 2^{10} = 1772$
$\hookrightarrow 0.11001 \cdot 1011101100$			
$\hookrightarrow 0x66EC$			

Exercise 3.33

Calculate $3.984375 \times 10^{-1} + (3.4375 \times 10^{-1} + 1.771 \times 10^3)$ by hand, assuming each of the values is stored in the 16-bit half precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating-point format and in decimal.

Solution

$$3.984375 \times 10^{-1} = 2^{-2} + 2^{-3} + 2^{-6} + 2^{-7} = 0.0110011 = 1.10011 \times 2^{-2}$$

$$3.4375 \times 10^{-1} = 2^{-2} + 2^{-4} + 2^{-5} = 0.01011 = 1.011 \times 2^{-2}$$

$$1.771 \times 10^3 = 2^{10} + 2^9 + 2^7 + 2^6 + 2^5 + 2^3 + 2^1 + 2^0 = 1101101011 = 1.101101011 \times 2^{10}$$

	M	GR	S
1.1011101011	00	000	
+ 0.0000000000	01	011	
1.1011101011	01	011	
1.1011101011			$\times 2^{10}$

	M	GR	S
1.1011101011	00	000000	
+ 0.0000000000	01	10011	
1.1011101011	01	10011	
1.1011101011			$\times 2^{10} = 1771$
$\hookrightarrow 0.11001 \cdot 1011101011$			
$\hookrightarrow 0x66EB$			

Exercise 3.34

Based on your answers to Exercises 3.32 and 3.33, does $(3.984375 \times 10^{-1} + 3.4375 \times 10^{-1}) + 1.771 \times 10^3 = 3.984375 \times 10^{-1} + (3.4375 \times 10^{-1} + 1.771 \times 10^3)$?

Solution

No it isn't because the precision is lost on the second operation and lost one unit

Exercise 3.35

Calculate $(3.41796875 \times 10^{-3} \times 6.34765625 \times 10^{-3}) \times 1.05625 \times 10^2$ by hand, assuming each of the values is stored in the 16-bit half-precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

Solution

$$\begin{aligned} 3.41796875 \times 10^{-3} &= 2^{-9} + 2^{-10} + 2^{-11} = 1.11 \times 2^{-9} \\ 6.34765625 \times 10^{-3} &= 2^{-8} + 2^{-9} + 2^{-11} = 1.101 \times 2^{-8} \\ 1.05625 \times 10^2 &= 2^6 + 2^5 + 2^3 + 2^0 + 2^{-1} + 2^{-3} = 1.101001101 \times 2^6 \end{aligned}$$

$$\begin{array}{r} \\ \times 1.1010000000 \\ \hline + 1.1100000000 \\ + 1.1100000000 \\ + 1.1100000000 \\ \hline 10.11011000000000 \\ \hline \text{GRS} \\ 1.0110110000 \times 2^{16} \rightarrow \text{Underflow} \end{array}$$

Exercise 3.36

Calculate $3.41796875 \times 10^{-3} \times (6.34765625 \times 10^{-3} \times 1.05625 \times 10^2)$ by hand, assuming each of the values is stored in the 16-bit half precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating-point format and in decimal.

Solution

$$\begin{aligned} 3.41796875 \times 10^{-3} &= 2^{-9} + 2^{-10} + 2^{-11} = 1.11 \times 2^{-9} \\ 6.34765625 \times 10^{-3} &= 2^{-8} + 2^{-9} + 2^{-11} = 1.101 \times 2^{-8} \\ 1.05625 \times 10^2 &= 2^6 + 2^5 + 2^3 + 2^0 + 2^{-1} + 2^{-3} = 1.101001101 \times 2^6 \end{aligned}$$

$$\begin{array}{r} \\ \times 1.1010000000 \\ \hline + 1.1010011010 \\ + 1.1010011010 \\ + 1.1010011010 \\ \hline 10.10101110100010 \\ \hline \text{M} \text{GRS} \\ 1.0101011101 \times 2^{-3} \times 2^6 \times 2^1 = 1.0101011101 \times 2^4 \end{array}$$

$$\begin{array}{r}
 1.0101011101 \\
 \times 1.1100000000 \\
 \hline
 + \quad 10101011101 \\
 \quad 10101011101 \\
 \quad 10101011101 \\
 \hline
 10.010110001011 \\
 \hline
 \begin{array}{cc}
 \text{M} & \text{GR S}
 \end{array}
 \end{array}$$

$$1.0010110001 \times 2^1 \times 2^1 \times 2^{-9} = 1.0010110001 \times 2^{-7}$$

$$\begin{array}{l}
 \hookrightarrow 0.0011010010110001 \\
 \hookrightarrow 18B1
 \end{array}$$

Exercise 3.37

Based on your answers to Exercises 3.35 and 3.36, does $(3.41796875 \times 10^{-3} \times 6.34765625 \times 10^{-3}) \times 1.05625 \times 10^2 = 3.41796875 \times 10^{-3} \times (6.34765625 \times 10^{-3} \times 1.05625 \times 10^2)$?

Solution

No because in the first operation the number is so small that underflow.

Exercise 3.38

Calculate $1.666015625 \times 10^0 \times (1.9760 \times 10^4 + -1.9744 \times 10^4)$ by hand, assuming each of the values is stored in the 16-bit half-precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating-point format and in decimal.

Solution

$$\begin{array}{l}
 1.666015625 \times 10^0 = 2^0 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} = 1.101010101 \times 2^0 \\
 1.9760 \times 10^4 = 2^{14} + 2^{11} + 2^{10} + 2^8 + 2^6 + 2^4 = 1.0011010011 \times 2^{14} \\
 -1.9744 \times 10^4 = -(2^{14} + 2^{11} + 2^{10} + 2^8 + 2^5) = -1.001101001 \times 2^{14}
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{cc}
 \text{M} & \text{GR} \\
 1.0011010011 & 00 \\
 -1.0011010010 & 00 \\
 \hline
 0.0000000001 & 00
 \end{array} \\
 \hline
 \end{array}$$

$$1.0 \times 2^4$$

$$\begin{array}{r}
 1.1010101010 \\
 \times 1.0000000000 \\
 \hline
 + \quad 11010101010 \\
 \hline
 1.1010101010 \quad 0000000000 \\
 \hline
 \begin{array}{ccc}
 \text{M} & \text{GR} & \text{S}
 \end{array}
 \end{array}$$

$$1.1010101010 \times 2^4$$

$$\hookrightarrow 0.100111010101010$$

$$\hookrightarrow 4EAA$$

Exercise 3.39

Calculate $(1.666015625 \times 10^0 \times 1.9760 \times 10^4) + (1.666015625 \times 10^0 \times -1.9744 \times 10^4)$ by hand, assuming each of the values is stored in the 16-bit half-precision format described in Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating-point format and in decimal.

Solution

$$1.666015625 \times 10^0 = 2^0 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} = 1.1010101010 \times 2^0$$

$$1.9760 \times 10^4 = 2^{14} + 2^{12} + 2^{10} + 2^8 + 2^5 + 2^4 = 1.0011010011 \times 2^{14}$$

$$-1.9744 \times 10^4 = -(2^{14} + 2^{12} + 2^{10} + 2^8 + 2^5) = -1.001101001 \times 2^{14}$$

$$\begin{array}{r}
 1.1010101010 \\
 \times 1.0011010011 \\
 \hline
 11010101010 \\
 11010101010 \\
 + \quad 11010101010 \\
 + \quad 11010101010 \\
 + \quad 11010101010 \\
 + \quad 11010101010 \\
 \hline
 10.00000010011000011110
 \end{array}$$

M GR S

$$1.0000000101 \times 2^0 \times 2^{14} = 1.0000000101 \times 2^{15}$$

$$\begin{array}{r}
 1.1010101010 \\
 \times 1.0011010010 \\
 \hline
 110101010100 \\
 11010101010 \\
 11010101010 \\
 11010101010 \\
 + \quad 11010101010 \\
 \hline
 10.00000001111101110100
 \end{array}$$

M GR S

$$-1.0000000100 \times 2^0 \times 2^{14} = -1.0000000100 \times 2^{15}$$

$$\begin{array}{r}
 1.0000000101 \quad 00 \\
 -1.0000000100 \quad 00 \\
 \hline
 0.0000000001 \quad 00
 \end{array}$$

$$1.0 \times 2^{-10} \times 2^{15} = 1.0 \times 2^5$$

$$\hookrightarrow 0.10100 \cdot 0000000000$$

$$\hookrightarrow 5000$$

Exercise 3.40

Based on your answers to Exercises 3.38 and 3.39, does $(1.666015625 \times 10^0 \times 1.9760 \times 10^4) + (1.666015625 \times 10^0 \times -1.9744 \times 10^4) = 1.666015625 \times 10^0 \times (1.9760 \times 10^4 + -1.9744 \times 10^4)$?

Solution

Is the same as the other situations but it seem worse because it accumulate to much errors that the real value and the wrong value relationship is about double.

Exercise 3.41

Using the IEEE 754 floating-point format, write down the bit pattern that would represent $-1/4$. Can you represent $-1/4$ exactly?

Solution

Yes it can be represented well in floating point because the number is a power of 2. The sign is 1, exponent is -2 and the mantissa is just the implicit 1.

Exercise 3.42

What do you get if you add $-1/4$ to itself four times? What is $-1/4 \times 4$? Are they the same? What should they be?

Solution

Yes. Precision is enough to get the exact answer.

Exercise 3.43

Write down the bit pattern in the fraction of value $1/3$ assuming a floating-point format that uses binary numbers in the fraction. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

Solution

No it never get an exact representation because $1/3$ isn't a power of 2 number. We could get an approximation or a near value using many decimals as we need. In this example we could get $1/3$ represented as
0.0101_0101_0101_0101_0101

Exercise 3.44

Write down the bit pattern in the fraction of value $1/3$ assuming a floating-point format that uses Binary Coded Decimal (base 10) numbers in the fraction instead of base 2. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

Solution

No as explained earlier if the number base isn't a multiple of the number that we want to represent it cannot be represented as exactly number. For decimal number $1/3$ is represented as
0.3333_3333_3333_3333_3333

Exercise 3.45

Write down the bit pattern assuming that we are using base 15 numbers in the fraction of value $1/3$ instead of base 2. (Base 16 numbers use the symbols 0–9 and A–F. Base 15 numbers would use 0–9 and A–E.) Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

Solution

No 16 isn't a multiple of 3. For hexadecimal numbers $1/3$ is represented as
0.5555_5555_5555_5555_5555

Exercise 3.46

Write down the bit pattern assuming that we are using base 30 numbers in the fraction of value $1/3$ instead of base 2. (Base 16 numbers use the symbols 0–9 and A–F. Base 30 numbers would use 0–9 and A–T.) Assume there are 20 bits, and you do not need to normalize. Is this representation exact?

Solution

With this base we can actually represent the number as an exact number. For number base 30 $1/3$ is represented as 0.A

Exercise 3.47

The following C code implements a four-tap FIR filter on input array `sig_in`. Assume that all arrays are 16-bit fixed-point values.

```
for (i = 3; i < 128; i++)  
    sig_out[i] = sig_in[i-3] * f[0] + sig_in[i-2] * f[1] + sig_in[i-1] * f[2] + sig_in[i] * f[3];
```

Assume you are to write an optimized implementation of this code in assembly language on a processor that has SIMD instructions and 128-bit registers. Without knowing the details of the instruction set, briefly describe how you would implement this code, maximizing the use of sub-word operations and minimizing the amount of data that is transferred between registers and memory. State all your assumptions about the instructions you use.

Solution

If the values of the arrays are 16-bits then we can group up to 8 values on a single 128-bits register. In that case we can unroll the for loop to take advantage of SIMD instructions. First we need to fix the values of `f` so we have the original values from `f[3:0]` but `f[7:4]` set to 0s. Then we load `sig_in[i+7:i]` and on the other register have the actual values of `sig_in` to be used, this values will occupy the spaces of 3:0 of that vector and the remaining 7:4 will be set in 0s. Then we can make use of some multiplier SIMD and then an addition of all elements. We can save it into another vector register. Every cycle has to be of 4 iterations, so we calculate the values of `sig_out[i+3:i]` and then we write it into memory.

Chapter 4

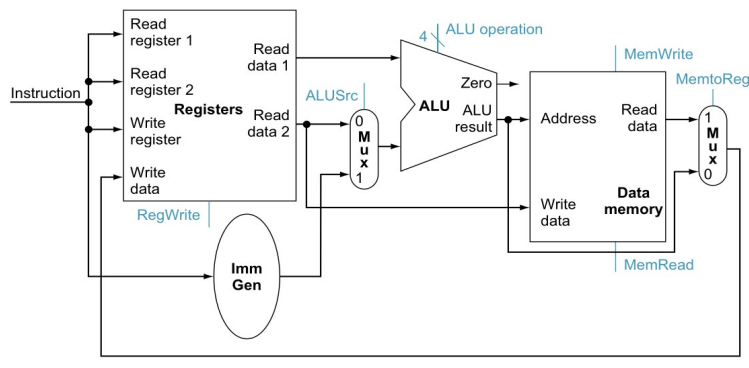
Exercise 4.1

Consider the following instruction:

Instruction: and rd, rs1, rs2

Interpretation: $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] \text{ AND } \text{Reg}[\text{rs2}]$

1. What are the values of control signals generated by the control in Figure 4.10 for this instruction?
2. Which resources (blocks) perform a useful function for this instruction?
3. Which resources (blocks) produce no output for this instruction? Which resources produce output that is not used?



Solution

1. RegWrite=1, ALUSrc=0, ALU_operation="AND", MemWrite=0, MemRead=X, MemtoReg=0
2. The register file, The ALU and both multiplexers
3. Imm_Gen and The Data Memory

Exercise 4.2

Explain each of the “don’t cares” in Figure 4.22.

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Solution

For both instructions SW and BEQ no matter the value of MemtoReg because the register won't write the value anyway.

Exercise 4.3

Consider the following instruction mix:

R-type	I-type (non-lw)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

1. What fraction of all instructions use data memory?
2. What fraction of all instructions use instruction memory?
3. What fraction of all instructions use the sign extend?
4. What is the sign extend doing during cycles in which its output is not needed?

Solution

1. 35%
2. 100%
3. 76%
4. It's discarded

Exercise 4.4

When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get “broken” and always register a logical 0. This is often called a “stuck-at-0” fault.

1. Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?
2. Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?

Solution

1. Loads instructions
2. Every instructions except for R-type

Exercise 4.5

In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6aa23.

1. What are the values of the ALU control unit's inputs for this instruction?
2. What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.
3. For each mux, show the values of its inputs and outputs during the execution of this instruction. List values that are register outputs at Reg [xn].
4. What are the input values for the ALU and the two add units?
5. What are the values of all inputs for the registers unit?

Solution

First at all we need decode the fields of the instruction, the op=0100011, funct3=011, rs2=01100, rs1=01101, imm=000000010100. Putting all this together the instruction decoded is

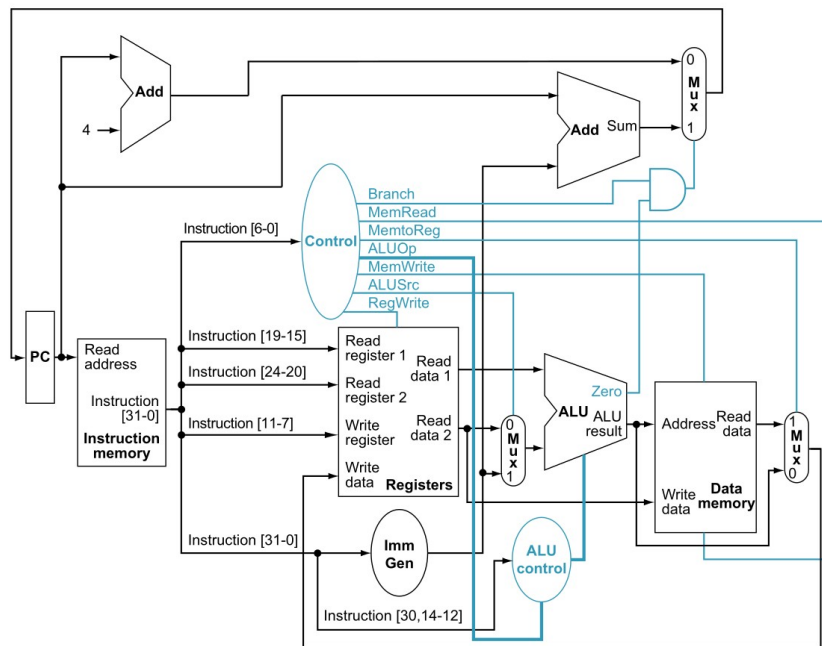
sw x12, 20(x13)

1. The operation is an addition so the control unit will send 00 which will be interpreted by the ALU Control and send 0010 to the ALU at last
2. NextPC = PC+4
3. $ALUOpB = (ALUSrc) ? imm : rs2$ //Will take imm value
 $WriteData = (MemtoReg) ? ReadData : ALUResult$ //No matter, can be any of these
 $NextPC = (Branch \&\& Zero) ? PC + imm : PC + 4$ // Will be PC+4
4. $ALUOpA = rs1$, $ALUOpB = imm$, $ALUControl = 0010$
5. ReadRegister1=01101, ReadRegister2=01100, WriteRegister=10100, WriteData=X, RegWrite=0

Exercise 4.6

Section 4.4 does not discuss I-type instructions like addi or andi.

1. What additional logic blocks, if any, are needed to add I-type instructions to the CPU shown in Figure 4.21? Add any necessary logic blocks to Figure 4.21 and explain their purpose.
2. List the values of the signals generated by the control unit for addi. Explain the reasoning for any “don't care” control signals.



Solution

1. Because the Control Unit will attend opcodes then both I-type ALU instructions and R-type ALU instructions behave the same because it shares the same values for funct3 and funct7. So it won't need any change.
2. ALUSrc=1, MemtoReg=0, RegWrite=1, MemRead=0, MemWrite=0, Branch=0, ALUOp=10

Exercise 4.7

Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem / D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

"Register read" is the time needed after the rising clock edge for the new register value to appear on the output. This value applies to the PC only. "Register setup" is the amount of time a register's data input must be stable before the rising edge of the clock. This value applies to both the PC and Register File.

1. What is the latency of an R-type instruction (i.e., how long must the clock period be to ensure that this instruction works correctly)?
2. What is the latency of lw? (Check your answer carefully. Many students place extra muxes on the critical path.)
3. What is the latency of sw? (Check your answer carefully. Many students place extra muxes on the critical path.)
4. What is the latency of beq?
5. What is the latency of an arithmetic, logical, or shift I-type (non-load) instruction?
6. What is the minimum clock period for this CPU?

Solution

1. $Latency_{RTYPE} = PC + IMEM + RegFile + Mux + ALU + Mux + setup$
 $Latency_{RTYPE} = 30\text{ ps} + 250\text{ ps} + 150\text{ ps} + 25\text{ ps} + 200\text{ ps} + 25\text{ ps} + 20\text{ ps} = 700\text{ ps}$
2. $Latency_{LW} = PC + IMEM + MAX(RegFile, SignExt) + Mux + ALU + DMEM + Mux + setup$
 $Latency_{LW} = 30\text{ ps} + 250\text{ ps} + 150\text{ ps} + 25\text{ ps} + 200\text{ ps} + 250\text{ ps} + 25\text{ ps} + 20\text{ ps} = 950\text{ ps}$
ALUSrc Mux is not considered because rs2 isn't necessary to be set and immediate is already available.

- $Latency_{sw} = PC + IMEM + MAX(RegFile, SignExt) + Mux + ALU + DMEM$
 $Latency_{sw} = 30\text{ ps} + 250\text{ ps} + 150\text{ ps} + 25\text{ ps} + 200\text{ ps} + 250\text{ ps} = 905\text{ ps}$
 MemtoReg Mux and setup is not considered because there's not any write back
- $Latency_{beq} = PC + IMEM + MAX((RegFile + Mux + ALU + Gate), (SignExt + Adder)) + Mux + setup$
 $Latency_{beq} = 30\text{ ps} + 250\text{ ps} + 150\text{ ps} + 25\text{ ps} + 200\text{ ps} + 5\text{ ps} + 25\text{ ps} + 20\text{ ps} = 705\text{ ps}$
- $Latency_{itype} = PC + IMEM + MAX(RegFile, SignExt) + Mux + ALU + Mux + setup$
 $Latency_{itype} = 30\text{ ps} + 250\text{ ps} + 150\text{ ps} + 25\text{ ps} + 200\text{ ps} + 25\text{ ps} + 20\text{ ps} = 700\text{ ps}$
- To assure a correct functioning we take the worst case scenario. So the processor need to be this fast or less than 950ps.

Exercise 4.8

Suppose you could build a CPU where the clock cycle time was different for each instruction. What would the speedup of this new CPU be over the CPU presented in Figure 4.25 given the instruction mix below?

R-type/I-type (non-ld)	lw	sw	beq
52%	25%	11%	12%

Solution

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{950\text{ ps}}{(0.52 * 700\text{ ps} + 0.25 * 950\text{ ps} + 0.11 * 905\text{ ps} + 0.12 * 705\text{ ps})} = 1.209$$

Exercise 4.9

Consider the addition of a multiplier to the CPU shown in Figure 4.25. This addition will add 300 ps to the latency of the ALU, but will reduce the number of instructions by 5% (because there will no longer be a need to emulate the multiply instruction).

- What is the clock cycle time with and without this improvement?
- What is the speedup achieved by adding this improvement?
- What is the slowest the new ALU can be and still result in improved performance?

Solution

- To get the clock cycle we just add the 300ps to the critical operation of the CPU which is LW so we had clock cycle of $950\text{ps} + 300\text{ps} = 1250\text{ps}$

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{Instr * CPI * 950\text{ ps}}{0.95 * Instr * CPI * 1250\text{ ps}} = 0.8$$

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{Instr * CPI * 950\text{ ps}}{0.95 * Instr * CPI * X} = 1$$

$$X = 950\text{ ps} / 0.95 = 1000\text{ ps}$$

This give us $1000\text{ps} - 950\text{ps} = 50\text{ps}$ or less to add to ALU latency be faster

Exercise 4.10

When processor designers consider a possible improvement to the processor datapath, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are beginning with the datapath from Figure 4.25, the latencies from Exercise 4.7, and the following costs:

I-Mem	Register File	Mux	ALU	Adder	D-Mem	Single Register	Sign extend	Single gate	Control
1000	200	10	100	30	2000	5	100	1	500

Suppose doubling the number of general purpose registers from 32 to 64 would reduce the number of lw and sw instructions executed by 12%, but increase the latency of the register file from 150 ps to 160 ps and double the cost from 200 to 400. (Use the instruction mix from Exercise 4.8.)

1. What is the speedup achieved by adding this improvement?
2. Compare the change in performance to the change in cost.
3. Given the cost/performance ratios you just calculated, describe a situation where it makes sense to add more registers and describe a situation where it doesn't make sense to add more registers.

Solution

1. Between loads and stores it sums 36%, now the 88% of 36% is 31.68% so the total percentage for the instructions are $31.68+52+12=95.68\%$. Now if we do the sum the new CPU latency shall be 960ps.

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{Instr * CPI * 950 \text{ ps}}{0.9568 * Instr * CPI * 960 \text{ ps}} = 1.034$$

$$Cost = (PC + 3 * Mux + 2 * Adder + IMem + DMem + ALU + RegFile + 2 * Control + SignExt + Gate)$$
2.
$$\frac{Cost_{AFT}}{Cost_{BEF}} = \frac{5 + 3 * 10 + 2 * 30 + 1000 + 2000 + 100 + 2 * 200 + 2 * 500 + 100 + 1}{5 + 3 * 10 + 2 * 30 + 1000 + 2000 + 100 + 200 + 2 * 500 + 100 + 1} = \frac{4696}{4496} = 1.044$$

We get a 3.4% of plus performance at the price of the core costing 4.4% more
3. Price rises more than performance, in that case that we use a lot of cores it doesn't make sense to use bigger cores. However if performance of some deadlines are critical then it better to spend more to make it faster.

Exercise 4.11

Examine the difficulty of adding a proposed `lwi.d rd, rs1, rs2` ("Load With Increment") instruction to RISC-V. Interpretation: $Reg[rd] = Mem[Reg[rs1] + Reg[rs2]]$

1. Which new functional blocks (if any) do we need for this instruction?
2. Which existing functional blocks (if any) require modification?
3. Which new data paths (if any) do we need for this instruction?
4. What new signals do we need (if any) from the control unit to support this instruction?

Solution

1. There's no need to add a new functional block
2. The control need to add this instruction (R-Type format) to be able to retrieve the data from memory just changing `MemtoReg`
3. There's no need
4. There's no need

Exercise 4.12

Examine the difficulty of adding a proposed `swap rs1, rs2` instruction to RISC-V. Interpretation: $Reg[rs2] = Reg[rs1]; Reg[rs1] = Reg[rs2]$

1. Which new functional blocks (if any) do we need for this instruction?
2. Which existing functional blocks (if any) require modification?
3. What new data paths do we need (if any) to support this instruction?
4. What new signals do we need (if any) from the control unit to support this instruction?
5. Modify Figure 4.25 to demonstrate an implementation of this new instruction.

Solution

1. 2 multiplexers.
2. Is needed a heavy modification to Register File, First we need the ability of write data on two register at the same time so we need to add the inputs `WriteRegister2`, `WriteData2` and `RegWrite2`.
3. We need to create a new datapath. As we modified later the register file we got `WriteRegister1` and `WriteRegister2` as register addresses. We need to add a multiplexer as the following

$$WriteRegister1 = (RegToWrite) ? rs2 : rd$$

Now is easy to wonder that

$$WriteRegister2 = rs1$$

Finally we can wire out ReadData1 and ReadData2 so
 WriteData1 = (WriteToReg) ? MemtoRegOut : ReadData2
 WriteData2 = ReadData1

4. We need modify the Control Unit to add the signal RegWrite2 and signal for the two multiplexers
5. /*Pending*/

Exercise 4.13

Examine the difficulty of adding a proposed ss rs1, rs2, imm (Store Sum) instruction to RISC-V. Interpretation:
 Mem[Reg[rs1]]=Reg[rs2]+immediate

1. Which new functional blocks (if any) do we need for this instruction?
2. Which existing functional blocks (if any) require modification?
3. What new data paths do we need (if any) to support this instruction?
4. What new signals do we need (if any) from the control unit to support this instruction?
5. Modify Figure 4.25 to demonstrate an implementation of this new instruction.

Solution

1. 2 multiplexers
2. None
3. The easy way is to put a multiplexer to decide between rs1 or rs2 to be the first operand of the ALU and then wire out rs1 and put a multiplexer between ALU output and rs1 as described

$$ALUOpA = (RegSel) ? ReadData1 : ReadData2$$

$$MemAddress = (AddressSel) ? ALUResult : ReadData1$$
4. We need to add 2 signals for the multiplexers
5. /*Pending*/

Exercise 4.14

For which instructions (if any) is the Imm Gen block on the critical path?

Solution

Only for U-type and J-type instructions because we don't need to wait to read the register file. Any read to the register file will be slower than sign extension.

Exercise 4.15

lw is the instruction with the longest latency on the CPU from Section 4.4. If we modified lw and sw so that there was no offset (i.e., the address to be loaded from/stored to must be calculated and placed in rs1 before calling lw/sw), then no instruction would use both the ALU and Data memory. This would allow us to reduce the clock cycle time. However, it would also increase the number of instructions, because many lw and sw instructions would need to be replaced with lw/add or sw/add combinations.

1. What would the new clock cycle time be?
2. Would a program with the instruction mix presented in Exercise 4.8 run faster or slower on this new CPU? By how much? (For simplicity, assume every lw and sw instruction is replaced with a sequence of two instructions.)
3. What is the primary factor that influences whether a program will run faster or slower on the new CPU?
4. Do you consider the original CPU (as shown in Figure 4.25) a better overall design; or do you consider the new CPU a better overall design? Why?

Solution

1. With this supposition we have D-Mem and ALU working in parallel. This mean that Load now will take 750ps and store will take 705ps. The remaining instructions will take the same time R-type: 700ps B-type: 705ps I-type: 675ps. So the clock cycle must be of 750ps.
2.
$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{950 \text{ ps}}{(0.52 + 0.25 * 2 + 0.11 * 2 + 0.12) * 750 \text{ ps}} = 0.9313$$

It will run slower but not for much

3. The proportion of loads and store instructions
4. Even if the new CPU was faster is important the way of handle code and the convenience of simplify instructions. Only when a significant gap of performance is achieved is when it worth to changing it.

Exercise 4.16

In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
250 ps	350 ps	150 ps	300 ps	200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic	Jump/Branch	Load	Store
45%	20%	20%	15%

1. What is the clock cycle time in a pipelined and non-pipelined processor?
2. What is the total latency of an lw instruction in a pipelined and non-pipelined processor?
3. If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?
4. Assuming there are no stalls or hazards, what is the utilization of the data memory?
5. Assuming there are no stalls or hazards, what is the utilization of the write-register port of the “Registers” unit?

Solution

1. For the non-pipelined we had
 $CC_{NP} = 250\text{ ps} + 350\text{ ps} + 150\text{ ps} + 300\text{ ps} + 200\text{ ps} = 1250\text{ ps}$
For a pipelined we had
 $CC_P = \text{MAX}(250\text{ ps}, 350\text{ ps}, 150\text{ ps}, 300\text{ ps}, 200\text{ ps}) = 350\text{ ps}$
2. The latency for the instruction of a non-pipelined will be of 1250ps. For the pipelined it would need to do 5 cycles to execute the whole instruction so $350\text{ps} \times 5 = 1750\text{ps}$
3. Instruction decode is the stage with more latency so if we split in 2 we could reduce clock cycle. With this change the clock cycle will be of 300ps
4. Of 35%
5. Of 65%

Exercise 4.17

What is the minimum number of cycles needed to completely execute n instructions on a CPU with a k stage pipeline? Justify your formula.

Solution

I think $ET = n + k - 1$. This because the instruction need at least k cycles to make it trough the whole pipeline but from here the throughput remain constant of one cycle per instruction

Exercise 4.18

Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline from Section 4.6 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of registers x13 and x14 be?

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
```

Solution

The second and third instructions depends of the first one. However the value of x11 wont be updated until the WB stage of the first instruction is completed so x13 is equal to 33 and x14 is equal to 36.

Exercise 4.19

Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline from Section 4.6 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of register x15 be? Assume the register file is written at the beginning of the cycle and read at the end of a cycle. Therefore, an ID stage will return the results of a WB state occurring during the same cycle. See Section 4.8 and Figure 4.68 for details.

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
add   x15, x11, x11
```

Solution

Like the last problem the value of x11 will update only after 3 instructions so when the last instruction is computed x11 will had the correct value so x15 will be equal to 54

Exercise 4.20

Add NOP instructions to the code below so that it will run correctly on a pipeline that does not handle data hazards.

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
add   x15, x13, x12
```

Solution

```
addi  x11, x12, 5
nop
nop
add   x13, x11, x12
addi  x14, x11, 15
nop
add   x15, x13, x12
```

Exercise 4.21

Consider a version of the pipeline from Section 4.6 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). Suppose that (after optimization) a typical n-instruction program requires an additional $0.4 \cdot n$ NOP instructions to correctly handle data hazards.

1. Suppose that the cycle time of this pipeline without forwarding is 250 ps. Suppose also that adding forwarding hardware will reduce the number of NOPs from $0.4 \cdot n$ to $0.05 \cdot n$, but increase the cycle time to 300 ps. What is the speedup of this new pipeline compared to the one without forwarding?
2. Different programs will require different amounts of NOPs. How many NOPs (as a percentage of code instructions) can remain in the typical program before that program runs slower on the pipeline with forwarding?
3. Repeat 4.21.2; however, this time let x represent the number of NOP instructions relative to n. (In 4.21.2, x was equal to 0.4.) Your answer will be with respect to x.

- Can a program with only $0.075 \cdot n$ NOPs possibly run faster on the pipeline with forwarding? Explain why or why not.
- At minimum, how many NOPs (as a percentage of code instructions) must a program have before it can possibly run faster on the pipeline with forwarding?

Solution

$$1. \frac{ET_{BEF}}{ET_{AFT}} = \frac{250 ps \cdot 1.4 \cdot n \cdot CPI}{300 ps \cdot 1.05 \cdot n \cdot CPI} = 350/315 = 1.11$$

$$250 ps \cdot 1.4 \cdot n > 300 ps \cdot (1+x) \cdot n$$

$$2. x < \left(\frac{250 ps \cdot 1.4 \cdot n}{300 ps \cdot n} - 1 \right) = 0.1667$$

It will need 0.16 or more nop per instruction to the pipeline with forwarding to be slower than the pipeline without it. That's mean 16.6%

- If we want to know the relation between nop between this 2 CPUs where both of this amounts are variable then we know that

$$250 ps \cdot (1+y) \cdot n > 300 ps \cdot (1+x) \cdot n$$

$$250 ps + 250 ps \cdot y > 300 ps + 300 ps \cdot x$$

$$x < \frac{250 ps \cdot y - 50 ps}{300 ps}$$

- No it can not. Because if the non pipelined run at ratio of $250 \cdot 1.075 \cdot n$ will be faster than $300 \cdot (1+x) \cdot n$ even if $x=0$
- If we suppose that the forwarding eliminate any need of insert nop then a non-forwarding pipeline will need 0.2 nop per instruction or less to start to be faster

$$0 < \frac{250 ps \cdot y - 50 ps}{300 ps}$$

$$y < 50 ps / 250 ps = 0.2$$

Exercise 4.22

Consider the fragment of RISC-V assembly below:

```
sw    x29, 12(x16)
lw    x29, 8(x16)
sub   x17, x15, x14
beqz  x17, label
add   x15, x11, x14
sub   x15, x30, x14
```

Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

- Draw a pipeline diagram to show where the code above will stall.
- In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code?
- Must this structural hazard be handled in hardware? We have seen that data hazards can be eliminated by adding NOPs to the code. Can you do the same with this structural hazard? If so, explain how. If not, explain why not.
- Approximately how many stalls would you expect this structural hazard to generate in a typical program? (Use the instruction mix from Exercise 4.8.)

Solution

- Diagram

Instructions / Cycle	1	2	3	4	5	6	7	8	9	10	11
----------------------	---	---	---	---	---	---	---	---	---	----	----

sw	x29, 12(x16)	IF	ID	EX	ME	WB						
lw	x29, 8(x16)		IF	ID	EX	ME	WB					
sub	x17, x15, x14			IF	ID	EX	ME	WB				
beqz	x17, label				**	IF	ID	EX	ME	WB		
add	x15, x11, x14					**	IF	ID	EX	ME	WB	
sub	x15, x30, x14							IF	ID	EX	ME	WB

2. No, for every load and store instruction it need to stall 1 cycle.
3. With this hazard is not possible to solve it with nop because even this instruction need to be fetched.
4. With this proportion then we need to stall the core 0.36 times per instruction.

Exercise 4.23

If we change load/store instructions to use a register (without an offset) as the address, these instructions no longer need to use the ALU. (See Exercise 4.15.) As a result, the MEM and EX stages can be overlapped and the pipeline has only four stages.

1. How will the reduction in pipeline depth affect the cycle time?
2. How might this change improve the performance of the pipeline?
3. How might this change degrade the performance of the pipeline?

Solution

1. The cycle time will be the same because the critical path and times will still the same
2. This change make less clock cycles to process a single instruction, this improvement is about 1.25 faster. However the throughput will be the same so the improvement at large incomes is negligible.
3. This change create the need of execute 2 instructions for doing the same. This makes the things a lot worse because you need the add instruction to finish to execute or doing forwarding to load or store have the right address

Exercise 4.24

Which of the two pipeline diagrams below better describes the operation of the pipeline's hazard detection unit? Why?

Choice 1:

lw	x11, 0(x12)	IF	ID	EX	ME	WB				
add	x13, x11, x14		IF	ID	EX	**	ME	WB		
or	x15, x16, x17			IF	ID	**	EX	ME	WB	

Choice 2:

lw	x11, 0(x12)	IF	ID	EX	ME	WB				
add	x13, x11, x14		IF	ID	**	EX	ME	WB		
or	x15, x16, x17			IF	**	ID	EX	ME	WB	

Solution

The second one because the updated value of a register must be available before the EX stage to do a right calculation.

Exercise 4.25

Consider the following loop.

LOOP:

```
lw    x10, 0(x13)
lw    x11, 8(x13)
add   x12, x10, x11
```

```

addi  x13, x13, 16
bnez  x12, LOOP

```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, that the pipeline has full forwarding support, and that branches are resolved in the EX (as opposed to the ID) stage.

1. Show a pipeline execution diagram for the first two iterations of this loop.
2. Mark pipeline stages that do not perform useful work. How often while the pipeline is full do we have a cycle in which all five pipeline stages are doing useful work? (Begin with the cycle during which the addi is in the IF stage. End with the cycle during which the bnez is in the IF stage.)

Solution

1. Diagram

Instructions / Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
lw x10, 0(x13)	IF	ID	EX	ME	WB											
lw x11, 8(x13)		IF	ID	EX	ME	WB										
add x12, x10, x11			IF	ID	**	EX	ME	WB								
addi x13, x13, -16				IF	**	ID	EX	ME	WB							
bnez x12, LOOP					**	IF	ID	EX	ME	WB						
lw x10, 0(x13)							IF	ID	EX	ME	WB					
lw x11, 8(x13)								IF	ID	**	EX	ME	WB			
add x12, x10, x11									IF	**	ID	EX	ME	WB		
addi x13, x13, -16										**	IF	ID	EX	ME	WB	
bnez x12, LOOP												IF	ID	EX	ME	WB

2. The not useful work is marked with bold type. The cycles where the pipeline is completely full is between 5 and 12.

Exercise 4.26

This exercise is intended to help you understand the cost/complexity/performance trade-offs of forwarding in a pipelined processor. Problems in this exercise refer to pipelined datapaths from Figure 4.70. These problems assume that, of all the instructions executed in a processor, the following fraction of these instructions has a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the next instruction that consumes the result (1st instruction that follows the one that produces the result, 2nd instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so “EX to 3rd” and “MEM to 3rd” dependences are not counted because they cannot result in data hazards. We also assume that branches are resolved in the EX stage (as opposed to the ID stage), and that the CPI of the processor is 1 if there are no data hazards.

EX to 1 st Only	MEM to 1 st Only	EX to 2 nd Only	MEM to 2 nd Only	EX to 1 st and EX to 2 nd
5%	20%	5%	10%	10%

Assume the following latencies for individual pipeline stages. For the EX stage, latencies are given separately for a processor without forwarding and for a processor with different kinds of forwarding.

IF	ID	EX (no FW)	EX (full FW)	EX (FW from EX/MEM only)	EX (FW from MEM/WB only)	MEM	WB
120 ps	100 ps	110 ps	130 ps	120 ps	120 ps	120 ps	100 ps

- For each RAW dependency listed above, give a sequence of at least three assembly statements that exhibits that dependency.
- For each RAW dependency above, how many NOPs would need to be inserted to allow your code from 4.26.1 to run correctly on a pipeline with no forwarding or hazard detection? Show where the NOPs could be inserted.
- Analyzing each instruction independently will over-count the number of NOPs needed to run a program on a pipeline with no forwarding or hazard detection. Write a sequence of three assembly instructions so that, when you consider each instruction in the sequence independently, the sum of the stalls is larger than the number of stalls the sequence actually needs to avoid data hazards.
- Assuming no other hazards, what is the CPI for the program described by the table above when run on a pipeline with no forwarding? What percent of cycles are stalls? (For simplicity, assume that all necessary cases are listed above and can be treated independently.)
- What is the CPI if we use full forwarding (forward all results that can be forwarded)? What percent of cycles are stalls?
- Let us assume that we cannot afford to have three-input multiplexors that are needed for full forwarding. We have to decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). What is the CPI for each option?
- For the given hazard probabilities and pipeline stage latencies, what is the speedup achieved by each type of forwarding (EX/MEM, MEM/WB, for full) as compared to a pipeline that has no forwarding?
- What would be the additional speedup (relative to the fastest processor from 4.26.7) be if we added “time-travel” forwarding that eliminates all data hazards? Assume that the yet-to-be-invented time-travel circuitry adds 100 ps to the latency of the full-forwarding EX stage.
- The table of hazard types has separate entries for “EX to 1st” and “EX to 1st and EX to 2nd”. Why is there no entry for “MEM to 1st and MEM to 2nd”?

Solution

- RAW dependencies

EX to 1st only

add x29, x30, x31

add x14, x29, x16 //dependency of first instruction EX stage

add x5, x6, x7 //no dependency

MEM to 1st only

lw x29, 0(x30)

add x14, x29, x16 //dependency of first instruction MEM stage

add x5, x6, x7 //no dependency

EX to 2nd only

add x29, x30, x31

add x5, x6, x7 //no dependency

add x14, x29, x16 //dependency of first instruction MEM stage

MEM to 2nd only

lw x29, 0(x30)

add x5, x6, x7 //no dependency

add x14, x29, x16 //dependency of first instruction WB stage

EX to 1st and EX to 2nd

add x29, x30, x31

add x14, x29, x16 //dependency of first instruction EX stage

add x5, x29, x7 //dependency of first instruction MEM stage

2. RAW dependencies with nop

EX to 1st only (2 stalls)

add x29, x30, x31

nop

nop

add x14, x29, x16

add x5, x6, x7

MEM to 1st only (2 stalls)

lw x29, 0(x30)

nop

nop

add x14, x29, x16

add x5, x6, x7

EX to 2nd only (1 stall)

add x29, x30, x31

add x5, x6, x7

nop

add x14, x29, x16

MEM to 2nd only (1 stall)

lw x29, 0(x30)

add x5, x6, x7

nop

add x14, x29, x16

EX to 1st and EX to 2nd (2 stalls)

add x29, x30, x31

nop

nop

add x14, x29, x16

add x5, x29, x7

3. If you overlap the necessity of nop for various hazard then you can save some nop

lw x29, 0(x30) // in theory it need 2 stalls

add x28, x7, x8 // in theory it need 1 stall

nop

add x6, x29, x28

4. $CPI = 0.05 * 3 + 0.2 * 3 + 0.05 * 2 + 0.1 * 2 + 0.1 * 3 + 0.5 * 1 = 1.85$

A perfect CPI will be of 1 so we have 0.85 stalls per instruction so we have $0.85/1.85 = 0.46$ proportion of stalls per program

5. The only kind of instruction that cannot be forwarded is MEM to 1st hazard, however we can reduce the number of nop at 1 because we only need to wait to MEM stage of this instruction complete so we have $CPI = 0.2 * 2 + 0.8 * 1 = 1.2$

With this we have 0.2 stalls per instruction so $0.2/1.2 = 0.1666$ proportion of stalls per program

6. If we had only EX/MEM then we had the next stalls per hazard

EX to 1st only: 0

MEM to 1st only: 2

EX to 2nd only: 1

MEM to 2nd only: 1

EX to 1st and EX to 2nd: 1

That's make an CPI of

$CPI = 0.05 * 1 + 0.2 * 3 + 0.05 * 2 + 0.1 * 2 + 0.1 * 2 + 0.5 * 1 = 1.65$

Now if we had only MEM/WB then we had the follow stalls per hazard

EX to 1st only: 1

MEM to 1st only: 1
 EX to 2nd only: 0
 MEM to 2nd only: 0
 EX to 1st and EX to 2nd: 1

Making up to an CPI of

$$CPI = 0.05 * 2 + 0.2 * 2 + 0.05 * 1 + 0.1 * 1 + 0.1 * 2 + 0.5 * 1 = 1.35$$

7. $ET_{NFW} = MAX(IF, ID, EX_{NFW}, MEM, WB) * CPI_{NFW}$
 $ET_{NFW} = MAX(120\text{ ps}, 100\text{ ps}, 110\text{ ps}, 120\text{ ps}, 100\text{ ps}) * 1.85 = 222 * 10^{-9} \text{ sec/instr}$
 $ET_{PFW1} = MAX(IF, ID, EX_{PFW1}, MEM, WB) * CPI_{PFW1}$
 $ET_{PFW1} = MAX(120\text{ ps}, 100\text{ ps}, 120\text{ ps}, 120\text{ ps}, 100\text{ ps}) * 1.65 = 198 * 10^{-9} \text{ sec/instr}$
 $ET_{PFW2} = MAX(IF, ID, EX_{PFW2}, MEM, WB) * CPI_{PFW2}$
 $ET_{PFW2} = MAX(120\text{ ps}, 100\text{ ps}, 120\text{ ps}, 120\text{ ps}, 100\text{ ps}) * 1.35 = 162 * 10^{-9} \text{ sec/instr}$
 $ET_{FFW} = MAX(IF, ID, EX_{FFW}, MEM, WB) * CPI_{FFW}$
 $ET_{FFW} = MAX(120\text{ ps}, 100\text{ ps}, 130\text{ ps}, 120\text{ ps}, 100\text{ ps}) * 1.2 = 156 * 10^{-9} \text{ sec/instr}$
8. $ET_{FFWTT} = MAX(IF, ID, EX_{FFWTT}, MEM, WB) * CPI_{FFWTT}$
 $ET_{FFWTT} = MAX(120\text{ ps}, 100\text{ ps}, (130\text{ ps} + 100\text{ ps}), 120\text{ ps}, 100\text{ ps}) * 1 = 230 * 10^{-9} \text{ sec/instr}$
 The speed up is of $156/230 = 0.678$
9. Because any memory operation will always need to stall by one so if you think there can not be any difference between MEM to 1st and MEM to 2nd and MEM to 1st only

Exercise 4.27

Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a five-stage pipelined datapath:

```
add    x15, x12, x11
lw     x13, 8(x15)
lw     x12, 0(x2)
or     x13, x15, x13
sw     x13, 0(x15)
```

1. If there is no forwarding or hazard detection, insert NOPs to ensure correct execution.
2. Now, change and/or rearrange the code to minimize the number of NOPs needed. You can assume register x17 can be used to hold temporary values in your modified code.
3. If the processor has forwarding, but we forgot to implement the hazard detection unit, what happens when the original code executes?
4. If there is forwarding, for the first seven cycles during the execution of this code, specify which signals are asserted in each cycle by hazard detection and forwarding units in Figure 4.76.
5. If there is no forwarding, what new input and output signals do we need for the hazard detection unit in Figure 4.76? Using this instruction sequence as an example, explain why each signal is needed.
6. For the new hazard detection unit from 4.26.5, specify which output signals it asserts in each of the first five cycles during the execution of this code.

Solution

1. add x15, x12, x11
 nop
 nop
 lw x13, 8(x15)
 lw x12, 0(x2)
 nop
 or x13, x15, x13
 nop

nop

sw x13, 0(x15)

2. It isn't possible to reduce nops. Dependencies are straight forward
3. It will run correctly because there is only one hazard with the or operation which depends of the load instruction to x13. But the load between it can make unnecessary to insert a bubble
4. Diagram

Instructions / Cycle	1	2	3	4	5	6	7	8	9
add x15, x12, x11	IF	ID	EX	ME	WB				
lw x13, 8(x15)		IF	ID	EX	ME	WB			
lw x12, 0(x2)			IF	ID	EX	ME	WB		
or x13, x15, x13				IF	ID	EX	ME	WB	
sw x13, 0(x15)					IF	ID	EX	ME	WB

Because there will be no stall then PCWrite and IF/IDWrite and ID/EXMemRead will assert 1

- Cycle 1: ForwardA=X, ForwardB=X
 - Cycle 2: ForwardA=X, ForwardB=X
 - Cycle 3: ForwardA=0, ForwardB=0. No dependency
 - Cycle 4: ForwardA=2, ForwardB=0. Retrieve ME stage result
 - Cycle 5: ForwardA=0, ForwardB=0. No dependency
 - Cycle 6: ForwardA=0, ForwardB=1. Retrieve WB stage result
 - Cycle 7: ForwardA=0, ForwardB=2. Retrieve ME stage result
5. If there is no forwarding then we need to move some of the logic to the hazard unit. First we need to retrieve the values of rd on EX and MEM stage to know dependencies of the next 2 instruction from ID stage. If is detected a dependency then we need to stall the CPU
 6. Diagram

Instructions / Cycle	1	2	3	4	5	6
add x15, x12, x11	IF	ID	EX	ME	WB	
lw x13, 8(x15)		IF	ID	**	**	EX
lw x12, 0(x2)			IF	**	**	ID
or x13, x15, x13						IF
sw x13, 0(x15)						

- Cycle 1: PCWrite=1, IF/IDWrite=1, ControlMux=0
- Cycle 2: PCWrite=1, IF/IDWrite=1, ControlMux=0
- Cycle 3: PCWrite=1, IF/IDWrite=1, ControlMux=0
- Cycle 4: PCWrite=0, IF/IDWrite=0, ControlMux=1
- Cycle 5: PCWrite=0, IF/IDWrite=0, ControlMux=1

Exercise 4.28

The importance of having a good branch predictor depends on how often conditional branches are executed. Together with branch predictor accuracy, this will determine how much time is spent stalling due to mispredicted branches. In this exercise, assume that the breakdown of dynamic instructions into various instruction categories is as follows:

R-type	beqz/bnez	jal	lw	sw
40%	25%	5%	25%	5%

Also, assume the following branch predictor accuracies:

Always-Taken	Always-Not-Taken	2-Bit
45%	55%	85%

1. Stall cycles due to mispredicted branches increase the CPI. What is the extra CPI due to mispredicted branches with the always-taken predictor? Assume that branch outcomes are determined in the ID stage and applied in the EX stage that there are no data hazards, and that no delay slots are used.
2. Repeat 4.28.1 for the “always-not-taken” predictor.
3. Repeat 4.28.1 for the 2-bit predictor.
4. With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions to some ALU instruction? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
5. With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaced each branch instruction with two ALU instructions? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
6. Some branch instructions are much more predictable than others. If we know that 80% of all executed branch instructions are easy-to-predict loop-back branches that are always predicted correctly, what is the accuracy of the 2-bit predictor on the remaining 20% of the branch instructions?

Solution

1. $CPI = (0.4 + (0.25 * 0.45) + 0.05 + 0.25 + 0.05) * 1 + 0.25 * (1 - 0.45) * 2 = 1.1375$
2. $CPI = (0.4 + (0.25 * 0.55) + 0.05 + 0.25 + 0.05) * 1 + 0.25 * (1 - 0.55) * 2 = 1.1125$
3. $CPI = (0.4 + (0.25 * 0.85) + 0.05 + 0.25 + 0.05) * 1 + 0.25 * (1 - 0.85) * 2 = 1.0375$
4. $CPI = (0.4 + (0.125 * 0.85) + 0.05 + 0.25 + 0.05 + 0.125) * 1 + 0.125 * (1 - 0.85) * 2 = 1.01875$

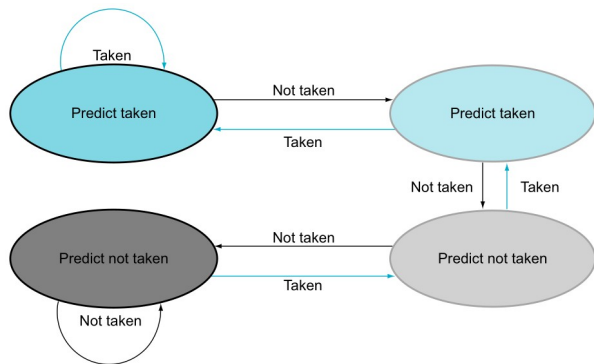
$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{1.0375 * Instr * CC}{1.01875 * Instr * CC} = 1.0184$$
5. $CPI = (0.4 + (0.125 * 0.85) + 0.05 + 0.25 + 0.05) * 1 + 0.125 * (1 - 0.85) * 2 + 0.125 * 2 = 1.14375$

$$\frac{ET_{BEF}}{ET_{AFT}} = \frac{1.0375 * Instr * CC}{1.14375 * Instr * CC} = 0.9071$$
6. $0.8 + 0.2 * X = 0.85$
 $X = (0.85 - 0.8) / 0.2 = 0.25$

Exercise 4.29

This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, NT, T, T, NT.

1. What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
2. What is the accuracy of the 2-bit predictor for the first four branches in this pattern, assuming that the predictor starts off in the bottom left state from Figure 4.78 (predict not taken)?
3. What is the accuracy of the 2-bit predictor if this pattern is repeated forever?
4. Design a predictor that would achieve a perfect accuracy if this pattern is repeated forever. Your predictor should be a sequential circuit with one output that provides a prediction (1 for taken, 0 for not taken) and no inputs other than the clock and the control signal that indicates that the instruction is a conditional branch.
5. What is the accuracy of your predictor from 4.29.4 if it is given a repeating pattern that is the exact opposite of this one?
6. Repeat 4.29.4, but now your predictor should be able to eventually (after a warm-up period during which it can make wrong predictions) start perfectly predicting both this pattern and its opposite. Your predictor should have an input that tells it what the real outcome was. Hint: this input lets your predictor determine which of the two repeating patterns it is given.



Solution

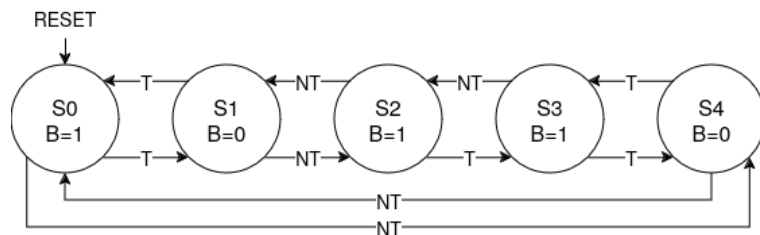
1. For an always-taken will be of $3/5=0.6$ and for always-not-taken will be $2/5=0.4$
2. The pattern of the taken branches can be viewed as (where the bottom left is state 0 and top left is state 3):

Sequence	Prediction	Result	Accuracy
T, NT, T, T	0->1->0->1	I, C, I, I	25%

3. If we repeat the pattern of the 4 branches eventually will reach a steady point, so from here the accuracy of the predictor will stay at 60%

Sequence	Prediction	Result	Accuracy
T, NT, T, T, NT	0->1->0->1->2	I, C, I, I, I	40%
T, NT, T, T, NT	1->2->1->2->3	I, I, I, C, I	20%
T, NT, T, T, NT	2->3->2->3->3	C, I, C, C, I	60%

4. The predictor would need of 5 states to do all the sequence.



5. If the sequence is inverted the we have NT, T, NT, NT, T. Now because it have been designed for the other pattern it always will have a 0% accuracy.
6. The predictor proposed we can eventually had a perfect predictor for both sequences just inverting the output. So we only need to compare which activate correctly (original or inverted) and then select this pattern. If it done like this we only need one branch to know which pattern is given.

Exercise 4.30

This exercise explores how exception handling affects pipeline design. The first three problems in this exercise refer to the following two instructions:

Instruction 1	Instruction 2
beqz x11, LABEL	lw x11, 0(x12)

1. Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected.
2. If there is a separate handler address for each exception, show how the pipeline organization must be changed to be able to handle this exception. You can assume that the addresses of these handlers are known when the processor is designed.
3. If the second instruction is fetched immediately after the first instruction, describe what happens in the pipeline when the first instruction causes the first exception you listed in Exercise 4.30.1. Show the pipeline execution diagram from the time the first instruction is fetched until the time the first instruction of the exception handler is completed.
4. In vectored exception handling, the table of exception handler addresses is in data memory at a known (fixed) address. Change the pipeline to implement this exception handling mechanism. Repeat Exercise 4.30.3 using this modified pipeline and vectored exception handling.
5. We want to emulate vectored exception handling (described in Exercise 4.30.4) on a machine that has only one fixed handler address. Write the code that should be at that fixed address. Hint: this code should identify the exception, get the right address from the exception vector table, and transfer execution to that handler.

Solution

1. For instruction 1 a possible exception could be an invalid target address which can be detected in EX stage. For the instruction 2 a possible exception could be protected memory address which can be detected in MEM stage
2. We need to add this addresses into the PC-Mux to be able to directly jump into the handler. Also we need to be able to detect this exceptions in the appropriate stage. This detector will deliver a signal into the hazard unit that has to flush any incoming affected instruction that has been loaded into the pipeline.
3. As said before when an exception happen is necessary to flush any instructions affected by it, in case of the branch it has to flush all because any further instruction. So if it then load instruction cannot even happen because it has to be replaced by NOP and then it load the handler. You can see it in the diagrams

Instructions / Cycle	1	2	3	4
beqz x11, LABEL	IF	ID	EX	**
lw x11, 0(x12)		IF	ID	**
add x0, x0, x0			IF	**
Handler				IF

4. In that case we need to have a table of saved offset related by an exceptions. Then this offset is added to a common vector exception address. When an exception is asserted by the control unit it has to flush the pipeline and then add the vector register with the corresponding offset and load it into the PC register.
5. In that case we need to assert the same process but by software. In that case we need an instruction that can access the offset or at least the cause of exception and then be able to access to the base exception vector address and then add it to then jump into that address.

Exercise 4.31

In this exercise we compare the performance of 1-issue and 2-issue processors, taking into account program transformations that can be made to optimize for 2-issue execution. Problems in this exercise refer to the following loop (written in C):

```
for(i=0; i!=j; i+=2)
    b[i]=a[i]-a[i+1];
```

A compiler doing little or no optimization might produce the following RISC-V assembly code:

```
addi x12, x0, 0
jal  ENT
```

```

TOP: slli    x5, x12, 2
     add     x6, x10, x5
     lw      x7, 0(x6)
     lw      x29, 4(x6)
     sub     x30, x7, x29
     add     x31, x11, x5
     sw      x30, 0(x31)
     addi    x12, x12, 2
ENT:  bne    x12, x13, TOP

```

The code above uses the following registers:

i	j	a	b	Temporary values
x12	x13	x10	x11	x5–x7, x29–x31

Assume the two-issue, statically scheduled processor for this exercise has the following properties:

1. One instruction must be a memory operation; the other must be an arithmetic/logic instruction or a branch.
2. The processor has all possible forwarding paths between stages (including paths to the ID stage for branch resolution).
3. The processor has perfect branch prediction.
4. Two instructions may not issue together in a packet if one depends on the other. (See page 342.)
5. If a stall is necessary, both instructions in the issue packet must stall. (See page 342.)

As you complete these exercises, notice how much effort goes into generating code that will produce a near-optimal speedup.

1. Draw a pipeline diagram showing how RISC-V code given above executes on the two-issue processor. Assume that the loop exits after two iterations.
2. What is the speedup of going from a one-issue to a two-issue processor? (Assume the loop runs thousands of iterations.)
3. Rearrange/rewrite the RISC-V code given above to achieve better performance on the one-issue processor. Hint: Use the instruction “beqz x13,DONE” to skip the loop entirely if $j = 0$.
4. Rearrange/rewrite the RISC-V code given above to achieve better performance on the two-issue processor. (Do not unroll the loop, however.)
5. Repeat Exercise 4.31.1, but this time use your optimized code from Exercise 4.31.4.
6. What is the speedup of going from a one-issue processor to a two-issue processor when running the optimized code from Exercises 4.31.3 and 4.31.4.
7. Unroll the RISC-V code from Exercise 4.31.3 so that each iteration of the unrolled loop handles two iterations of the original loop. Then, rearrange/rewrite your unrolled code to achieve better performance on the one-issue processor. You may assume that j is a multiple of 4.
8. Unroll the RISC-V code from Exercise 4.31.4 so that each iteration of the unrolled loop handles two iterations of the original loop. Then, rearrange/rewrite your unrolled code to achieve better performance on the two-issue processor. You may assume that j is a multiple of 4. (Hint: Re-organize the loop so that some calculations appear both outside the loop and at the end of the loop. You may assume that the values in temporary registers are not needed after the loop.)
9. What is the speedup of going from a one-issue processor to a two-issue processor when running the unrolled, optimized code from Exercises 4.31.7 and 4.31.8?
10. Repeat Exercises 4.31.8 and 4.31.9, but this time assume the two-issue processor can run two arithmetic/logic instructions together. (In other words, the first instruction in a packet can be any type of instruction, but the second must be an arithmetic or logic instruction. Two memory operations cannot be scheduled at the same time.)

Solution

1. Double issue core pipeline diagram

Instructions/Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
addi x12, x0, 0	IF	ID	EX	ME	WB																						
jal ENT	IF	ID	**	EX	ME	WB																					
bne x12, x13, TOP		IF	**	ID	EX	ME	WB																				
slli x5, x12, 2		IF	**	ID	**	EX	ME	WB																			
add x6, x10, x5				IF	**	ID	EX	ME	WB																		
lw x7, 0(x6)				IF	**	ID	**	EX	ME	WB																	
lw x29, 4(x6)						IF	**	ID	EX	ME	WB																
sub x30, x7, x29						IF	**	ID	**	**	EX	ME	WB														
add x31, x11, x5								IF	**	**	ID	EX	ME	WB													
sw x30, 0(x31)								IF	**	**	ID	**	EX	ME	WB												
addi x12, x12, 2											IF	**	ID	EX	ME	WB											
bne x12, x13, TOP											IF	**	ID	**	EX	ME	WB										
slli x5, x12, 2													IF	**	ID	EX	ME	WB									
add x6, x10, x5													IF	**	ID	**	EX	ME	WB								
lw x7, 0(x6)															IF	**	ID	EX	ME	WB							
lw x29, 4(x6)															IF	**	ID	**	EX	ME	WB						
sub x30, x7, x29																	IF	**	ID	**	EX	ME	WB				
add x31, x11, x5																		**	ID	**	**	EX	ME	WB			
sw x30, 0(x31)																			IF	**	**	ID	EX	ME	WB		
addi x12, x12, 2																			IF	**	**	ID	EX	ME	WB		
bne x12, x13, TOP																						IF	ID	EX	ME	WB	
slli x5, x12, 2																						IF	ID	**	EX	ME	WB

2. There wasn't any speed up gained in this CPU compared to a single issue CPU. In both cases it need 26 cycles to complete 2 loops cycles. The first 2 static instructions, 9 instructions per cycle and one stall caused by a dependency $(2+(9+1)*2)+4 = 26$ cycles.
3. We add "beqz x13,DONE" before anything and because it discard the cycle when j=0 then the instructions "jal ENT" is not necessary anymore. Also we can put "addi x12, x12, 2" before the subtract instruction to avoid the stall caused by the dependency. With these changes we can save 1 loop iterations and the loop pass to complete in 10 cycles to 9 cycles.

```
    beqz    x13, DONE
```

```
    addi    x12, x0, 0
```

TOP:

```
    slli    x5, x12, 2
```

```
    add     x6, x10, x5
```

```
    lw      x7, 0(x6)
```

```
    lw      x29, 4(x6)
```

```
    addi    x12, x12, 2
```

```
    sub     x30, x7, x29
```

```
    add     x31, x11, x5
```

```
    sw      x30, 0(x31)
```

```
    bne     x12, x13, TOP
```

DONE:

4. Is best to arrange to pack together memory and non memory instructions with no dependencies between themselves

```
    beqz    x13, DONE
```

```
    addi    x12, x0, 0
```

TOP:

```
    slli    x5, x12, 2
```

```
    add     x6, x10, x5
```

```
    lw      x7, 0(x6)
```

```
    add     x31, x11, x5
```

```
    lw      x29, 4(x6)
```

```
    addi    x12, x12, 2
```

```
    sub     x30, x7, x29
```

```
    sw      x30, 0(x31)
```

```
    bne     x12, x13, TOP
```

DONE:

5. Multi issue pipeline diagram (optimized)

Instructions/Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
beqz x13, DONE	IF	ID	EX	ME	WB																
addi x12, x0, 0	IF	ID	**	EX	ME	WB															
slli x5, x12, 2		IF	**	ID	EX	ME	WB														
add x6, x10, x5		IF	**	ID	**	EX	ME	WB													
lw x7, 0(x6)				IF	**	ID	EX	ME	WB												
add x31, x11, x5				IF	**	ID	EX	ME	WB												
lw x29, 4(x6)						IF	ID	EX	ME	WB											
addi x12, x12, 2						IF	ID	EX	ME	WB											
sub x30, x7, x29							IF	ID	**	EX	ME	WB									
sw x30, 0(x31)							IF	ID	**	**	EX	ME	WB								
bne x12, x13, TOP								IF	**	**	ID	EX	ME	WB							
slli x5, x12, 2								IF	**	**	ID	**	EX	ME	WB						
add x6, x10, x5											IF	**	ID	EX	ME	WB					
lw x7, 0(x6)											IF	**	ID	**	EX	ME	WB				
add x31, x11, x5													IF	**	ID	EX	ME	WB			
lw x29, 4(x6)													IF	**	ID	EX	ME	WB			
addi x12, x12, 2															IF	ID	EX	ME	WB		
sub x30, x7, x29															IF	ID	**	EX	ME	WB	
sw x30, 0(x31)																IF	**	ID	EX	ME	WB
bne x12, x13, TOP																IF	**	ID	EX	ME	WB

- In a single issue it takes 9 cycles per loop iterations meanwhile with a double issue it takes 7.5 cycles per loop. This give us a speed up of 1.2.
- For unrolling the code we need to change the step up of i. We reused x5, x29 and x30 in order to avoid stalls, so with this rewrite we can do a loop iteration (old 2 loop iterations) using 13 cycles.

```

beqz x13, DONE
addi x12, x0, 0

```

TOP:

```

slli x5, x12, 2
add x6, x10, x5
add x31, x11, x5
lw x7, 0(x6)
lw x29, 4(x6)
lw x5, 8(x6)
lw x30, 12(x6)
addi x12, x12, 4
sub x29, x7, x29
sub x30, x5, x30
sw x29, 0(x31)
sw x30, 8(x31)
bne x12, x13, TOP

```

DONE:

- We need to organize in a way to pack memory and not memory. Is difficult if we need to load the memory value of a[i] because all those instructions causes a chain of dependencies. However this doesn't prevent for us to duplicate those instructions to execute before the loop to foresee this calculations and then on the next iteration do the calculation of the next a[i] before the current loop ends. With this change we can pack every memory instruction with a non memory instruction. Also with this change of the order of the program we need the use of another register (x28)

```

beqz x13, DONE
addi x6, x10, 0

```

```

    addi    x12, x0, 0
TOP:
    lw      x7, 0(x6)
    slli    x5, x12, 2
    lw      x29, 4(x6)
    add     x31, x11, x5
    lw      x30, 8(x6)
    sub     x29, x7, x29
    lw      x28, 12(x6)
    addi    x12, x12, 4
    sw      x29, 0(x31)
    sub     x30, x30, x28
    sw      x30, 8(x31)
    add     x6, x10, x5
    bne     x12, x13, TOP

```

DONE:

9. Well with this rewriting of the program we can achieve one loop iteration (equivalent to 2 old loop iterations) in 9 cycles. In comparison the two issue core is 1.44 time faster than single issue
Multi issue pipeline diagram (unrolled loop)

Instructions/Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
beqz x13, DONE	IF	ID	EX	ME	WB																		
addi x6, x10, 0	IF	ID	**	EX	ME	WB																	
addi x12, x0, 0		IF	**	ID	EX	ME	WB																
lw x7, 0(x6)		IF	**	ID	EX	ME	WB																
slli x5, x12, 2				IF	ID	EX	ME	WB															
lw x29, 4(x6)				IF	ID	EX	ME	WB															
add x31, x11, x5					IF	ID	EX	ME	WB														
lw x30, 8(x6)					IF	ID	EX	ME	WB														
sub x29, x7, x29						IF	ID	EX	ME	WB													
lw x28, 12(x6)						IF	ID	EX	ME	WB													
addi x12, x12, 4							IF	ID	EX	ME	WB												
sw x29, 0(x31)							IF	ID	EX	ME	WB												
sub x30, x30, x28								IF	ID	EX	ME	WB											
sw x30, 8(x31)								IF	ID	**	EX	ME	WB										
add x6, x10, x5									IF	**	ID	EX	ME	WB									
bne x12, x13, TOP									IF	**	ID	**	EX	ME	WB								
lw x7, 0(x6)											IF	**	ID	EX	ME	WB							
slli x5, x12, 2											IF	**	ID	EX	ME	WB							
lw x29, 4(x6)													IF	ID	EX	ME	WB						
add x31, x11, x5													IF	ID	EX	ME	WB						
lw x30, 8(x6)														IF	ID	EX	ME	WB					
sub x29, x7, x29														IF	ID	**	EX	ME	WB				
lw x28, 12(x6)															IF	**	ID	EX	ME	WB			
addi x12, x12, 4															IF	**	ID	EX	ME	WB			
sw x29, 0(x31)																	IF	ID	EX	ME	WB		
sub x30, x30, x28																	IF	ID	EX	ME	WB		
sw x30, 8(x31)																		IF	ID	EX	ME	WB	
add x6, x10, x5																		IF	ID	EX	ME	WB	
bne x12, x13, TOP																			IF	ID	EX	ME	WB

10. /*Pending*/

Exercise 4.32

This exercise explores energy efficiency and its relationship with performance. Problems in this exercise assume the following energy consumption for activity Instruction memory, Registers, and Data memory. You can assume that the other components of the datapath consume a negligible amount of energy. (“Register Read” and “Register Write” refer to the register file only.)

I-Mem	1 Register Read	Register Write	D-Mem Read	D-Mem Write
140pJ	70pJ	60pJ	140pJ	120pJ

Assume that components in the datapath have the following latencies. You can assume that the other components of the datapath have negligible latencies.

I-Mem	Control	Register Read or Write	ALU	D-Mem Read or Write
200 ps	150 ps	90 ps	90 ps	250 ps

1. How much energy is spent to execute an add instruction in a single-cycle design and in the five-stage pipelined design?
2. What is the worst-case RISC-V instruction in terms of energy consumption? What is the energy spent to execute it?
3. If energy reduction is paramount, how would you change the pipelined design? What is the percentage reduction in the energy spent by an lw instruction after this change?
4. What other instructions can potentially benefit from the change discussed in Exercise 4.32.3?
5. How do your changes from Exercise 4.32.3 affect the performance of a pipelined CPU?
6. We can eliminate the MemRead control signal and have the data memory be read in every cycle, i.e., we can permanently have MemRead=1. Explain why the processor still functions correctly after this change. If 25% of instructions are loads, what is the effect of this change on clock frequency and energy consumption?

Solution

1. $E_{ADD} = IMem + 2 * RegRead + RegWrite = 140 \text{ pJ} + 2 * 70 \text{ pJ} + 60 \text{ pJ} = 340 \text{ pJ}$
2. Memory operations are the principal suspects. Actually a load operations consume
 $E_{LW} = IMem + 2 * RegRead + DMemRead + RegWrite = 140 \text{ pJ} + 2 * 70 \text{ pJ} + 140 \text{ pJ} + 60 \text{ pJ} = 480 \text{ pJ}$
3. There's no way to redesign the CPU for LW to consume less. It always need to be fetched, read a register, load data from memory and write back. However we can enable or disable some register reads if it going to use an immediate value instead rs2. Did this change we had a reduction of $70\text{pJ}/480\text{pJ}=0.145$
4. I-Type, S-Type, U-Type (can also had an enable flag for rs1) instructions.
5. Well it can impact because enable or disable reading a register imply to control unit to decode and send the signal to register file so ID will had a time latency of $150\text{ps}+90\text{ps}=240\text{ps}$. However for a pipelined core it doesn't affect clock cycle because still memory stage consume 250ps.
6. It will work because no matter the value of the read signal it will discarded on the write back multiplexer. In term of clock cycle there isn't improvement because we still had to wait for memory to be read. However if the memory is it always working then this add $0.75*140\text{pJ}=105\text{pJ}$ to the consumption for the 250ps of one clock cycle. That means this add $105\text{pJ}/250\text{ps}=0.42\text{W}$ to total consumption.

Exercise 4.33

When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one wire to affect the signal in another. This is called a “cross-talk fault”. A special class of cross-talk faults is when a signal is connected to a wire that has a constant logical value (e.g., a power supply wire). These faults, where the affected signal always has a logical value of either 0

2. MemRead: we cannot be sure because to know it fails we need to pass the multiplexer and the only instruction that enables it is the load instruction but this instruction will assert 1
3. MemtoReg: we can execute a normal operation like "add x5, x6, 12". If has the value of the memory address x6+12 then this signal fails
4. ALUOp: there is no operation assigned to 11 so it probably will be read as 10 and this is only asserted when do an I-type or R-type operation. Now this selection depends of the bits in funct7 and funct3 and we need to read the result from somewhere so the any instruction could perform a test is a load being precise on the selection of the immediate bits so we send an or operation. Then the result will be written in memory
5. MemWrite: we can execute "add x11, x12, x13". We need to know the result before because it will be the address where we need to read the memory (which will be x12+13). If the memory have been written with the value of x12(rs2) then we know the signal fails.
6. ALUSrc: we do an R-type instruction and see if use rs2 instead immediate value
7. RegWrite: we check if writes on operations like store whom doesn't write to register
3. If only fails one signal at the time yes, we can convert normal instructions onto an equivalent instructions. To achieve that we need to write a compiler that can issue all the collateral fails of a signal stuck
4. The only instruction that can prove and is affected by this is a load instruction because it isn't a branch instruction the signal Branch is set at 0 and this instruction need MemRead be 1
5. The only kind of instructions that trigger RegRd are R-Type instructions so in theory if it trigger MemRead it doesn't affect because the read will be discarded. However if it always on then you can make an operation like add x1, x0, x0 which will try to read the memory address of 0 which can be protected and it has to trigger a fatal exception.

Chapter 5

Exercise 5.1

In this exercise we look at memory locality properties of matrix computation. The following code is written in C, where elements within the same row are stored contiguously. Assume each word is a 64-bit integer.

```
for (I=0; I<8; I++)  
    for (J=0; J<8000; J++)  
        A[I][J]=B[I][0]+A[J][I];
```

1. How many 64-bit integers can be stored in a 16-byte cache block?
2. Which variable references exhibit temporal locality?
3. Which variable references exhibit spatial locality?

Locality is affected by both the reference order and data layout. The same computation can also be written below in Matlab, which differs from C in that it stores matrix elements within the same column contiguously in memory.

```
for I=1:8  
    for J=1:8000  
        A(I,J)=B(I,0)+A(J,I);  
    end  
end
```

4. Which variable references exhibit temporal locality?
5. Which variable references exhibit spatial locality?
6. How many 16-byte cache blocks are needed to store all 64-bit matrix elements being referenced using Matlab's matrix storage? How many using C's matrix storage? (Assume each row contains more than one element.)

Solution

1. 64-bits are 8 bytes so only 2 variables can be stored on the cache
2. B[I][0], I and J. B and I are values that not changes the whole J loop and they are needed. J in other part is constantly accessed
3. A[I][J]. Values inside the same row are near of each other
4. B(I,0), I and J. It still this for the same reasons
5. A(J,I). Now this is the variable with spatial locality because now values inside the same column are near each other
6. Matrix A access $8 \times 8000 = 64000$ different values inside. If we divide them by the number of values per block give us 32000 blocks of memory needed for store A in both languages. Now for B elements if we follow C rules each element is individual form themselves so they are stored in 8 different blocks, now for Matlab rules we have all elements are contiguous so it need to be stored on $8/2=4$ blocks. So in total C use 32008 and Matlab 32004.

Exercise 5.2

Caches are important to providing a high-performance memory hierarchy to processors. Below is a list of 64-bit memory address references, given as word addresses.

0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58, 0xbe, 0x0e, 0xb5, 0x2c, 0xba, 0xfd

1. For each of these references, identify the binary word address, the tag, and the index given a direct-mapped cache with 16 one-word blocks. Also list whether each reference is a hit or a miss, assuming the cache is initially empty.
2. For each of these references, identify the binary word address, the tag, the index, and the offset given a direct-mapped cache with two-word blocks and a total size of eight blocks. Also list if each reference is a hit or a miss, assuming the cache is initially empty.

3. You are asked to optimize a cache design for the given references. There are three direct-mapped cache designs possible, all with a total of eight words of data:
- C1 has 1-word blocks
 - C2 has 2-word blocks
 - C3 has 4-word blocks

Solution

1. For this configuration we had 4 bits of of index and 4 bits of tag values

Word address	Binary value		Reference
	Tag	Index	
0x03	0000	0011	Miss
0xB4	1011	0100	Miss
0x2B	0010	1011	Miss
0x02	0000	0010	Miss
0xBF	1011	1111	Miss
0x58	0101	1000	Miss
0xBE	1011	1110	Miss
0x0E	0000	1110	Miss
0xB5	1011	0101	Miss
0x2C	0010	1100	Miss
0xBA	1011	1010	Miss
0xFD	1111	1101	Miss

2. Now in this configuration we have 4 bits tag 3 bits index and 1 bit offset

Word address	Binary value			Reference
	Tag	Index	Offset	
0x03	0000	001	1	Miss
0xB4	1011	010	0	Miss
0x2B	0010	101	1	Miss
0x02	0000	001	0	Hit
0xBF	1011	111	1	Miss
0x58	0101	100	0	Miss
0xBE	1011	111	0	Hit
0x0E	0000	111	0	Miss
0xB5	1011	010	1	Hit
0x2C	0010	110	0	Miss
0xBA	1011	101	0	Miss
0xFD	1111	110	1	Miss

3. Now with 8 words of data we have 5 bits of tag and 3 bits between index and offset. With this set of data we can conclude that C2 is better

Word addresses	Tag	Index / Offset C1	Reference	Index / Offset C2	Reference	Index / Offset C3	Reference
0x03	00000	011	Miss	01_1	Miss	0_11	Miss
0xB4	10110	100	Miss	10_0	Miss	1_00	Miss
0x2B	00101	011	Miss	01_1	Miss	0_11	Miss
0x02	00000	010	Miss	01_0	Miss	0_10	Miss
0xBF	10111	111	Miss	11_1	Miss	1_11	Miss
0x58	01011	000	Miss	00_0	Miss	0_00	Miss
0xBE	10111	110	Miss	11_0	Hit	1_10	Hit
0x0E	00001	110	Miss	11_0	Miss	1_10	Miss
0xB5	10110	101	Miss	10_1	Hit	1_01	Miss
0x2C	00101	100	Miss	10_0	Miss	1_00	Miss
0xBA	10111	010	Miss	01_0	Miss	0_10	Miss
0xFD	11111	101	Miss	10_1	Miss	1_01	Miss

Exercise 5.3

By convention, a cache is named according to the amount of data it contains (i.e., a 4 KiB cache can hold 4 KiB of data); however, caches also require SRAM to store metadata such as tags and valid bits. For this exercise, you will examine how a cache's configuration affects the total amount of SRAM needed to implement it as well as the performance of the cache. For all parts, assume that the caches are direct-mapped, byte addressable, and that addresses and the words are 32 bits.

1. Calculate the total number of bits required to implement a 32 KiB cache with two-word blocks.
2. Calculate the total number of bits required to implement a 64 KiB cache with 16-word blocks. How much bigger is this cache than the 32 KiB cache described in Exercise 5.3.1? (Notice that, by changing the block size, we doubled the amount of data without doubling the total size of the cache.)
3. Explain why this 64 KiB cache, despite its larger data size, might provide slower performance than the first cache.
4. Generate a series of read requests that have a lower miss rate on a 32 KiB two-way set associative cache than on the cache described in Exercise 5.3.1.

Solution

$$\text{Index} = \log_2(32 \text{ KiB} / (2 * 4 \text{ B})) = \log_2(4 \text{ Ki}) = \log_2(2^{12}) = 12 \text{ bits}$$

$$1. \text{ Offset} = \log_2(2 * 4) = 3 \text{ bits}$$

$$\text{Tag} = 32 - (12 + 3) = 17 \text{ bits}$$

$$\text{TotalBits} = 2^{12} * (17 + 1 + 2 * 32) = 335872 \text{ bits}$$

$$\text{Index} = \log_2(64 \text{ KiB} / (16 * 4 \text{ B})) = \log_2(1 \text{ Ki}) = \log_2(2^{10}) = 10 \text{ bits}$$

$$2. \text{ Offset} = \log_2(16 * 4) = 6 \text{ bits}$$

$$\text{Tag} = 32 - (10 + 6) = 16 \text{ bits}$$

$$\text{TotalBits} = 2^{10} * (16 + 1 + 16 * 32) = 541696 \text{ bits}$$

$$\text{Increase} = 541696 / 335872 = 1.61 \text{ times bigger}$$

3. Because the data size of the block is bigger and can cause collision misses. Also can impact a higher hit time and time penalty
4. On the first cache we have 4096 addresses so any series of read that are separated by 4096 Bytes will start to make collision. In comparison a 2 way set association will perform better because can handle 2 different values mapped to the same index

Exercise 5.4

Section 5.3 shows the typical method to index a direct-mapped cache, specifically (Block address) modulo (Number of blocks in the cache). Assuming a 32-bit address and 1024 blocks in the cache, consider a different indexing function, specifically (Block address[31:22] XOR Block address[21:12]). Is it possible to use this to index a direct-mapped cache? If so, explain why and discuss any changes that might need to be made to the cache. If it is not possible, explain why.

Solution

Yes it is possible to create a direct mapped cache. However this doesn't seem like a good idea because it breaks spatial locality.

Exercise 5.5

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
63-10	9-5	4-0

1. What is the cache block size (in words)?
2. How many blocks does the cache have?
3. What is the ratio between total bits required for such a cache implementation over the data storage bits?

Beginning from power on, the following byte-addressed cache references are recorded.

Address												
Hex	00	04	10	84	E8	A0	400	1E	8C	C1C	B4	884
Dec	0	4	16	132	232	160	1024	30	140	3100	180	2180

4. For each reference, list (1) its tag, index, and offset, (2) whether it is a hit or a miss, and (3) which bytes were replaced (if any).
5. What is the hit ratio?
6. List the final state of the cache, with each valid entry represented as a record of <index, tag, data>. For example, <0, 3, Mem[0xC00]-Mem[0xC1F]>

Solution

1. 8 Words
2. 32 Blocks
3.
$$\frac{\text{TotalBits}}{\text{StorageBits}} = \frac{2^5 * (54 + 1 + 8 * 32 \text{ bits})}{2^5 * 2^5 * 8 \text{ bits}} = 9952 / 8192 = 1.2148$$
4. Reference list

Word addresses	Tag	Index	Offset	Reference	Replace
0x00	00	00000	00000	Miss	
0x04	00	00000	00100	Hit	
0x10	00	00000	10000	Hit	
0x84	00	00100	00100	Miss	

0xE8	00	00111	01000	Miss	
0xA0	00	00101	00000	Miss	
0x400	01	00000	00000	Miss	0x00-0x1F
0x1E	00	00000	11110	Miss	0x400-0x41F
0x8C	00	00100	01100	Hit	
0xC1C	11	00000	11100	Miss	0x00-0x1F
0xB4	00	00101	10100	Hit	
0x884	10	00100	00100	Miss	0x80-0x9F

5. $HitRate = \frac{NumberHits}{TotalRef} = \frac{4}{12} = 0.33$
6. $\langle 0, 3, Mem[0xC00]-Mem[0xC1F] \rangle$
 $\langle 4, 2, Mem[0x880]-Mem[0x89F] \rangle$
 $\langle 5, 0, Mem[0xA0]-Mem[0xBF] \rangle$
 $\langle 7, 0, Mem[0xE0]-Mem[0xFF] \rangle$

Exercise 5.6

Recall that we have two write policies and two write allocation policies, and their combinations can be implemented either in L1 or L2 cache. Assume the following choices for L1 and L2 caches:

L1	L2
Write through, non-write allocate	Write back, write allocate

- Buffers are employed between different levels of memory hierarchy to reduce access latency. For this given configuration, list the possible buffers needed between L1 and L2 caches, as well as L2 cache and memory.
- Describe the procedure of handling an L1 write-miss, considering the components involved and the possibility of replacing a dirty block.
- For a multilevel exclusive cache configuration (a block can only reside in one of the L1 and L2 caches), describe the procedures of handling an L1 write-miss and an L1 read-miss, considering the components involved and the possibility of replacing a dirty block.

Solution

- For buffers it only makes sense write buffers and it can be really useful one from L1 to L2 for the write through policy. Also a write buffer between L2 and main memory can be useful.
- When a write miss happen then it will try to write directly to L2 without allocating data. If L2 has a write miss then it will check to a dirty bit (if a data has been written before). If the block has a dirty bit then it will write the data to main memory and then will load the value of the requested memory and the it will write the value to L2 block.
- In that case then if L1 has a write miss then it checks into L2. If the data is in L2 then if it has a dirty bit it must be written in main memory, then the next step is to copy L2 block inside L1 and then change the valid bit of L2 block to 0. In a miss read try to read it from L2, if it is there then will copy to L1 and change valid bit of L2 to 0. If L2 has a read miss the best option then is to copy from main memory directly to L1 because the copy and invalid bit will repeat trough all the memory hierarchy chain.

Exercise 5.7

Consider the following program and cache behaviours.

Data Reads per 1000 Instructions	Data Writes per 1000 Instructions	Instruction Cache Miss Rate	Data Cache Miss Rate	Block Size (bytes)
250	100	0.30%	2%	64

1. Suppose a CPU with a write-through, write-allocate cache achieves a CPI of 2. What are the read and write bandwidths (measured by bytes per cycle) between RAM and the cache? (Assume each miss generates a request for one block.)
2. For a write-back, write-allocate cache, assuming 30% of replaced data cache blocks are dirty, what are the read and write bandwidths needed for a CPI of 2?

Solution

1. For read bandwidth we had to know the amount of data and instructions reads in a miss. However this cache do write-allocation so when a data write miss happen it reads from RAM to allocate to cache so it adds read bandwidth. Also instructions read are performed always (Because every cycle the CPU fetch an instruction).

For write bandwidth because the cache do a write-through then every write is done to RAM directly. However these writes are performed from the CPU at CPU bandwidth, in this case 32 bits.

$$ReadBandwidth = \frac{1}{CPI} * ((DataRead + DataWrite) * DataMiss + InstrRead * InstrMiss) * BlockSize$$

$$ReadBandwidth = 0.5 \frac{Instr}{Cycle} * \left(\frac{250+100}{1000} \frac{Reads}{Instr} * 0.02 + \frac{1000}{1000} \frac{Reads}{Instr} * 0.003 \right) * 64 \frac{Bytes}{Read}$$

$$ReadBandwidth = 0.32 \frac{Bytes}{Cycle}$$

$$WriteBandwidth = \frac{1}{CPI} * DataWrite * CPUWriteSize$$

$$WriteBandwidth = 0.5 \frac{Instr}{Cycle} * \frac{100}{1000} \frac{Reads}{Instr} * 4 \frac{Bytes}{Read}$$

$$WriteBandwidth = 0.2 \frac{Bytes}{Cycle}$$

2. The read bandwidth remain the same. However write bandwidth change drastically, Now because we do a write back then it only writes to main memory when replace a dirty page and it happens with read or write cache miss so it writes the whole block to RAM

$$WriteBandwidth = \frac{1}{CPI} * (DataWrite + DataRead) * DataMiss * DirtyPage * BlockSize$$

$$ReadBandwidth = 0.5 \frac{Instr}{Cycle} * \frac{100+250}{1000} \frac{Reads}{Instr} * 0.02 * 0.3 * 64 \frac{Bytes}{Read}$$

$$ReadBandwidth = 0.0672 \frac{Bytes}{Cycle}$$

Exercise 5.8

Media applications that play audio or video files are part of a class of workloads called “streaming” workloads (i.e., they bring in large amounts of data but do not reuse much of it). Consider a video streaming workload that accesses a 512 KiB working set sequentially with the following word address stream:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 . . .

1. Assume a 64 KiB direct-mapped cache with a 32-byte block. What is the miss rate for the address stream above? How is this miss rate sensitive to the size of the cache or the working set? How would you categorize the misses this workload is experiencing, based on the 3C model?
2. Re-compute the miss rate when the cache block size is 16 bytes, 64 bytes, and 128 bytes. What kind of locality is this workload exploiting?

- “Prefetching” is a technique that leverages predictable address patterns to speculatively bring in additional cache blocks when a particular cache block is accessed. One example of prefetching is a stream buffer that prefetches sequentially adjacent cache blocks into a separate buffer when a particular cache block is brought in. If the data are found in the prefetch buffer, it is considered as a hit, moved into the cache, and the next cache block is prefetched. Assume a two-entry stream buffer; and, assume that the cache latency is such that a cache block can be loaded before the computation on the previous cache block is completed. What is the miss rate for the address stream above?

Solution

- These misses will enter in compulsory misses because all the data will just passing into the blocks but it wont be used again.

$$MissRate = \frac{1}{(32/4)} = 0.125$$

- $MissRate = \frac{1}{(16/4)} = 0.25$

$$MissRate = \frac{1}{(64/4)} = 0.0625$$

$$MissRate = \frac{1}{(128/4)} = 0.03125$$

- It will be of 0 because if it prefetch the next word then this will follow up the stream

Exercise 5.9

Cache block size (B) can affect both miss rate and miss latency. Assuming a machine with a base CPI of 1, and an average of 1.35 references (both instruction and data) per instruction, find the block size that minimizes the total miss latency given the following miss rates for various block sizes.

8: 4%	16: 3%	32: 2%	64: 1.5%	128: 1%
-------	--------	--------	----------	---------

- What is the optimal block size for a miss latency of $20 \times B$ cycles?
- What is the optimal block size for a miss latency of $24 + B$ cycles?
- For constant miss latency, what is the optimal block size?

Solution

- The best option is the block size of 8

$$AMAT_8 = 1.35 + 0.04 * (20 * 8) = 7.75$$

$$AMAT_{16} = 1.35 + 0.03 * (20 * 16) = 10.95$$

$$AMAT_{32} = 1.35 + 0.02 * (20 * 32) = 14.15$$

$$AMAT_{64} = 1.35 + 0.015 * (20 * 64) = 20.55$$

$$AMAT_{128} = 1.35 + 0.01 * (20 * 128) = 26.95$$

- The best option is the block size of 32

$$AMAT_8 = 1.35 + 0.04 * (24 + 8) = 2.63$$

$$AMAT_{16} = 1.35 + 0.03 * (24 + 16) = 2.55$$

$$AMAT_{32} = 1.35 + 0.02 * (24 + 32) = 2.47$$

$$AMAT_{64} = 1.35 + 0.015 * (24 + 64) = 2.67$$

$$AMAT_{128} = 1.35 + 0.01 * (24 + 128) = 2.87$$

- In the same logic because it will be a constant then the cache that give the lowest miss rate will be the best. In this case the block size of 128

Exercise 5.10

In this exercise, we will look at the different ways capacity affects overall performance. In general, cache access time is proportional to capacity. Assume that main memory accesses take 70 ns and that 36% of all instructions access data memory. The following table shows data for L1 caches attached to each of two processors, P1 and P2.

	L1 Size	L1 Miss Rate	L1 Hit Time
P1	2 KiB	8.0%	0.66 ns
P2	4 KiB	6.0%	0.90 ns

1. Assuming that the L1 hit time determines the cycle times for P1 and P2, what are their respective clock rates?
2. What is the Average Memory Access Time for P1 and P2 (in cycles)?
3. Assuming a base CPI of 1.0 without any memory stalls, what is the total CPI for P1 and P2? Which processor is faster? (When we say a “base CPI of 1.0”, we mean that instructions complete in one cycle, unless either the instruction access or the data access causes a cache miss.)

For the next three problems, we will consider the addition of an L2 cache to P1 (to presumably make up for its limited L1 cache capacity). Use the L1 cache capacities and hit times from the previous table when solving these problems. The L2 miss rate indicated is its local miss rate.

L2 Size	L2 Miss Rate	L2 Hit Time
1 MiB	95%	5.62 ns

4. What is the AMAT for P1 with the addition of an L2 cache? Is the AMAT better or worse with the L2 cache?
5. Assuming a base CPI of 1.0 without any memory stalls, what is the total CPI for P1 with the addition of an L2 cache?
6. What would the L2 miss rate need to be in order for P1 with an L2 cache to be faster than P1 without an L2 cache?
7. What would the L2 miss rate need to be in order for P1 with an L2 cache to be faster than P2 without an L2 cache?

Solution

$$1. \quad CR_{P1} = \frac{1}{CC_{P1}} = \frac{1}{(0.66 * 10^{-9} \text{ sec})} = 1.5151 \text{ GHz}$$

$$CR_{P2} = \frac{1}{CC_{P2}} = \frac{1}{(0.90 * 10^{-9} \text{ sec})} = 1.1111 \text{ GHz}$$

$$2. \quad AMAT_{P1L1} = \left\lceil \frac{0.66 \text{ ns}}{0.66 \text{ ns}} \right\rceil + 0.08 * \left\lceil \frac{70 \text{ ns}}{0.66 \text{ ns}} \right\rceil = 1 + 0.08 * 107 = 9.56 \text{ cycles}$$

$$AMAT_{P2L1} = \left\lceil \frac{0.90 \text{ ns}}{0.90 \text{ ns}} \right\rceil + 0.06 * \left\lceil \frac{70 \text{ ns}}{0.90 \text{ ns}} \right\rceil = 1 + 0.06 * 78 = 5.68 \text{ cycles}$$

$$CPI = CPI_{Base} + InstrMemStall + DataMemStall$$

$$3. \quad CPI_{P1L1} = 1 + 0.08 * 107 + 0.36 * 0.08 * 107 = 12.64 \text{ cycles}$$

$$CPI_{P2L2} = 1 + 0.06 * 78 + 0.36 * 0.06 * 78 = 7.3648 \text{ cycles}$$

$$ET_{P1L1} = Instr * CPI_{P1} * CC_{P1} = \frac{Instr * 12.64 \text{ cycles}}{1.5151 \text{ GHz}} = 8.3426 \frac{\text{ns}}{\text{instr}}$$

$$ET_{P2L1} = Instr * CPI_{P2} * CC_{P2} = \frac{Instr * 7.3648 \text{ cycles}}{1.1111 \text{ GHz}} = 6.6283 \frac{\text{ns}}{\text{instr}}$$

Even if P1 has a greater clock rate is slower than P2

$$4. \quad AMAT_{P1L2} = \left\lceil \frac{0.66 \text{ ns}}{0.66 \text{ ns}} \right\rceil + 0.08 * \left(\left\lceil \frac{5.62 \text{ ns}}{0.66 \text{ ns}} \right\rceil + 0.95 * \left\lceil \frac{70 \text{ ns}}{0.66 \text{ ns}} \right\rceil \right) = 1 + 0.08 * (9 + 0.95 * 107)$$

$$AMAT_{P1L2} = 9.852 \text{ cycles}$$

Is worse but for a little bit because even if the mean time of a miss in L2 it compensate that L1 miss rate is small.

$$5. \text{CPI}_{P1L2} = 1 + 0.08 * (9 + 0.95 * 107) + 0.36 * 0.08 * (9 + 0.95 * 107) = 13.03872 \text{ cycles}$$

$$ET_{P1L1} > ET_{P1L2}$$

$$8.3426 > \frac{1 + 0.08 * (9 + X * 107) + 0.36 * 0.08 * (9 + X * 107)}{1.5151}$$

$$6. \quad 8.3426 * 1.5151 > 1 + 0.72 + 8.56 * X + 0.2592 + 3.0816 * X$$

$$X < \frac{8.3426 * 1.5151 - 1 - 0.72 - 0.2592}{8.56 + 3.0816} = 0.91573$$

$$ET_{P2L1} > ET_{P1L2}$$

$$6.6283 > \frac{1 + 0.08 * (9 + X * 107) + 0.36 * 0.08 * (9 + X * 107)}{1.5151}$$

$$7. \quad 6.6283 * 1.5151 > 1 + 0.72 + 8.56 * X + 0.2592 + 3.0816 * X$$

$$X < \frac{6.6283 * 1.5151 - 1 - 0.72 - 0.2592}{8.56 + 3.0816} = 0.69263$$

Exercise 5.11

This exercise examines the effect of different cache designs, specifically comparing associative caches to the direct-mapped caches from Section 5.4. For these exercises, refer to the sequence of word address shown below.

0x03, 0xb4, 0x2b, 0x02, 0xbe, 0x58, 0xbf, 0x0e, 0x1f, 0xb5, 0xbf, 0xba, 0x2e, 0xce

1. Sketch the organization of a three-way set associative cache with two-word blocks and a total size of 48 words. Your sketch should have a style similar to Figure 5.18, but clearly show the width of the tag and data fields.
2. Trace the behaviour of the cache from Exercise 5.11.1. Assume a true LRU replacement policy. For each reference, identify
 - The binary word address
 - The tag
 - The index
 - The offset
 - Whether the reference is a hit or a miss
 - Which tags are in each way of the cache after the reference has been handled
3. Sketch the organization of a fully associative cache with one word blocks and a total size of eight words. Your sketch should have a style similar to Figure 5.18, but clearly show the width of the tag and data fields.
4. Trace the behaviour of the cache from Exercise 5.11.3. Assume a true LRU replacement policy. For each reference, identify
 - The binary word address
 - The tag
 - The index
 - The offset
 - Whether the reference is a hit or a miss
 - The contents of the cache after each reference has been handled
5. Sketch the organization of a fully associative cache with two word blocks and a total size of eight words. Your sketch should have a style similar to Figure 5.18, but clearly show the width of the tag and data fields
6. Trace the behaviour of the cache from Exercise 5.11.5. Assume an LRU replacement policy. For each reference, identify
 - The binary word address
 - The tag
 - The index
 - The offset

- Whether the reference is a hit or a miss
 - The contents of the cache after each reference has been handled
7. Repeat Exercise 5.11.6 using MRU (most recently used) replacement
 8. Repeat Exercise 5.11.6 using the optimal replacement policy (i.e., the one that gives the lowest miss rate).

Solution

1. With this organization will had 1 bit offset, $\log_2(48 \text{ words} / (3 \text{ Sets} * 2 \text{ Words})) = 3$ bits of indexing and $32 - 1 - 3 = 28$ bits of tag.
2. 3 Way cache 2 word block LRU policy

Word Address	Binary Address	Tag	Index	Offset	Reference	Way 1	Way 2	Way3
0x03	0000_0011	0000	001	1	Miss	T(1)=0x0		
0xB4	1011_0100	1011	010	0	Miss	T(1)=0x0 T(2)=0xB		
0x2B	0010_1011	0010	101	1	Miss	T(1)=0x0 T(2)=0xB T(5)=0x2		
0x02	0000_0010	0000	001	0	Hit	T(1)=0x0 T(2)=0xB T(5)=0x2		
0xBE	1011_1110	1011	111	0	Miss	T(1)=0x0 T(2)=0xB T(5)=0x2 T(7)=0xB		
0x58	0101_1000	0101	100	0	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(7)=0xB		
0xBF	1011_1111	1011	111	1	Hit	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(7)=0xB		
0x0E	0000_1110	0000	111	0	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(7)=0xB	T(7)=0x0	
0x1F	0001_1111	0001	111	1	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(7)=0xB	T(7)=0x0	T(7)=0x1
0xB5	1011_0101	1011	010	1	Hit	T(1)=0x0 T(2)=0xB	T(7)=0x0	T(7)=0x1

						T(4)=0x5 T(5)=0x2 T(7)=0xB		
0xBF	1011_1111	1011	111	1	Hit	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(7)=0xB	T(7)=0x0	T(7)=0x1
0xBA	1011_1010	1011	101	0	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(6)=0xB T(7)=0xB	T(5)=0xB T(7)=0x0	T(7)=0x1
0x2E	0010_1110	0010	111	0	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(6)=0xB T(7)=0xB	T(5)=0xB T(7)=0x2	T(7)=0x1
0xCE	1100_1110	1100	111	0	Miss	T(1)=0x0 T(2)=0xB T(4)=0x5 T(5)=0x2 T(6)=0xB T(7)=0xB	T(5)=0xB T(7)=0x2	T(7)=0xC

- On this configuration will no had an offset either index. In that case we had the whole address as tag bits. This cache had 8 blocks.
- Fully association 1 word block cache LRU policy

Word Address	Binary Address	Tag	Reference	Cache Data
0x03	0000_0011	0000_0011	Miss	0x03
0xB4	1011_0100	1011_0100	Miss	0x03, 0xB4
0x2B	0010_1011	0010_1011	Miss	0x03, 0xB4, 0x2B
0x02	0000_0010	0000_0010	Miss	0x03, 0xB4, 0x2B, 0x02
0xBE	1011_1110	1011_1110	Miss	0x03, 0xB4, 0x2B, 0x02, 0xBE
0x58	0101_1000	0101_1000	Miss	0x03, 0xB4, 0x2B, 0x02, 0xBE, 0x58
0xBF	1011_1111	1011_1111	Miss	0x03, 0xB4, 0x2B, 0x02, 0xBE, 0x58, 0xBF
0x0E	0000_1110	0000_1110	Miss	0x03 , 0xB4, 0x2B, 0x02, 0xBE, 0x58, 0xBF, 0x0E
0x1F	0001_1111	0001_1111	Miss	0xB4 , 0x2B, 0x02, 0xBE,

				0x58, 0xBF, 0x0E, 0x1F
0xB5	1011_0101	1011_0101	Miss	0x2B , 0x02, 0xBE, 0x58, 0xBF, 0x0E, 0x1F, 0xB5
0xBF	1011_1111	1011_1111	Hit	0x2B , 0x02, 0xBE, 0x58, 0x0E, 0x1F, 0xB5, 0xBF
0xBA	1011_1010	1011_1010	Miss	0x02 , 0xBE, 0x58, 0x0E, 0x1F, 0xB5, 0xBF, 0xBA
0x2E	0010_1110	0010_1110	Miss	0xBE , 0x58, 0x0E, 0x1F, 0xB5, 0xBF, 0xBA, 0x2E
0xCE	1100_1110	1100_1110	Miss	0x58 , 0x0E, 0x1F, 0xB5, 0xBF, 0xBA, 0x2E, 0xCE

5. It will be similar as the previous model. But in this will had 7 tag bits and 1 offset bit. This cache had 4 blocks.
6. Fully association 2 word block cache LRU policy

Word Address	Binary Address	Tag	Offset	Reference	Cache Data
0x03	0000_0011	0000_001	1	Miss	[0x02, 0x03]
0xB4	1011_0100	1011_010	0	Miss	[0x02, 0x03], [0xB4, 0xB5]
0x2B	0010_1011	0010_101	1	Miss	[0x02, 0x03], [0xB4, 0xB5], [0x2A, 0x2B]
0x02	0000_0010	0000_001	0	Hit	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03]
0xBE	1011_1110	1011_111	0	Miss	[0xB4, 0xB5] , [0x2A, 0x2B], [0x02, 0x03], [0xBE, 0xBF]
0x58	0101_1000	0101_100	0	Miss	[0x2A, 0x2B] , [0x02, 0x03], [0xBE, 0xBF], [0x58, 0x59]
0xBF	1011_1111	1011_111	1	Hit	[0x2A, 0x2B] , [0x02, 0x03], [0x58, 0x59], [0xBE, 0xBF]
0x0E	0000_1110	0000_111	0	Miss	[0x02, 0x03] , [0x58, 0x59], [0xBE, 0xBF], [0x0E, 0x0F]
0x1F	0001_1111	0001_111	1	Miss	[0x58, 0x59] , [0xBE, 0xBF], [0x0E, 0x0F], [0x1E, 0x1F]
0xB5	1011_0101	1011_010	1	Miss	[0xBE, 0xBF] , [0x0E, 0x0F], [0x1E, 0x1F], [0xB4, 0xB5]
0xBF	1011_1111	1011_111	1	Hit	[0x0E, 0x0F] , [0x1E, 0x1F], [0xB4, 0xB5], [0xBE, 0xBF]
0xBA	1011_1010	1011_101	0	Miss	[0x1E, 0x1F] , [0xB4, 0xB5], [0xBE, 0xBF], [0xBA, 0xBB]
0x2E	0010_1110	0010_111	0	Miss	[0xB4, 0xB5] , [0xBE, 0xBF], [0xBA, 0xBB], [0x2E, 0x2F]
0xCE	1100_1110	1100_111	0	Miss	[0xBE, 0xBF] , [0xBA, 0xBB],

					[0x2E, 0x2F], [0xCE, 0xCF]
--	--	--	--	--	----------------------------

7. Fully association 2 word block cache MRU policy

Word Address	Binary Address	Tag	Offset	Reference	Cache Data
0x03	0000_0011	0000_001	1	Miss	[0x02, 0x03]
0xB4	1011_0100	1011_010	0	Miss	[0x02, 0x03], [0xB4, 0xB5]
0x2B	0010_1011	0010_101	1	Miss	[0x02, 0x03], [0xB4, 0xB5], [0x2A, 0x2B]
0x02	0000_0010	0000_001	0	Hit	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03]
0xBE	1011_1110	1011_111	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03], [0xBE, 0xBF]
0x58	0101_1000	0101_100	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03], [0x58, 0x59]
0xBF	1011_1111	1011_111	1	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03], [0xBE, 0xBF]
0x0E	0000_1110	0000_111	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03], [0x0E, 0x0F]
0x1F	0001_1111	0001_111	1	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F]
0xB5	1011_0101	1011_010	1	Hit	[0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F], [0xB4, 0xB5]
0xBF	1011_1111	1011_111	1	Miss	[0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F], [0xBE, 0xBF]
0xBA	1011_1010	1011_101	0	Miss	[0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F], [0xBA, 0xBB]
0x2E	0010_1110	0010_111	0	Miss	[0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F], [0x2E, 0x2F]
0xCE	1100_1110	1100_111	0	Miss	[0x2A, 0x2B], [0x02, 0x03], [0x1E, 0x1F], [0xCE, 0xCF]

8. Fully association with a custom replacement policy

Word Address	Binary Address	Tag	Offset	Reference	Cache Data
0x03	0000_0011	0000_001	1	Miss	[0x02, 0x03]
0xB4	1011_0100	1011_010	0	Miss	[0x02, 0x03], [0xB4, 0xB5]
0x2B	0010_1011	0010_101	1	Miss	[0x02, 0x03], [0xB4, 0xB5], [0x2A, 0x2B]
0x02	0000_0010	0000_001	0	Hit	[0xB4, 0xB5], [0x2A, 0x2B], [0x02, 0x03]
0xBE	1011_1110	1011_111	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B],

					[0x02, 0x03] , [0xBE, 0xBF]
0x58	0101_1000	0101_100	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x58, 0x59] , [0xBE, 0xBF]
0xBF	1011_1111	1011_111	1	Hit	[0xB4, 0xB5], [0x2A, 0x2B], [0x58, 0x59] , [0xBE, 0xBF]
0x0E	0000_1110	0000_111	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x0E, 0x0F] , [0xBE, 0xBF]
0x1F	0001_1111	0001_111	0	Miss	[0xB4, 0xB5], [0x2A, 0x2B], [0x1E, 0x1F] , [0xBE, 0xBF]
0xB5	1011_0101	1011_010	1	Hit	[0xB4, 0xB5], [0x2A, 0x2B] , [0x1E, 0x1F], [0xBE, 0xBF]
0xBF	1011_1111	1011_111	1	Hit	[0xB4, 0xB5], [0x2A, 0x2B] , [0x1E, 0x1F], [0xBE, 0xBF]
0xBA	1011_1010	1011_101	0	Miss	[0xB4, 0xB5], [0xBA, 0xBB] , [0x1E, 0x1F], [0xBE, 0xBF]
0x2E	0010_1110	0010_111	0	Miss	[0xB4, 0xB5], [0x2E, 0x2F] , [0x1E, 0x1F], [0xBE, 0xBF]
0xCE	1100_1110	1100_111	0	Miss	[0xB4, 0xB5], [0xCE, 0xCF] , [0x1E, 0x1F], [0xBE, 0xBF]

Exercise 5.12

Multilevel caching is an important technique to overcome the limited amount of space that a first-level cache can provide while still maintaining its speed. Consider a processor with the following parameters:

Base CPI, No Memory Stalls	Processor Speed	Main Memory Access Time	First-Level Cache Miss Rate per Instruction	Second-Level Cache, Direct-Mapped Speed	Miss Rate with Second-Level Cache, Direct-Mapped	Second-Level Cache, Eight-Way Set Associative Speed	Miss Rate with Second-Level Cache, Eight-Way Set Associative
1.5	2 GHz	100ns	7%	12 cycles	3.5%	28 cycles	1.5%

**First Level Cache miss rate is per instruction. Assume the total number of L1 cache misses (instruction and data combined) is equal to 7% of the number of instructions.

1. Calculate the CPI for the processor in the table using: 1) only a first-level cache, 2) a second-level direct-mapped cache, and 3) a second level eight-way set associative cache. How do these numbers change if main memory access time doubles? (Give each change as both an absolute CPI and a percent change.) Notice the extent to which an L2 cache can hide the effects of a slow memory
2. It is possible to have an even greater cache hierarchy than two levels? Given the processor above with a second-level, direct-mapped cache, a designer wants to add a third-level cache that takes 50 cycles to access and will have a 13% miss rate. Would this provide better performance? In general, what are the advantages and disadvantages of adding a third-level cache?
3. In older processors, such as the Intel Pentium or Alpha 21264, the second level of cache was external (located on a different chip) from the main processor and the first-level cache. While this allowed for large second-level caches, the latency to access the cache was much higher, and the bandwidth was typically lower because the second-level cache ran at a lower frequency. Assume a 512 KiB off-chip

second-level cache has a miss rate of 4%. If each additional 512 KiB of cache lowered miss rates by 0.7%, and the cache had a total access time of 50 cycles, how big would the cache have to be to match the performance of the second-level direct-mapped cache listed above?

Solution

$$CPI = CPI_{Base} + InstrMemStall + DataMemStall$$

- $$CPI_{L1} = 1.5 + 0.07 * [2 * 10^9 \text{ Hz} * 100 * 10^9] = 1.5 + 0.07 * 200 = 15.5$$

$$CPI_{L2D} = 1.5 + 0.07 * (12 + 0.035 * 200) = 2.83$$

$$CPI_{L2W} = 1.5 + 0.07 * (28 + 0.015 * 200) = 3.67$$
- Of course it is possible however an extra level will be expensive in memory technology and capacity and make the design of a CPU more complicated however if the memory proposed for the new memory level is between

$$CPI_{L3} = 1.5 + 0.07 * (12 + 0.035 * (50 + 0.13 * 200)) = 2.5262$$
- In theory it is possible to outmatch the cache from previous exercise however that's not really the case because with this number the total miss rate of the new L2 should be negative to outperform.

$$CPI_{L2D} > 1.5 + 0.07 * (50 + (0.04 - 0.007 * X) * 200)$$

$$2.83 > 1.5 + 0.07 * (58 - 1.4 * X)$$

$$2.83 > 1.5 + 4.06 - 0.098 * X$$

$$X > \frac{2.83 - 1.5 - 4.06}{-0.098} = 27.8571$$

Exercise 5.13

Mean time between failures (MTBF), mean time to replacement (MTTR), and mean time to failure (MTTF) are useful metrics for evaluating the reliability and availability of a storage resource. Explore these concepts by answering the questions about a device with the following metrics:

MTTF	MTTR
3 Years	1 Day

- Calculate the MTBF for such a device.
- Calculate the availability for such a device.
- What happens to availability as the MTTR approaches 0? Is this a realistic situation?
- What happens to availability as the MTTR gets very high, i.e., a device is difficult to repair Does this imply the device has low availability?

Solution

- $MTBF = MTTF + MTTR = 365 \text{ days} * 3 + 1 \text{ days} = 1096 \text{ days}$
- $Availability = \frac{MTTF}{MTTF + MTTR} = \frac{1095 \text{ days}}{1096 \text{ days}} = 0.99908$
- The availability grows however more the MTTR is reduced make it more difficult to make it possible.
- Availability shrinks but still can had a good availability

Exercise 5.14

This exercise examines the single error correcting, double error detecting (SEC/DED) Hamming code.

- What is the minimum number of parity bits required to protect a 128-bit word using the SEC/DED code?
- Section 5.5 states that modern server memory modules (DIMMs) employ SEC/DED ECC to protect each 64 bits with 8 parity bits. Compute the cost/performance ratio of this code to the code from Exercise 5.14.1. In this case, cost is the relative number of parity bits needed while performance is the relative number of errors that can be corrected. Which is better?
- Consider a SEC code that protects 8 bit words with 4 parity bits. If we read the value 0x375, is there an error? If so, correct the error

Solution

1. Is necessary to find the minimal number that accomplish the following property

$$2^p \geq p + d + 1$$

$$2^8 \geq 8 + 128 + 1$$

$$256 \geq 137$$

$$\frac{\text{cost}_{\text{Code 1}}}{\text{perf}_{\text{Code 1}}} = \frac{9/128}{1/137} = 9.6328125$$

2.
$$\frac{\text{cost}_{\text{Code 2}}}{\text{perf}_{\text{Code 2}}} = \frac{8/64}{1/72} = 9$$

The cost of the first code is more expensive and makes sense because when the data bits grow at power of 2 then it adds 1 to the parity bits but this code can only correct one mistake.

3. First we transform the Hex number into binary

0011_0111_0101

Then locate the first parity bit and check for even parity with bits of 1,3,5,7,9,11

001101110101

$$0 = (1 + 0 + 1 + 0 + 0) \% 2$$

First parity bit is correct so we continue with the next bit for bits 2,3,6,7,10,11

001101110101

$$0 = (1 + 1 + 1 + 1 + 0) \% 2$$

Second parity bit is correct so we continue with the next bit for bits 4,5,6,7,12

001101110101

$$1 = (0 + 1 + 1 + 1) \% 2$$

Third parity bit is correct so we continue with the final bit for bits 8,9,10,11,12

0011011110101

$$1 = (0 + 1 + 0 + 1) \% 2$$

This bit has an error, now because it failed just this bit then the error is the parity bit itself so we need to flip it to get 0x365

Exercise 5.15

For a high-performance system such as a B-tree index for a database, the page size is determined mainly by the data size and disk performance. Assume that, on average, a B-tree index page is 70% full with fix-sized entries. The utility of a page is its B-tree depth, calculated as $\log_2(\text{entries})$. The following table shows that for 16-byte entries, and a 10-year-old disk with a 10 ms latency and 10 MB/s transfer rate, the optimal page size is 16 K.

Page Size (KiB)	Page Utility or B-Tree Depth (Number of Disk Accesses Saved)	Index Page Access Cost (ms)	Utility/Cost
2	6.49 (or $\log_2(2048/16 \times 0.7)$)	10.2	0.64
4	7.49	10.4	0.72
8	8.49	10.8	0.79
16	9.49	11.6	0.82
32	10.49	13.2	0.79
64	11.49	16.4	0.70
128	12.49	22.8	0.55
256	13.49	35.6	0.38

1. What is the best page size if entries now become 128 bytes?
2. Based on Exercise 5.15.1, what is the best page size if pages are half full?
3. Based on Exercise 5.15.2, what is the best page size if using a modern disk with a 3 ms latency and 100 MB/s transfer rate? Explain why future servers are likely to have larger pages.

Keeping “frequently used” (or “hot”) pages in DRAM can save disk accesses, but how do we determine the exact meaning of “frequently used” for a given system? Data engineers use the cost ratio between DRAM and disk access to quantify the reuse time threshold for hot pages. The cost of a disk access is $\text{\$/disk}/\text{accesses_per_sec}$, while the cost to keep a page in DRAM is $\text{\$/MiB}/\text{page_size}$. The typical DRAM and disk costs and typical database page sizes at several time points are listed below:

Year	DRAM Cost (\\$/MiB)	Page Size (KiB)	Disk Cost (\\$/disk)	Disk Access Rate (access/sec)
1987	5000	1	15,000	15
1997	15	8	2000	64
2007	0.05	64	80	83

- What other factors can be changed to keep using the same page size (thus avoiding software rewrite)? Discuss their likeliness with current technology and cost trends.

Solution

- Given the next table we know that the best page size is 32KiB

Page Size (KiB)	Page Utility or B-Tree Depth	Index Page Access Cost (ms)	Utility/Cost
2	$\log_2(2048/128*0.7) = 3.4854$	$10+2KB/10MB = 10.2$	0.341708
4	4.485426827	10.4	0.431291
8	5.485426827	10.8	0.508909
16	6.485426827	11.6	0.559088
32	7.485426827	13.2	0.567077
64	8.485426827	16.4	0.517404
128	9.485426827	22.8	0.416027
256	10.485426827	35.6	0.294534

- Now the best page size is still 32KiB

Page Size (KiB)	Page Utility or B-Tree Depth	Index Page Access Cost (ms)	Utility/Cost
2	$\log_2(2048/128*0.5) = 3$	$10+2KB/10MB = 10.2$	0.294117
4	4	10.4	0.384615
8	5	10.8	0.462962
16	6	11.6	0.517241
32	7	13.2	0.530303
64	8	16.4	0.487804
128	9	22.8	0.394736
256	10	35.6	0.280898

- Now the best page size is 64KiB. Servers tend to use bigger page size because the disk bandwidth are getting better and because the latency time don't get better as disk bandwidth so it makes sense to make the page sizes larger

Page Size (KiB)	Page Utility or B-Tree Depth	Index Page Access Cost (ms)	Utility/Cost
--------------------	------------------------------	--------------------------------	--------------

2	$\log_2(2048/128*0.5) = 3$	$3+2\text{KB}/100\text{MB} = 3.02$	0.993377
4	4	3.04	1.315789
8	5	3.08	1.623376
16	6	3.16	1.898734
32	7	3.32	2.108433
64	8	3.64	2.197802
128	9	4.28	2.102803
256	10	5.56	1.798561

- Keeping the page size the same only makes sense if the price of DRAM maintains the same or if access rate per seconds increases

Exercise 5.16

As described in Section 5.7, virtual memory uses a page table to track the mapping of virtual addresses to physical addresses. This exercise shows how this table must be updated as addresses are accessed. The following data constitute a stream of virtual byte addresses as seen on a system. Assume 4 KiB pages, a four entry fully associative TLB, and true LRU replacement. If pages must be brought in from disk, increment the next largest page number.

0x123D, 0x08B3, 0x365C, 0x871B, 0xBEE6, 0x3140, 0xC049

TLB

Valid	Tag	Physical Page Number	Time Since Last Access
1	0xb	12	4
1	0x7	4	1
1	0x3	6	3
0	0x4	9	7

Page table

Index	Valid	Physical Page or in Disk
0	1	5
1	0	Disk
2	0	Disk
3	1	6
4	1	9
5	1	11
6	0	Disk
7	1	4
8	0	Disk
9	0	Disk
a	1	3
b	1	12

- For each access shown above, list
 - Whether the access is a hit or miss in the TLB
 - Whether the access is a hit or miss in the page table
 - Whether the access is a page fault
 - The updated state of the TLB

- Repeat Exercise 5.16.1, but this time use 16 KiB pages instead of 4 KiB pages. What would be some of the advantages of having a larger page size? What are some of the disadvantages?
- Repeat Exercise 5.16.1, but this time use 4 KiB pages and a two-way set associative TLB.
- Repeat Exercise 5.16.1, but this time use 4 KiB pages and a direct mapped TLB.
- Discuss why a CPU must have a TLB for high performance. How would virtual memory accesses be handled if there were no TLB?

Solution

- With 4KiB page size leave us with 12 offset bits and the remaining as tag bits

Word Address	Page Access	Tag	TLB Access	Page Table Access	Page Fault	Valid	Tag	Physical Page Number	Time Since Last Access
0x123D	0x1	0x1	Miss	Miss	Yes	1	0xB	12	5
						1	0x7	4	2
						1	0x3	6	4
						1	0x1	13	1
0x08B3	0x0	0x0	Miss	Hit	No	1	0x0	5	1
						1	0x7	4	3
						1	0x3	6	5
						1	0x1	13	2
0x365C	0x3	0x3	Hit	Hit	No	1	0x0	5	2
						1	0x7	4	4
						1	0x3	6	1
						1	0x1	13	3
0x871B	0x8	0x8	Miss	Miss	Yes	1	0x0	5	3
						1	0x8	14	1
						1	0x3	6	2
						1	0x1	13	4
0xBEE6	0xB	0xB	Miss	Hit	No	1	0x0	5	4
						1	0x8	14	2
						1	0x3	6	3
						1	0xB	12	1
0x3140	0x3	0x3	Hit	Hit	No	1	0x0	5	5
						1	0x8	14	3
						1	0x3	6	1
						1	0xB	12	2
0xC049	0xC	0xC	Miss	Miss	Yes	1	0xC	15	1
						1	0x8	14	4
						1	0x3	6	2

						1	0xB	12	3
--	--	--	--	--	--	---	-----	----	---

2. With a page size of 16KiB leave us with 14 offset bits and the remaining as tag bits. With more larger pages size can improve TLB hits but also makes worse fragmentation

Word Address	Page Access	Tag	TLB Access	Page Table Access	Page Fault	Valid	Tag	Physical Page Number	Time Since Last Access
0x123D	0x0	0x0	Miss	Hit	No	1	0xB	12	5
						1	0x7	4	2
						1	0x3	6	4
						1	0x0	5	1
0x08B3	0x0	0x0	Hit	Hit	No	1	0xB	12	6
						1	0x7	4	3
						1	0x3	6	5
						1	0x0	5	1
0x365C	0x0	0x0	Hit	Hit	No	1	0xB	12	7
						1	0x7	4	4
						1	0x3	6	6
						1	0x0	5	1
0x871B	0x2	0x2	Miss	Miss	Yes	1	0x2	13	1
						1	0x7	4	5
						1	0x3	6	7
						1	0x0	5	2
0xBEE6	0x2	0x2	Hit	Hit	No	1	0x2	13	1
						1	0x7	4	6
						1	0x3	6	8
						1	0x0	5	3
0x3140	0x0	0x0	Hit	Hit	No	1	0x2	13	2
						1	0x7	4	7
						1	0x3	6	9
						1	0x0	5	1
0xC049	0x3	0x3	Hit	Hit	No	1	0x2	13	3
						1	0x7	4	8
						1	0x3	6	1
						1	0x0	5	2

3. With a two way associative 4KiB page size we had 12 offset bits, 1 index bit and the remaining as tag bits

Word Address	Page Access	Tag	Index	TLB Access	Page Table Access	Page Fault	Valid	Tag	Index	Physical Page Number	Time Since Last Access
0x123D	0x1	0x0	0x1	Miss	Miss	Yes	1	0xB	0x0	12	5
							1	0x7	0x1	4	2
							1	0x3	0x0	6	4
							1	0x0	0x1	13	1
0x08B3	0x0	0x0	0x0	Miss	Hit	No	1	0x0	0x0	5	1
							1	0x7	0x1	4	3
							1	0x3	0x0	6	5
							1	0x0	0x1	13	2
0x365C	0x3	0x1	0x1	Miss	Hit	No	1	0x0	0x0	5	2
							1	0x1	0x1	6	1
							1	0x3	0x0	6	6
							1	0x0	0x1	13	3
0x871B	0x8	0x4	0x0	Miss	Hit	No	1	0x0	0x0	5	3
							1	0x1	0x1	6	2
							1	0x4	0x0	14	1
							1	0x0	0x1	13	4
0xBEE6	0xB	0x5	0x1	Miss	Hit	No	1	0x0	0x0	5	4
							1	0x1	0x1	6	3
							1	0x4	0x0	14	2
							1	0x5	0x1	12	1
0x3140	0x3	0x1	0x1	Hit	Hit	No	1	0x0	0x0	5	5
							1	0x1	0x1	6	1
							1	0x4	0x0	14	3
							1	0x5	0x1	12	2
0xC049	0xC	0x6	0x0	Miss	Miss	Yes	1	0x6	0x0	15	1
							1	0x1	0x1	6	2
							1	0x4	0x0	14	4
							1	0x5	0x1	12	3

4. With a directed mapped cache for 4KiB page size we had 12 offset bits, 2 index bits and the remaining as tag bits

Word Address	Page Access	Tag	Index	TLB Access	Page Table Access	Page Fault	Valid	Tag	Index	Physical Page Number
--------------	-------------	-----	-------	------------	-------------------	------------	-------	-----	-------	----------------------

0x123D	0x1	0x0	0x1	Miss	Miss	Yes	1	0xB	0x0	12
							1	0x0	0x1	13
							1	0x3	0x2	6
							0	0x4	0x3	9
0x08B3	0x0	0x0	0x0	Miss	Hit	No	1	0x0	0x0	5
							1	0x0	0x1	13
							1	0x3	0x2	6
							0	0x4	0x3	9
0x365C	0x3	0x0	0x3	Miss	Hit	No	1	0x0	0x0	5
							1	0x0	0x1	13
							1	0x3	0x2	6
							1	0x0	0x3	6
0x871B	0x8	0x2	0x0	Miss	Miss	Yes	1	0x2	0x0	14
							1	0x0	0x1	13
							1	0x3	0x2	6
							1	0x0	0x3	6
0xBEE6	0xB	0x2	0x3	Miss	Hit	No	1	0x2	0x0	14
							1	0x0	0x1	13
							1	0x3	0x2	6
							1	0x2	0x3	12
0x3140	0x3	0x0	0x3	Miss	Hit	No	1	0x2	0x0	14
							1	0x0	0x1	13
							1	0x3	0x2	6
							1	0x0	0x3	6
0xC049	0xC	0x3	0x0	Miss	Miss	Yes	1	0x3	0x0	15
							1	0x0	0x1	13
							1	0x3	0x2	6
							1	0x0	0x3	6

Exercise 5.17

There are several parameters that affect the overall size of the page table. Listed below are key page table parameters.

Virtual Address Size	Page Size	Page Table Entry Size
32 bits	8 KiB	4 bytes

- Given the parameters shown above, calculate the maximum possible page table size for a system running five processes.

- Given the parameters shown above, calculate the total page table size for a system running five applications that each utilize half of the virtual memory available, given a two-level page table approach with up to 256 entries at the 1st level. Assume each entry of the main page table is 6 bytes. Calculate the minimum and maximum amount of memory required for this page table.
- A cache designer wants to increase the size of a 4 KiB virtually indexed, physically tagged cache. Given the page size shown above, is it possible to make a 16 KiB direct-mapped cache, assuming four 32-bit words per block? How would the designer increase the data size of the cache?

Solution

- To get how many had to be the page table we need to get the bits for virtual page number. For 8KiB it uses 13 bits. That give us 19 bits for page entries. That give us

$$TotalMem = 5 * (2^{19} Entries * 4 Bytes / Entries) = 10 MiB$$
- With 256 entries for the first level table then this uses 8 bits and the remain 11 bits are used for level 2. If each process uses just half of the virtual memory then half of the total possible entries will be occupied. This give us

$$TotalMem = 5 * (256 Entries * 6 Bytes / Entries + 128 * (2^{11} Entries * 4 Bytes / Entries)) = 5 MiB$$

In this page memory scheme that means the minimum page table size per process is of

$$MinMem = 256 Entries * 6 Bytes / Entries + 1 * (2^{11} Entries * 4 Bytes / Entries) = 9.5 KiB$$

And the maximum is of

$$MaxMem = 256 Entries * 6 Bytes / Entries + 256 * (2^{11} Entries * 4 Bytes / Entries) = 2 MiB$$
- The page size is of 13 bits. To make an efficient direct mapped cache then a whole page must be mapped inside the cache. With 16 KiB of size and 4 word per block give us 4 bits of offset and 10 bits of index. So in order to improve performance the cache index must match the page size.

Exercise 5.18

In this exercise, we will examine space/time optimizations for page tables. The following list provides parameters of a virtual memory system.

Virtual Address (bits)	Physical DRAM Installed	Page Size	PTE Size (byte)
43	16 GiB	4 KiB	4

- For a single-level page table, how many page table entries (PTEs) are needed? How much physical memory is needed for storing the page table?
- Using a multi-level page table can reduce the physical memory consumption of page tables by only keeping active PTEs in physical memory. How many levels of page tables will be needed if the segment tables (the upper-level page tables) are allowed to be of unlimited size? How many memory references are needed for address translation if missing in TLB?
- Suppose the segments are limited to the 4 KiB page size (so that they can be paged). Is 4 bytes large enough for all page table entries including those in the segment tables?
- How many levels of page tables are needed if the segments are limited to the 4 KiB page size?
- An inverted page table can be used to further optimize space and time. How many PTEs are needed to store the page table? Assuming a hash table implementation, what are the common case and worst case numbers of memory references needed for servicing a TLB miss?

Solution

- With 4 KiB we had 12 bits of page offset so $43 - 12 = 31$ bits will be used for page number. Therefore the page table will occupy half of the physical memory for running 1 process.

$$TotalMem = 2^{31} Entries * 4 Bytes / Entries = 8 GiB$$
- There's no limit with how many levels of page tables can be segmented the virtual memory. How ever when a page fault occurs it need to search and consult all the segment tables to get to the desired page.

- Every page entry will need to store the physical page number and a valid bit. If you add up the total of bits will need $31+1=32$ bits which is enough to room up an entry. The dirty bit is not counted because is only needed on the final level table.
- If the segment tables need to be of the size of a page then we need to contemplate that every entry has 4 bytes size then this leave us with 1024 entries per segment (2^{10} bits). So in order to be able to segment the whole virtual memory (2^{31}) inside a segment of a size of a page will need 4 levels.
- An inverted page table mean that we index just the number of pages that can be placed in physical memory with a hash function. In case that a virtual memory has the same index then it make a collision and the page need to be searched in disk making comparisons. So then with this in mind the size of a page table are of $2^{(34-12)} \cdot 4 = 16\text{MiB}$. And for a page fault it depends of how many collisions can happen for a certain index so if we believe that the hash function distributes nicely then it is probably to get a worse scenario of $(43-34)=9$ page misses and the best scenario of 0.

Exercise 5.19

The following table shows the contents of a four-entry TLB.

Entry-ID	Valid	VA Page	Modified	Protection	PA Page
1	1	140	1	RW	30
2	0	40	0	RX	34
3	1	200	1	RO	32
4	1	280	0	RW	31

- Under what scenarios would entry 3's valid bit be set to zero?
- What happens when an instruction writes to VA page 30? When would a software managed TLB be faster than a hardware managed TLB?
- What happens when an instruction writes to VA page 200?

Solution

- It will be invalid if the page is written to the disk to allocate a new page
- It going to happen a TLB miss and it will check to main memory to retrieve the page address. A software managed TLB will be faster only if has some algorithms that predicts the next paged needed.
- It will trough an exception because it read only protected

Exercise 5.20

In this exercise, we will examine how replacement policies affect miss rate. Assume a two-way set associative cache with four one-word blocks. Consider the following word address sequence: 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0.

Consider the following address sequence: 0, 2, 4, 8, 10, 12, 14, 16, 0

- Assuming an LRU replacement policy, which accesses are hits?
- Assuming an MRU (most recently used) replacement policy, which accesses are hits?
- Simulate a random replacement policy by flipping a coin. For example, "heads" means to evict the first block in a set and "tails" means to evict the second block in a set. How many hits does this address sequence exhibit?
- Describe an optimal replacement policy for this sequence. Which accesses are hits using this policy?
- Describe why it is difficult to implement a cache replacement policy that is optimal for all address sequences.
- Assume you could make a decision upon each memory reference whether or not you want the requested address to be cached. What effect could this have on miss rate?

Solution

- There's no hits because it happen a compulsive miss.

Word	Refer	Cache	Word	Refer	Cache	Word	Refer	Cache
------	-------	-------	------	-------	-------	------	-------	-------

0	Miss	<0: 0 , X> <1: X, X>	0	Miss	<0: 0 , 6> <1: 5, 7>	0	Miss	<0: 0 , 6> <1: 5, 7>
1	Miss	<0: 0, X> <1: 1 , X>	1	Miss	<0: 0, 6> <1: 1 , 7>			
2	Miss	<0: 0, 2 > <1: 1, X>	2	Miss	<0: 0, 2 > <1: 1, 7>			
3	Miss	<0: 0, 2> <1: 1, 3 >	3	Miss	<0: 0, 2> <1: 1, 3 >			
4	Miss	<0: 4 , 2> <1: 1, 3>	4	Miss	<0: 4 , 2> <1: 1, 3>			
5	Miss	<0: 4, 2> <1: 5 , 3>	5	Miss	<0: 4, 2> <1: 5 , 3>			
6	Miss	<0: 4, 6 > <1: 5, 3>	6	Miss	<0: 4, 6 > <1: 5, 3>			
7	Miss	<0: 4, 6> <1: 5, 7 >	7	Miss	<0: 4, 6> <1: 5, 7 >			

2. It's going to hit 4 times

Word	Refer	Cache	Word	Refer	Cache	Word	Refer	Cache
0	Miss	<0: 0 , X> <1: X, X>	0	Hit	<0: 0 , 6> <1: 1, 7>	0	Miss	<0: 4, 0 > <1: 5, 7>
1	Miss	<0: 0, X> <1: 1 , X>	1	Hit	<0: 0, 6> <1: 1 , 7>			
2	Miss	<0: 0, 2 > <1: 1, X>	2	Miss	<0: 2 , 6> <1: 1, 7>			
3	Miss	<0: 0, 2> <1: 1, 3 >	3	Miss	<0: 2, 6> <1: 3 , 7>			
4	Miss	<0: 0, 4 > <1: 1, 3>	4	Miss	<0: 4 , 6> <1: 3, 7>			
5	Miss	<0: 0, 4> <1: 1, 5 >	5	Miss	<0: 4, 6> <1: 5 , 7>			
6	Miss	<0: 0, 6 > <1: 1, 5>	6	Hit	<0: 4, 6 > <1: 5, 7>			
7	Miss	<0: 0, 6> <1: 1, 7 >	7	Hit	<0: 4, 6> <1: 5, 7 >			

3. It has 0 hits

Word	Refer	Cache	Word	Refer	Cache	Word	Refer	Cache
0	Miss	<0: X, 0 > <1: X, X>	0	Miss	<0: 0 , 4> <1: 7, 5>	0	Miss	<0: 0 , 6> <1: 5, 7>
1	Miss	<0: X, 0> <1: X, 1 >	1	Miss	<0: 0, 4> <1: 7, 1 >			

2	Miss	<0: X, 2> <1: X, 1>	2	Miss	<0: 0, 2> <1: 7, 1>			
3	Miss	<0: X, 2> <1: 3, 1>	3	Miss	<0: 0, 2> <1: 7, 3>			
4	Miss	<0: X, 4> <1: 3, 1>	4	Miss	<0: 0, 4> <1: 7, 3>			
5	Miss	<0: X, 4> <1: 3, 5>	5	Miss	<0: 0, 4> <1: 5, 3>			
6	Miss	<0: 6, 4> <1: 3, 5>	6	Miss	<0: 0, 6> <1: 5, 3>			
7	Miss	<0: 6, 4> <1: 7, 5>	7	Miss	<0: 0, 6> <1: 5, 7>			

- Because the sequence goes from 0 to 7 and we only have 4 blocks then is impossible to get more than a half of hits per total consults so MRU is actually the best policy.
- Because there's no similarities between all possibilities. The best replace policy is some that can abstract a pattern so it can foresee which block is better to replace.
- It can improve greatly cache hits but it can also increase cache time hit and time penalty.

Exercise 5.21

One of the biggest impediments to widespread use of virtual machines is the performance overhead incurred by running a virtual machine. Listed below are various performance parameters and application behaviour.

Base CPI	Privileged O/S accesses per 10,000 instructions	Overhead to trap to the guest O/S	Overhead to trap to VMM	I/O access per 10,000 instructions	I/O access time (includes time to trap to guest O/S)
1.5	120	15 cycles	175 cycles	30	1100 cycles

- Calculate the CPI for the system listed above assuming that there are no accesses to I/O. What is the CPI if the VMM overhead doubles? If it is cut in half? If a virtual machine software company wishes to limit the performance degradation to 10%, what is the longest possible penalty to trap to the VMM?
- I/O accesses often have a large effect on overall system performance. Calculate the CPI of a machine using the performance characteristics above, assuming a non-virtualized system. Calculate the CPI again, this time using a virtualized system. How do these CPIs change if the system has half the I/O accesses?

Solution

- CPI with VM overhead

$$CPI_{VM} = 1.5 + \frac{120}{10000} * 15 \text{ Cycles} + \frac{120}{10000} * 175 \text{ Cycles} = 3.78$$

CPI with VM overhead halved

$$CPI_{VM} = 1.5 + \frac{120}{10000} * 15 \text{ Cycles} + \frac{120}{10000} * 175 \text{ Cycles} / 2 = 2.73$$

CPI with VM overhead doubled

$$CPI_{VM} = 1.5 + \frac{120}{10000} * 15 \text{ Cycles} + \frac{120}{10000} * 175 \text{ Cycles} * 2 = 5.88$$

Original CPI

$$CPI_{Orig} = 1.5 + \frac{120}{10000} * 15 Cycles = 1.68$$

The number of cycles of VM overhead has to be of 14 cycles or less to had a degradation of 10% or less in performance.

$$CPI_{Orig} * 1.1 \geq CPI_{VM}$$

$$1.68 * 1.1 \geq 1.5 + \frac{120}{10000} * 15 Cycles + \frac{120}{10000} * X$$

$$\frac{(1.68 * 1.1 - 1.5 - 0.18) * 10000}{120} \geq X$$

$$X \leq 14$$

2. For a non virtualized system the CPI is of

$$CPI_{IO} = 1.5 + \frac{120}{10000} * 15 Cycles + \frac{30}{10000} * 1100 Cycles = 4.98$$

Now the I/O overhead is added the trap time in VM because it traps inside guest and the this trap is forwarded into the host machine

$$CPI_{vMIO} = 1.5 + \frac{120}{10000} * (15 + 175) Cycles + \frac{30}{10000} * (1100 + 175) Cycles = 7.605$$

CPI of only host machine with halved I/O requests

$$CPI_{IO} = 1.5 + \frac{120}{10000} * 15 Cycles + \frac{15}{10000} * 1100 Cycles = 3.33$$

CPI of VM with halved I/O request

$$CPI_{vMIO} = 1.5 + \frac{120}{10000} * (15 + 175) Cycles + \frac{15}{10000} * (1100 + 175) Cycles = 5.6925$$

Exercise 5.22

Compare and contrast the ideas of virtual memory and virtual machines. How do the goals of each compare? What are the pros and cons of each? List a few cases where virtual memory is desired, and a few cases where virtual machines are desired.

Solution

Virtual memory and virtual machines concepts just had one area in common. As a process they do not share memory mapping space between other processes. However this is where it ends. Virtual memory only ensures that an application don't share physical memory mapping with other app. However a Virtual Machine memory uses physical mapping but with special translations, so can use hardware but assigned by the hypervisor which will control this execute environment.

Exercise 5.23

Section 5.6 discusses virtualization under the assumption that the virtualized system is running the same ISA as the underlying hardware. However, one possible use of virtualization is to emulate non-native ISAs. An example of this is QEMU, which emulates a variety of ISAs such as MIPS, SPARC, and PowerPC. What are some of the difficulties involved in this kind of virtualization? Is it possible for an emulated system to run faster than on its native ISA?

Solution

This kind of emulation are complicated because it need first save process status in memory (which is obviously less quickly as registers speed). Then every guest ISA instruction has to be matched with a host ISA instructions (Which it hasn't a directly translations always). Also the overhead to first translate and then run the virtual machine increase the time needed to access I/O request because the layered trap instructions and other stuff. It is possible for a guest machine to run quickly than an original machine if the ISA has some hardware acceleration, special instructions and/or optimized compilers which can help to execute the software even faster than target hardware.

Exercise 5.24

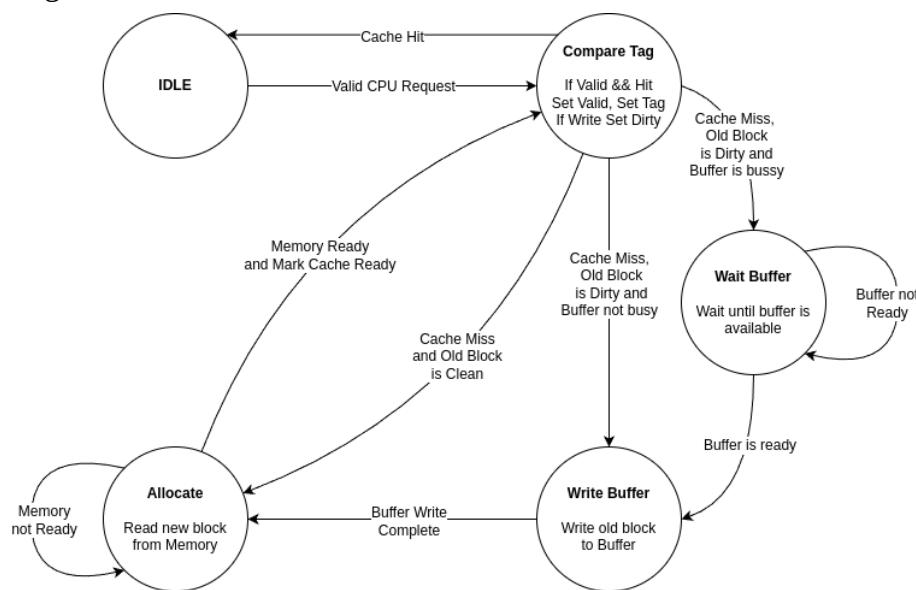
In this exercise, we will explore the control unit for a cache controller for a processor with a write buffer. Use the finite state machine found in Figure 5.39 as a starting point for designing your own finite state machines. Assume that the cache controller is for the simple direct-mapped cache described on page 474 (Figure 5.38 in Section 5.9), but you will add a write buffer with a capacity of one block.

Recall that the purpose of a write buffer is to serve as temporary storage so that the processor doesn't have to wait for two memory accesses on a dirty miss. Rather than writing back the dirty block before reading the new block, it buffers the dirty block and immediately begins reading the new block. The dirty block can then be written to main memory while the processor is working.

1. What should happen if the processor issues a request that hits in the cache while a block is being written back to main memory from the write buffer?
2. What should happen if the processor issues a request that misses in the cache while a block is being written back to main memory from the write buffer?
3. Design a finite state machine to enable the use of a write buffer.

Solution

1. It should not stop. Is supposed that if the block that is written into the buffer has already free the space in cache so there's no problem if the cache is being read
2. The cache has to wait then for the write back to finish because it need to write that block into the buffer and the buffer will be free until finish to write
3. Diagram



Exercise 5.25

Cache coherence concerns the views of multiple processors on a given cache block. The following data show two processors and their read/write operations on two different words of a cache block X (initially $X[0] = X[1] = 0$).

P1	P2
$X[0] ++$; $X[1] = 3$;	$X[0] = 5$; $X[1] += 2$;

1. List the possible values of the given cache block for a correct cache coherence protocol implementation. List at least one more possible value of the block if the protocol doesn't ensure cache coherency.
2. For a snooping protocol, list a valid operation sequence on each processor/cache to finish the above read/write operations.

3. What are the best-case and worst-case numbers of cache misses needed to execute the listed read/write instructions?

Memory consistency concerns the views of multiple data items. The following data show two processors and their read/write operations on different cache blocks (A and B initially 0).

P1	P2
A = 1; B = 2; A+=2; B++;	C = B; D = A;

4. List the possible values of C and D for all implementations that ensure both consistency assumptions on page 476.
5. List at least one more possible pair of values for C and D if such assumptions are not maintained.
6. For various combinations of write policies and write allocation policies, which combinations make the protocol implementation simpler?

Solution

1. List of possible combinations

Case 1		Case 2		Case 3	
P1	P2	P1	P2	P1	P2
X[0]++;			X[0] = 5;	X[0]++;	
X[1] = 3;			X[1] += 2;		X[0] = 5;
	X[0] = 5;	X[0]++;		X[1] = 3;	
	X[1] += 2;	X[1] = 3;			X[1] += 2;
Values		Values		Values	
X[0] : 5	X[1] : 5	X[0] : 6	X[1] : 3	X[0] : 5	X[1] : 5
Case 4		Case 5		Case 6	
P1	P2	P1	P2	P1	P2
X[0]++;			X[0] = 5;		X[0] = 5;
	X[0] = 5;	X[0]++;		X[0]++;	
	X[1] += 2;	X[1] = 3;			X[1] += 2;
X[1] = 3;			X[1] += 2;	X[1] = 3;	
Values		Values		Values	
X[0] : 5	X[1] : 3	X[0] : 6	X[1] : 5	X[0] : 6	X[1] : 3

2. Applying snooping protocol to case 1

Processor Activity		Bus Activity	Contents of P1's Cache	Contents of P2's Cache	Content of Memory
P1	P2				
		N/A	X[0] : 0 X[1] : 0	X[0] : 0 X[1] : 0	X[0] : 0 X[1] : 0
X[0]++;		Cache content X[0] is updated by P1 and it invalidates P2's X[0] copy	X[0] : 1 X[1] : 0	X[0] : ? X[1] : 0	X[0] : 0 X[1] : 0
X[1] = 3;		Cache content X[1] is updated by P1 and it invalidates P2's X[1] copy	X[0] : 1 X[1] : 3	X[0] : ? X[1] : ?	X[0] : 0 X[1] : 0

	X[0] = 5;	Cache content X[0] is snooped by P2 fetching it from P1 Cache content X[0] is updated by P2 and it invalidates P1's X[0] copy	X[0] : ? X[1] : 3	X[0] : 5 X[1] : ?	X[0] : 0 X[1] : 0
	X[1] += 2;	Cache content X[1] is snooped by P2, fetching it from P1 Cache content X[1] is updated by P2 and it invalidates P1's X[1] copy	X[0] : ? X[1] : ?	X[0] : 5 X[1] : 5	X[0] : 0 X[1] : 0
FENCE	FENCE	Cache content is forced to write back			X[0] : 5 X[1] : 5

3. The worst cases are 3 and 6 with 4 cache misses. Then it follows case 4 and 5 with 3 cache misses and finally the best case scenarios are 1 and 2
4. List of possible combinations

Case 1		Case 2		Case 3		Case 4	
P1	P2	P1	P2	P1	P2	P1	P2
A = 1;		A = 1;		A = 1;		A = 1;	
B = 2;		B = 2;		B = 2;			C = B;
A += 2;		A += 2;			C = B;		D = A;
B++;			C = B;		D = A;	B = 2;	
	C = B;		D = A;	A += 2;		A += 2;	
	D = A;	B++;		B++;		B++;	
Values		Values		Values		Values	
C : 3	D : 3	C : 2	D : 3	C : 2	D : 1	C : 0	D : 1
Case 5		Case 6		Case 7		Case 8	
P1	P2	P1	P2	P1	P2	P1	P2
	C = B;		C = B;		C = B;		C = B;
	D = A;	A = 1;		A = 1;		A = 1;	
A = 1;			D = A;	B = 2;		B = 2;	
B = 2;		B = 2;			D = A;	A += 2;	
A += 2;		A += 2;		A += 2;			D = A;
B++;		B++;		B++;		B++;	
Values		Values		Values		Values	
C : 0	D : 0	C : 0	D : 1	C : 0	D : 1	C : 0	D : 3
Case 9		Case 10		Case 11		Case 12	
P1	P2	P1	P2	P1	P2	P1	P2
	C = B;	A = 1;		A = 1;		A = 1;	
A = 1;			C = B;		C = B;		C = B;

B = 2;		B = 2;		B = 2;		B = 2;	
A += 2;			D = A;	A += 2;		A += 2;	
B++;		A += 2;			D = A;	B++;	
	D = A;	B++;		B++;			D = A;
Values		Values		Values		Values	
C : 0	D : 3	C : 0	D : 1	C : 0	D : 3	C : 0	D : 3
Case 13		Case 14		Case 15			
P1	P2	P1	P2	P1	P2		
A = 1;		A = 1;		A = 1;			
B = 2;		B = 2;		B = 2;			
	C = B;		C = B;	A += 2;			
A += 2;		A += 2;			C = B;		
	D = A;	B++;		B++;			
B++;			D = A;		D = A;		
Values		Values		Values			
C : 2	D : 3	C : 2	D : 3	C : 2	D : 3		

5. If the instructions doesn't follow the coherence rules then it could happen that CPUs seen the a distinct order of instructions like

Processor Activity		Bus Activity	Contents of P1's Cache	Contents of P2's Cache
P1	P2			
		N/A	A : 0 B : 0	C : ? D : ?
A = 1;		Executed normally	A : 1 B : 0	C : ? D : ?
B = 2;		Executed normally	A : 1 B : 2	C : ? D : ?
	C = B;	Executed normally	A : 1 B : 2	C : 2 D : ?
A += 2;	D = A;	Coherence is not maintained and P1 instruction is executed before P2 instruction	A : 3 B : 2	C : 2 D : 1

6. Write back model is better than write trough because:
1. Latency is reduced because if some write is performed to main memory then the coherence protocol has to wait until it complete the write
 2. The same dirty bit can be used to track the copies that has to be changed so the protocol can use unicast writes instead broadcast ones
- Also for write allocation has no much impact. Probably no allocation could had better performance but it wont change the protocol function.

Exercise 5.26

Chip multiprocessors (CMPs) have multiple cores and their caches on a single chip. CMP on-chip L2 cache design has interesting trade-offs. The following table shows the miss rates and hit latencies for two benchmarks with private vs. shared L2 cache designs. Assume the L1 cache has a 3% miss rate and a 1-cycle access time.

	Private	Shared
Benchmark A miss rate	10%	4%
Benchmark B miss rate	2%	1%

Assume the following hit latencies:

Private Cache	Shared Cache	Memory
5	20	180

1. Which cache design is better for each of these benchmarks? Use data to support your conclusion.
2. Off-chip bandwidth becomes the bottleneck as the number of CMP cores increases. How does this bottleneck affect private and shared cache systems differently? Choose the best design if the latency of the first off-chip link doubles.
3. Discuss the pros and cons of shared vs. private L2 caches for both single-threaded, multi-threaded, and multiprogrammed workloads, and reconsider them if having on-chip L3 caches.
4. Would a non-blocking L2 cache produce more improvement on a CMP with a shared L2 cache or a private L2 cache? Why?
5. Assume new generations of processors double the number of cores every 18 months. To maintain the same level of per-core performance, how much more off-chip memory bandwidth is needed for a processor released in three years?
6. Consider the entire memory hierarchy. What kinds of optimizations can improve the number of concurrent misses?

Solution

1. It seems that for both benchmarks a private cache for L2 is better.
 $AMAT_{PrivateBenchA} = 1 \text{ cycles} + 0.03 * (5 \text{ cycles} + 0.10 * 180 \text{ cycles}) = 1.69 \text{ cycles}$
 $AMAT_{SharedBenchA} = 1 \text{ cycles} + 0.03 * (20 \text{ cycles} + 0.04 * 180 \text{ cycles}) = 1.816 \text{ cycles}$
 $AMAT_{PrivateBenchB} = 1 \text{ cycles} + 0.03 * (5 \text{ cycles} + 0.02 * 180 \text{ cycles}) = 1.258 \text{ cycles}$
 $AMAT_{SharedBenchB} = 1 \text{ cycles} + 0.03 * (20 \text{ cycles} + 0.01 * 180 \text{ cycles}) = 1.654 \text{ cycles}$
2. For memory access shared between many cores, it is important to be sure that no other core is making a write or read so it has to wait if it is. For private L2 memory then the main memory latency doubles and for shared L2 memory latency is what is doubled.
 $AMAT_{PrivateBenchA} = 1 \text{ cycles} + 0.03 * (5 \text{ cycles} + 0.10 * 180 * 2 \text{ cycles}) = 2.23 \text{ cycles}$
 $AMAT_{SharedBenchA} = 1 \text{ cycles} + 0.03 * (20 * 2 \text{ cycles} + 0.04 * 180 \text{ cycles}) = 2.416 \text{ cycles}$
 $AMAT_{PrivateBenchB} = 1 \text{ cycles} + 0.03 * (5 \text{ cycles} + 0.02 * 180 * 2 \text{ cycles}) = 1.366 \text{ cycles}$
 $AMAT_{SharedBenchB} = 1 \text{ cycles} + 0.03 * (20 * 2 \text{ cycles} + 0.01 * 180 \text{ cycles}) = 2.254 \text{ cycles}$
3. Cache L2 memory is there to provide quick response and/or high throughput for L1-DATA and L1-INSTR caches so. In single threading is better the use of private caches because the info is just used by one thread at the same time so having more space and high response between main memory and L2 is priority. For multi threading the use of shared memory can be handy because some of the same data has to be processed by many cores and high throughput is more important this kind of workloads. Now for multi programming (concurrency) it depends on the OS scheduler because for concurrency to work it needs to make context switches for process execution. So every context switch will most likely have a compulsory miss between process data and OS data. A private can be great for high performance at task but slow context switch and a shared cache can make more quickly context switches but slow task execution. For L3 cache level is ideal for multi programming because with a private L2 and shared L3 can balance task performance and context switches.

4. Shared memories can take more advantage of a non blocking scheme if more threads share the same cache
5. It will need 4 times more bandwidth
6. Higher level of associativity and more cache levels (if needed)

Exercise 5.27

In this exercise we show the definition of a web server log and examine code optimizations to improve log processing speed. The data structure for the log is defined as follows:

```
struct entry {
    int srcIP;           // remote IP address
    char URL[128];       // request URL (e.g., "GET index.html")
    long long refTime;   // reference time
    int status;          // connection status
    char browser[64];    // client browser name
} log [NUM_ENTRIES];
```

Assume the following processing function for the log:

```
topK_sourceIP (int hour);
```

This function determines the most frequently observed source IPs during the given hour.

1. Which fields in a log entry will be accessed for the given log processing function? Assuming 64-byte cache blocks and no prefetching, how many cache misses per entry does the given function incur on average?
2. How can you reorganize the data structure to improve cache utilization and access locality?
3. Give an example of another log processing function that would prefer a different data structure layout. If both functions are important, how would you rewrite the program to improve the overall performance? Supplement the discussion with code snippet and data.

Solution

1. The function most likely use the values of srcIP and refTime. When the function read the entry will load the elements of the structure in order so when cache load first load srcIP, then URL and then refTime. In all makes $4+128+8=140$ bytes to load so it takes 2 cache misses
2. Is best to put both terms together at the bottom of the struct declaration or put them inside an array. The use of the structure can lead to have inconsistent read and write times operations.
3. pageTracker(char name[128]) will use the values of srcIP, refTime and URL. The best here is to put URL and srcIP at first for read and search process. Then refTime will be summoned to add into an accumulator.

Exercise 5.28

For the problems below, use data from "Cache Performance for SPEC CPU2000 Benchmarks"

(<https://research.cs.wisc.edu/multifacet/misc/spec2000cache-data/>) for the pairs of benchmarks shown in the following table.

- Mesa/gcc
 - mfc/swim
1. For 64 KiB data caches with varying set associativities, what are the miss rates broken down by miss types (cold, capacity, and conflict misses) for each benchmark?
 2. Select the set associativity to be used by a 64 KiB L1 data cache shared by both benchmarks. If the L1 cache has to be directly mapped, select the set associativity for the 1 MiB L2 cache.
 3. Give an example in the miss rate table where higher set associativity actually increases miss rate. Construct a cache configuration and reference stream to demonstrate this.

Solution

In order to get all type of misses we had to remember that full associativity caches cannot suffer of conflict misses and misses increases if associativity decreases. Cold misses isn't affected by associativity or cache capacity, only for block size. So because Full LRU misses eliminate the variable of conflict misses so we can use it from point of comparison to get how much does influence every factor.

For capacity misses we can be sure that

$$CapacityMissRate_{CacheSize} = FullAssociativityMissRate_{CacheSize} - CompulsiveMissRate$$

And for conflict misses can be sure that

$$ConflictMissRate_{CacheSize} = TotalMissRate_{CacheSize} - FullAssociativityMissRate_{CacheSize}$$

With that in mind I choose the tables of gcc/mesa to begin with (I used the table with 10 Billions instructions)

1. 64 KiB Data Cache Benchmark 10BGCC

D-cache misses/inst: 16,569,264,006 data refs (0.331385-/inst);					
8,966,239,890 D-cache 64-Byte block accesses (0.179325-/inst)					
Size	Direct	2-way LRU	4-way LRU	8-way LRU	Full LRU
1KB	0.06346574-	0.04668562-	0.04257458-	0.04156217-	0.04141388-
2KB	0.04693398-	0.03612483-	0.03304530-	0.03231412-	0.03183556-
4KB	0.03649746-	0.02924187-	0.02708643-	0.02646106-	0.02623912-
8KB	0.02916864-	0.02391504-	0.02262363-	0.02210198-	0.02178520-
16KB	0.02429691-	0.02055703-	0.01968944-	0.01943604-	0.01924308-
32KB	0.02067646-	0.01866539-	0.01823121-	0.01812939-	0.01808305-
64KB	0.01852919-	0.01736967-	0.01718012-	0.01713949-	0.01713261-
128KB	0.01597118-	0.01545810-	0.01547442-	0.01558514-	0.01601386-
256KB	0.01072520-	0.01109094-	0.01231593-	0.01260210-	0.01312512-
512KB	0.00622656-	0.00645999-	0.00544070-	0.00515160-	0.00514910-
1MB	0.00336203-	0.00322058-	0.00134859-	0.00136141-	0.00109549-
Compulsory: 0.00010408020-					

Associativity /Misses	Compulsory	Capacity	Conflict	Total
Full	0.00010408020	0.01702853	0	0.01713261
8-Way	0.00010408020	0.01702853	0.00000688	0.01713949
4-Way	0.00010408020	0.01702853	0.00004751	0.01718012
2-Way	0.00010408020	0.01702853	0.00023706	0.01736967
Direct	0.00010408020	0.01702853	0.00139658	0.01852919

64 KiB Data Cache benchmark 10B MESA

D-cache misses/inst: 3,559,197,730 data refs (0.3559198/inst);					
2,158,604,490 D-cache 64-Byte block accesses (0.2158604/inst)					
Size	Direct	2-way LRU	4-way LRU	8-way LRU	Full LRU
1KB	0.079958932	0.079105806	0.073480496	0.068735430	0.069280499
2KB	0.055273084	0.041601217	0.040010862	0.040933266	0.039412338
4KB	0.034712863	0.026421567	0.022550709	0.022654044	0.013712667
8KB	0.021392715	0.011821312	0.007720155	0.002952809	0.001662711
16KB	0.006878697	0.003716589	0.001780209	0.001593385	0.001568172
32KB	0.003222467	0.002557930	0.001344878	0.001358034	0.001351044
64KB	0.001351974	0.001133983	0.001131918	0.001135739	0.001174064
128KB	0.001058828	0.000833088	0.000781407	0.000755959	0.000624757
256KB	0.000802858	0.000667360	0.000613190	0.000605119	0.000586836
512KB	0.000703759	0.000603053	0.000586633	0.000585360	0.000584191
1MB	0.000636369	0.000585791	0.000580986	0.000580968	0.000581322
Compulsory (256MB 2-way): 0.000015074900					

Associativity/Misses	Compulsory	Capacity	Conflict	Total
Full	0.000015074900	0.001158989	0	0.001174064
8-Way	0.000015074900	0.001158989	-0.000038325	0.001135739
4-Way	0.000015074900	0.001158989	-0.000042146	0.001131918
2-Way	0.000015074900	0.001158989	-0.000040081	0.001133983
Direct	0.000015074900	0.001158989	0.00017791	0.001351974

A negative result in conflict misses as associativity decreases seems illogical however we are not counting processes like pref-etching or how much spatial temporality exist in the program

- For GCC benchmark full associativity in cache has the best result and for MESA the 4-Way associativity is best. Maybe a intermediate point like 8-Way perform better for both benchmarks. To know which L2 cache is the best we have the following tables:

1 MiB Unified L2 Cache GCC Benchmark 10B

```

-----
U-cache prefetch misses/inst: 819,335,909 prefetches (0.016387-/inst);
819,350,796 U-cache 64-Byte prefetch block accesses (0.016387-/inst)
-----
Size | Direct | 2-way LRU | 4-way LRU | 8-way LRU | Full LRU
-----+-----+-----+-----+-----+-----
1KB | 0.00828899- | 0.00669127- | 0.00555708- | 0.00555628- | 0.00519002-
2KB | 0.00686216- | 0.00550161- | 0.00519368- | 0.00516844- | 0.00516876-
4KB | 0.00588643- | 0.00521796- | 0.00512487- | 0.00513526- | 0.00514098-
8KB | 0.00522179- | 0.00483165- | 0.00487789- | 0.00491485- | 0.00496480-
16KB | 0.00467325- | 0.00443295- | 0.00439049- | 0.00438233- | 0.00440440-
32KB | 0.00436516- | 0.00418463- | 0.00414975- | 0.00414352- | 0.00414457-
64KB | 0.00417738- | 0.00409308- | 0.00408129- | 0.00408147- | 0.00408481-
128KB | 0.00370765- | 0.00373583- | 0.00379404- | 0.00386883- | 0.00400716-
256KB | 0.00288219- | 0.00298218- | 0.00330395- | 0.00333938- | 0.00334633-
512KB | 0.00204050- | 0.00188568- | 0.00161755- | 0.00152249- | 0.00134759-
1MB | 0.00095238- | 0.00092958- | 0.00029582- | 0.00035459- | 0.00010577-
-----
Compulsory: 0.0000023214-

```

1 MiB Unified L2 Cache MESA Benchmark 10B

```

-----
U-cache misses/inst: 13,559,197,730 unified refs (1.3559198/inst);
7,994,038,955 U-cache 64-Byte block accesses (0.7994039/inst)
-----
Size | Direct | 2-way LRU | 4-way LRU | 8-way LRU | Full LRU
-----+-----+-----+-----+-----+-----
1KB | 0.232144912 | 0.194796142 | 0.189168866 | 0.175680586 | 0.175000615
2KB | 0.187074859 | 0.154752895 | 0.149163131 | 0.149706949 | 0.147130700
4KB | 0.119265548 | 0.110172532 | 0.105244647 | 0.105799619 | 0.099364995
8KB | 0.081810479 | 0.072100631 | 0.073484452 | 0.071935096 | 0.083446781
16KB | 0.045508677 | 0.026684708 | 0.023006210 | 0.018058040 | 0.001923260
32KB | 0.020560579 | 0.009523350 | 0.003399767 | 0.001900798 | 0.001566081
64KB | 0.010958759 | 0.001705872 | 0.001318394 | 0.001311938 | 0.001301715
128KB | 0.004714194 | 0.000979602 | 0.000900821 | 0.000877330 | 0.000858849
256KB | 0.004358955 | 0.000725762 | 0.000635524 | 0.000615878 | 0.000589340
512KB | 0.004212568 | 0.000629993 | 0.000590974 | 0.000588392 | 0.000586606
1MB | 0.001736093 | 0.000600644 | 0.000583608 | 0.000583475 | 0.000583810
-----
Compulsory (256MB 2-way): 0.000015227900

```

For GCC the full associativity give the best result but MESA 8-Way give the best results. I can argue is best to use full associativity because GCC improves like 2/3 times with this change

- As previously seen MESA benchmark is an example of it. And as I said before this can be likely to happen when the less association caches benefits of prefetching.

Exercise 5.29

To support multiple virtual machines, two levels of memory virtualization are needed. Each virtual machine still controls the mapping of virtual address (VA) to physical address (PA), while the hypervisor maps the physical address (PA) of each virtual machine to the actual machine address (MA). To accelerate such mappings, a software approach called “shadow paging” duplicates each virtual machine’s page tables in the hypervisor, and intercepts VA to PA mapping changes to keep both copies consistent. To remove the complexity of shadow page tables, a hardware approach called nested page table (NPT) explicitly supports two classes of page tables (VA⇒PA and PA⇒MA) and can walk such tables purely in hardware.

Consider the following sequence of operations: (1) Create process; (2) TLB miss; (3) page fault; (4) context switch;

1. What would happen for the given operation sequence for shadow page table and nested page table, respectively?
2. Assuming an x86-based four-level page table in both guest and nested page table, how many memory references are needed to service a TLB miss for native vs. nested page table?
3. Among TLB miss rate, TLB miss latency, page fault rate, and page fault handler latency, which metrics are more important for shadow page table? Which are important for nested page table?

Assume the following parameters for a shadow paging system.

TLB Misses per 1000 Instructions	NPT TLB Miss Latency	Page Faults per 1000 Instructions	Shadowing Page Fault Overhead
0.2	200 cycles	0.001	30,000 cycles

4. For a benchmark with native execution CPI of 1, what are the CPI numbers if using shadow page tables vs. NPT (assuming only page table virtualization overhead)?
5. What techniques can be used to reduce page table shadowing induced overhead?
6. What techniques can be used to reduce NPT induced overhead?

Solution

1. For a shadow page table (1) is going to create a page table into the virtual machine memory space and the hypervisor is going to create the shadow table as well. Then (2) if a TLB miss occur isn’t going to change anything. But (3) when a page fault happen the registry of the shadow table has to update and with in it need to change the TLB associated entry as well. If (4) a context switch happen then all the entries in TLB and VM page tables has to be dropped because there’s not going to be needed and has to be replaced by the hypervisor pages. For a NPT (1) is going to create a page table inside the VM with the VA/PA association and the hypervisor is going to create a page table with the PA/MA association. Then (2) if a TLB miss occur it is going to search the right page going from VA to PA to MA. When (3) a page fault happen the VM and hypervisor has to update their page entries and invalidate affected pages in TLB. If (4) a context switch happen it is going to drop the TLB and VA/PA entires and load TLB with just hypervisor pages.
2. For native pages is obviously it going to take 4. However for nested page tables can depend but in the worst case scenario it need to search among 4 nested page entries to search 1 native entry. So it can be up to 20 memory references.
3. For shadow page table the most important metrics are page fault rate and page fault handler latency because TLB changes only when page table entries updates. Fot NPT is going to be TLB miss rate and TLB miss latency because the page translation can take a lot to be done.

$$CPI_{Shadow} = 1 \frac{\text{cycle}}{\text{instruction}} + \left(\frac{0.001 \text{ Page Misses}}{1000 \text{ Instructions}} \right) * 30000 \text{ cycles} = 1.03 \frac{\text{cycle}}{\text{instruction}}$$

$$4. \quad CPI_{NPT} = 1 \frac{\text{cycle}}{\text{instruction}} + \left(\frac{0.2 \text{ TLB Misses}}{1000 \text{ Instructions}} \right) * 200 \text{ cycles} = 1.04 \frac{\text{cycle}}{\text{instruction}}$$

5. There is not much that can be done. Maybe a parallel write between original table and shadow table.
6. It can be designed a kind of TLB that point a direct to PA/MA table so it can save translation time and easy context switch.

Chapter 6

Exercise 6.1

First, write down a list of your daily activities that you typically do on a weekday. For instance, you might get out of bed, take a shower, get dressed, eat breakfast, dry your hair, brush your teeth. Make sure to break down your list so you have a minimum of 10 activities.

1. Now consider which of these activities is already exploiting some form of parallelism (e.g., brushing multiple teeth at the same time, versus one at a time, carrying one book at a time to school, versus loading them all into your backpack and then carry them “in parallel”). For each of your activities, discuss if they are already working in parallel, but if not, why they are not.
2. Next, consider which of the activities could be carried out concurrently (e.g., eating breakfast and listening to the news). For each of your activities, describe which other activity could be paired with this activity.
3. For Exercise 6.1.2, what could we change about current systems (e.g., showers, clothes, TVs, cars) so that we could perform more tasks in parallel?
4. Estimate how much shorter time it would take to carry out these activities if you tried to carry out as many tasks in parallel as possible.

Solution

/*Pending*/

Exercise 6.2

You are trying to bake three blueberry pound cakes. Cake ingredients are as follows:

1 cup butter, softened
1 cup sugar
4 large eggs
1 teaspoon vanilla extract
1/2 teaspoon salt
1/4 teaspoon nutmeg
1 1/2 cups flour
1 cup blueberries

The recipe for a single cake is as follows:

Step 1: Preheat oven to 325°F (160°C). Grease and flour your cake pan.

Step 2: In large bowl, beat together with a mixer butter and sugar at medium speed until light and fluffy. Add eggs, vanilla, salt and nutmeg. Beat until thoroughly blended. Reduce mixer speed to low and add flour, 1/2 cup at a time, beating just until blended.

Step 3: Gently fold in blueberries. Spread evenly in prepared baking pan. Bake for 60 minutes.

1. Your job is to cook three cakes as efficiently as possible. Assuming that you only have one oven large enough to hold one cake, one large bowl, one cake pan, and one mixer, come up with a schedule to make three cakes as quickly as possible. Identify the bottlenecks in completing this task.
2. Assume now that you have three bowls, three cake pans and three mixers. How much faster is the process now that you have additional resources?
3. Assume now that you have two friends that will help you cook, and that you have a large oven that can accommodate all three cakes. How will this change the schedule you arrived at in Exercise 6.2.1 above?
4. Compare the cake-making task to computing three iterations of a loop on a parallel computer. Identify data-level parallelism and task-level parallelism in the cake-making loop.

Solution

1. With those resources you must do all task in order. However there's an opportunity to parallelize waiting for baking to be done so you can do steps from first to mid of third step. You can think it like pipeline the work
2. With more resources there is even some bottlenecks because you are only one person that can do all those tasks. However you can bake 3 cakes at the same time so you can think it like superscale the operation and doing some concurrent task improving throughput of baked cakes
3. With those friends you can actually do 3 cakes at the same time. Full parallelism
4. Data level parallelism is comparable when you have more resources to dispose like the quantity of mixers, ovens, etc. For task parallelism is comparable with the amount of people baking cakes

Exercise 6.3

Many computer applications involve searching through a set of data and sorting the data. A number of efficient searching and sorting algorithms have been devised in order to reduce the runtime of these tedious tasks. In this problem we will consider how best to parallelize these tasks.

1. Consider the following binary search algorithm (a classic divide and conquer algorithm) that searches for a value X in a sorted N -element array A and returns the index of matched entry:

```
BinarySearch(A[0..N-1], X) {
    low = 0
    high = N - 1
    while (low <= high) {
        mid = (low + high) / 2
        if (A[mid] > X)
            high = mid - 1
        else if (A[mid] < X)
            low = mid + 1
        else
            return mid    // found
    }
    return -1    // not found
}
```

Assume that you have Y cores on a multi-core processor to run BinarySearch. Assuming that Y is much smaller than N , express the speed-up factor you might expect to obtain for values of Y and N . Plot these on a graph.

2. Next, assume that Y is equal to N . How would this affect your conclusions in your previous answer? If you were tasked with obtaining the best speed-up factor possible (i.e., strong scaling), explain how you might change this code to obtain it.

Solution

1. As the name explain a binary search will try to search a term on a presorted list where in each iteration will discard half of the data to search the value on the other half of the data. If you think it will halve the data until it find it leaving the possibility that half until have a single item. The function of time needed to solve the algorithm will be of $\log_2(N)$, so if we use a multi core process to subdivide the set of data then the function changes to $\log_2(N/Y)$.
2. Not necessarily. When $N=Y$ the search takes $\log_2(1)=0$ so it takes just one iteration to know.

Exercise 6.4

Consider the following piece of C code:

```
for (j=2;j<=1000;j++)
    D[j] = D[j-1]+D[j-2];
```

The RISC-V code corresponding to the above fragment is:

```
addi    x5, x0, 8000
add     x12, x10, x5
```

```

    addi    x11, x10, 16
LOOP:
    fld     f0, -16(x11)
    fld     f1, -8(x11)
    fadd.d  f2, f0, f1
    fsd     f2, 0(x11)
    addi    x11, x11, 8
    ble     x11, x12, LOOP

```

The latency of an instruction is the number of cycles that must come between that instruction and an instruction using the result. Assume floating point instructions have the following associated latencies (in cycles):

fadd.d	fld	fsd
4	6	1

1. How many cycles does it take to execute this code?
2. Re-order the code to reduce stalls. Now, how many cycles does it take to execute this code? (Hint: You can remove additional stalls by changing the offset on the fsd instruction.)
3. When an instruction in a later iteration of a loop depends upon a data value produced in an earlier iteration of the same loop, we say that there is a loop-carried dependence between iterations of the loop. Identify the loop carried dependencies in the above code. Identify the dependent program variable and assembly-level registers. You can ignore the loop induction variable j.
4. Rewrite the code by using registers to carry the data between iterations of the loop (as opposed to storing and re-loading the data from main memory). Show where this code stalls and calculate the number of cycles required to execute. Note that for this problem you will need to use the assembler pseudo instruction “fmv.d rd, rs1,” which writes the value of floating-point register rs1 into floating-point register rd. Assume that fmv.d executes in a single cycle.
5. Loop unrolling was described in Chapter 4. Unroll and optimize the loop above so that each unrolled loop handles three iterations of the original loop. Show where this code stalls and calculate the number of cycles required to execute.
6. The unrolling from Exercise 6.4.5. works nicely because we happen to want a multiple of three iterations. What happens if the number of iterations is not known at compile time? How can we efficiently handle a number of iterations that isn’t a multiple of the number of iterations per unrolled loop?
7. Consider running this code on a two-node distributed memory message passing system. Assume that we are going to use message passing as described in Section 6.8, where we introduce a new operation send (x, y) that sends to node x the value y, and an operation receive() that waits for the value being sent to it. Assume that send operations take one cycle to issue (i.e., later instructions on the same node can proceed on the next cycle), but take several cycles to be received on the receiving node. Receive instructions stall execution on the node where they are executed until they receive a message. Can you use such a system to speed up the code for this exercise? If so, what is the maximum latency for receiving information that can be tolerated? If not, why not?

Solution

1. In order to wait the number of cycles of some instructions and respecting the dependencies and order of the instructions we should add nop instructions like this


```

    addi    x5, x0, 8000
    add     x12, x10, x5
    addi    x11, x10, 16
LOOP:
    fld     f0, -16(x11)
    fld     f1, -8(x11)
    nop
    nop

```

```

nop
nop
nop
nop
fadd.d f2, f0, f1
nop
nop
nop
nop
fsd    f2, 0(x11)
addi   x11, x11, 8
ble    x11, x12, LOOP

```

The program takes 999 loop iterations to go from 2 to 1000. so the total is of $3+999*16=15987$ cycles

2. If we change the offset of fsd we can execute the addi instruction before saving one instruction each iteration making a total of $3+999*15=14988$ cycles
3. Well we have a chain of dependencies. First the 2 instructions fld are dependencies of fadd.d (because the values inside f0 and f1) and itself is a dependency of fsd (because the value of f3). All 3 instruction has a dependency with x11 as well
4. We can save the value of $D[j-1]$ because in the next iteration will become the value of $D[j-2]$. So in addition with the modification of changing the order of the program will get:

```

addi   x11, x10, 16
fld    f0, -16(x11)
fld    f1, -8(x11)
addi   x5, x0, 8000
add    x12, x10, x5
nop
nop
nop
nop

```

LOOP:

```

fadd.d f2, f0, f1
addi   x11, x11, 8
fmv.d  f0, f1
fmv.d  f1, f2
nop
fsd    f2, -8(x11)
ble    x11, x12, LOOP

```

And with this change the program takes $9+999*7=7002$ cycles

5. In order to unroll the loop and making a third of loop iterations we need the use of extra registers. If we thought how many different load are needed we would get to the conclusion of 5 different loads.

```

addi   x11, x10, 16
fld    f0, -8(x11)
fld    f1, -16(x11)
addi   x5, x0, 8000
add    x12, x10, x5
nop
nop
nop
nop

```

LOOP:

```

fadd.d f2, f0, f1
addi   x11, x11, 24

```

```

nop
nop
nop
fadd.d f0, f1, f2
fsd    f2, -24(x11)
nop
nop
nop
fadd.d f1, f2, f0
fsd    f0, -16(x11)
nop
nop
nop
fsd    f1, -8(x11)
ble    x11, x12, LOOP

```

With loop unrolling we have that the program takes $9 + 333 \cdot 17 = 5670$ cycles to complete

6. Well in that case we need the 2 loops in the same program (unrolled and original). We can verify if the number of remaining loops is less than the unrolled times iterations so if it is true we switch to the original loop.
7. There's no way to parallelized this. This is because the data to do the operation depends on previous results. The technique of doing distributed computing is for the task to be able to execute and has no dependencies in the whole sub-process.

Exercise 6.5

Consider the following recursive mergesort algorithm (another classic divide and conquer algorithm). Mergesort was first described by John Von Neumann in 1945. The basic idea is to divide an unsorted list x of m elements into two sublists of about half the size of the original list. Repeat this operation on each sublist, and continue until we have lists of size 1 in length. Then starting with sublists of length 1, “merge” the two sublists into a single sorted list.

```

Mergesort(m)
  var list left, right, result
  if length(m) <= 1
    return m
  else
    var middle = length(m) / 2
    for each x in m up to middle
      add x to left
    for each x in m after middle
      add x to right
    left = Mergesort(left)
    right = Mergesort(right)
    result = Merge(left, right)
    return result

```

The merge step is carried out by the following code:

```

Merge(left, right)
  var list result
  while length (left) > 0 and length (right) > 0
    if first(left) to result
      append first(left) to result
      left = rest(left)
    else
      append first(right) to result

```

```

        right = rest(right)
    if length (left) > 0
        append rest(left) to result
    if length (right) > 0
        append rest(right) to result
    return result

```

1. Assume that you have Y cores on a multicore processor to run Mergesort. Assuming that Y is much smaller than length (m), express the speed-up factor you might expect to obtain for values of Y and length (m). Plot these on a graph.
2. Next, assume that Y is equal to length (m). How would this affect your conclusions in your previous answer? If you were tasked with obtaining the best speed-up factor possible (i.e., strong scaling), explain how you might change this code to obtain it.

Solution

1. This algorithm will compose it of 3 mayor steps, divide, conquer and merge. At first we subdivide the array in halves until we get a m sets of a single term elements. If we run it in sequentially it will take $m-1$ operations at total. The conquer step consist in taking in account that when you merge the sub arrays every array is sorted by itself so if you merge a two $m/2$ subsets will take you $m-1$ comparisons to be sorted. Finally in merging step will do all those comparisons $\log_2(m)$ times. At the end if we do the algorithm in a sequential the algorithm takes a function of time of $m \cdot \log_2(m)$ to complete. If we want to run this algorithm in parallel we can make that every function call be forked and compute it in parallel. Making the complexity of this algorithm about $m/Y \cdot \log_2(m)$.
2. If we had the same number of cores or at least $m/2$ cores we can do the comparison of a couple of terms in parallel. Doing this comparison for m times we can be sure that the array will be sort so it takes a function of time m to solve it.

Exercise 6.6

Matrix multiplication plays an important role in a number of applications. Two matrices can only be multiplied if the number of columns of the first matrix is equal to the number of rows in the second.

Let's assume we have an $m \times n$ matrix A and we want to multiply it by an $n \times p$ matrix B . We can express their product as an $m \times p$ matrix denoted by AB (or $A \cdot B$). If we assign $C = AB$, and $c_{i,j}$ denotes the entry in C at

position (i, j), then for each element i and j with $1 \leq i \leq m$ and $1 \leq j \leq p$ $c_{i,j} = \sum_{k=1}^n a_{i,k} \times b_{k,j}$. Now we want to see

if we can parallelize the computation of C . Assume that matrices are laid out in memory sequentially as follows:

$a_{1,1}, a_{2,1}, a_{3,1}, a_{4,1}, \dots$, etc.

1. Assume that we are going to compute C on both a single-core shared-memory machine and a four-core shared-memory machine. Compute the speed-up we would expect to obtain on the four-core machine, ignoring any memory issues.
2. Repeat Exercise 6.6.1, assuming that updates to C incur a cache miss due to false sharing when consecutive elements are in a row (i.e., index i) are updated.
3. How would you fix the false sharing issue that can occur?

Solution

1. In order to a process to be fully parallelized is needed to every small step to be free of any dependency, which in this case is true. However as the Amdahl's laws explain this speed up is close of the number of resources put in speed up. That mean if we have 4 cores instead of 1 we will accomplish a speed up close of x4 but never equal.
2. A feature of cache memory is that when you request a missing data in cache it will load near data inside the same block, this can be wonderful for matrix multiplication however if the cache size is smaller that the number of m or n or p (it depends how memory is structured) it can fall into conflict misses cache. That happens because proximity doesn't work in both ways for rows and columns (In one case the data

needed is load in a single block request but for the other case will need to load a block for every data in a pattern that can lead to conflict loading data on the same block)

3. The best recommendation for speed up using parallel computing is subdividing the target matrix in 4 submatrix so every core will compute $m/4 * n * p/4$ multiplications and $m/4 * (n-1) * p/4$ additions. If the fourth of size of m or p can be loaded all inside the cache then parallelize calculations will be close of $x4$ gain but if it can't fit in the cache can actually lead in worst performance.

Exercise 6.7

Consider the following portions of two different programs running at the same time on four processors in a symmetric multicore processor (SMP). Assume that before this code is run, both x and y are 0.

Core 1: $x = 2$;

Core 2: $y = 2$;

Core 3: $w = x + y + 1$;

Core 4: $z = x + y$;

1. What are all the possible resulting values of w, x, y , and z ? For each possible outcome, explain how we might arrive at those values. You will need to examine all possible interleavings of instructions.
2. How could you make the execution more deterministic so that only one set of values is possible?

Solution

1. I will list all the possible outcomes

Case 1		Case 2		Case 3	
Order of execution	Values	Order of execution	Values	Order of execution	Values
$x = 2$; $y = 2$; $w = x + y + 1$; $z = x + y$;	$x = 2$ $y = 2$ $w = 5$ $z = 4$	$x = 2$; $w = x + y + 1$; $y = 2$; $z = x + y$;	$x = 2$ $y = 2$ $w = 3$ $z = 4$	$x = 2$; $w = x + y + 1$; $z = x + y$; $y = 2$;	$x = 2$ $y = 2$ $w = 3$ $z = 2$
Case 4		Case 5		Case 6	
Order of execution	Values	Order of execution	Values	Order of execution	Values
$w = x + y + 1$; $x = 2$; $y = 2$; $z = x + y$;	$x = 2$ $y = 2$ $w = 1$ $z = 4$	$w = x + y + 1$; $x = 2$; $z = x + y$; $y = 2$;	$x = 2$ $y = 2$ $w = 1$ $z = 2$	$w = x + y + 1$; $z = x + y$; $x = 2$; $y = 2$;	$x = 2$ $y = 2$ $w = 1$ $z = 0$
Case 7		Case 8		Case 9	
Order of execution	Values	Order of execution	Values	Order of execution	Values
$x = 2$; $z = x + y$; $y = 2$; $w = x + y + 1$;	$x = 2$ $y = 2$ $w = 5$ $z = 2$	$z = x + y$; $x = 2$; $y = 2$; $w = x + y + 1$;	$x = 2$ $y = 2$ $w = 5$ $z = 0$	$z = x + y$; $x = 2$; $w = x + y + 1$; $y = 2$;	$x = 2$ $y = 2$ $w = 3$ $z = 0$

2. You can use a synchronization instruction like FENCE that update the status of memory modified by one core to be seen by all cores.

Exercise 6.8

The dining philosopher's problem is a classic problem of synchronization and concurrency. The general problem is stated as philosophers sitting at a round table doing one of two things: eating or thinking. When they are eating, they are not thinking, and when they are thinking, they are not eating. There is a bowl of pasta in the center. A fork is placed in between each philosopher. The result is that each philosopher has one fork to her left

and one fork to her right. Given the nature of eating pasta, the philosopher needs two forks to eat, and can only use the forks on her immediate left and right. The philosophers do not speak to one another.

1. Describe the scenario where none of philosophers ever eats (i.e., starvation). What is the sequence of events that happen that lead up to this problem?
2. Describe how we can solve this problem by introducing the concept of a priority. Can we guarantee that we will treat all the philosophers fairly? Explain.

Now assume we hire a waiter who is in charge of assigning forks to philosophers. Nobody can pick up a fork until the waiter says they can. The waiter has global knowledge of all forks. Further, if we impose the policy that philosophers will always request to pick up their left fork before requesting to pick up their right fork, then we can guarantee to avoid deadlock.

3. We can implement requests to the waiter as either a queue of requests or as a periodic retry of a request. With a queue, requests are handled in the order they are received. The problem with using the queue is that we may not always be able to service the philosopher whose request is at the head of the queue (due to the unavailability of resources). Describe a scenario with five philosophers where a queue is provided, but service is not granted even though there are forks available for another philosopher (whose request is deeper in the queue) to eat.
4. If we implement requests to the waiter by periodically repeating our request until the resources become available, will this solve the problem described in Exercise 6.8.3? Explain.

Solution

1. This could happen if one philosopher take one fork and their partner also take one fork. Because both wants to eat but doesn't have both forks then it could lead to starvation.
2. The waiter can attend the request in the order so it can put them in a queue where even some kind of score which take on count about the last time since they eat or how much they eat can be provided to prioritize some philosophers among others. With it it the waiter can guard one of the forks and give them to the head of the queue.
3. In this case is supposed that many philosophers can eat at the same time. If the table is round and the forks are put between them they had 5 forks but they need 2 so it can happen that the 2 neighbours of the head queue are busy. Meaning it's their turn but there's no forks because their neighbours took them.
4. Can solve it but by saturating the queue with many request will deny to other philosophers to be attended.

Exercise 6.9

Consider the following three CPU organizations:

CPU SS: A two-core superscalar microprocessor that provides out-of-order issue capabilities on two function units (FUs). Only a single thread can run on each core at a time.

CPU MT: A fine-grained multithreaded processor that allows instructions from two threads to be run concurrently (i.e., there are two functional units), though only instructions from a single thread can be issued on any cycle.

CPU SMT: An SMT processor that allows instructions from two threads to be run concurrently (i.e., there are two functional units), and instructions from either or both threads can be issued to run on any cycle.

Assume we have two threads X and Y to run on these CPUs that include the following operations:

Thread X	Thread Y
A1 – takes three cycles to execute	B1 – take two cycles to execute
A2 – no dependences	B2 – conflicts for a functional unit with B1
A3 – conflicts for a functional unit with A1	B3 – depends on the result of B2
A4 – depends on the result of A3	B4 – no dependences and takes two cycles to execute

Assume all instructions take a single cycle to execute unless noted otherwise or they encounter a hazard.

1. Assume that you have one SS CPU. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?
2. Now assume you have two SS CPUs. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?
3. Assume that you have one MT CPU. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?
4. Assume you have one SMT CPU. How many cycles will it take to execute the two threads? How many issue slots are wasted due to hazards?

Solution

1. If we assume that every task takes one cycle but if says otherwise then we can track the executions as the following table. Because is a speculative out of order processor we can pack many instructions if there's no dependency, also the best strategy is to process the task which monopolize functions units

Core 1		Core 2	
Functional Unit 1	Function Unit 2	Functional Unit 1	Function Unit 2
A3	-	B2	-
A1	A2	B1	B4
A1	A4	B1	B4
A1	-	B3	-

2. It will be the same, because it isn't possible to subdivide this task in more cores.
3. This core execute the process in order and it probably will switch from tread for every batch of instructions packed in one cycle.

Core 1	
Functional Unit 1	Functional Unit 2
A1	A2
A1	-
A1	-
B1	-
B1	-
A3	-
B2	-
A4	-
B3	B4
-	B4

4. We get the follow execution order

Core 1

Functional Unit 1	Functional Unit 2
A1	B1
A1	B1
A1	A2
B2	A3
B3	A4
B4	-
B4	-

Exercise 6.10

Virtualization software is being aggressively deployed to reduce the costs of managing today's high-performance servers. Companies like VMWare, Microsoft, and IBM have all developed a range of virtualization products. The general concept, described in Chapter 5, is that a hypervisor layer can be introduced between the hardware and the operating system to allow multiple operating systems to share the same physical hardware. The hypervisor layer is then responsible for allocating CPU and memory resources, as well as handling services typically handled by the operating system (e.g., I/O).

Virtualization provides an abstract view of the underlying hardware to the hosted operating system and application software. This will require us to rethink how multi-core and multiprocessor systems will be designed in the future to support the sharing of CPUs and memories by a number of operating systems concurrently.

1. Select two hypervisors on the market today, and compare and contrast how they virtualize and manage the underlying hardware (CPUs and memory).
2. Discuss what changes may be necessary in future multi-core CPU platforms in order to better match the resource demands placed on these systems. For instance, can multithreading play an effective role in alleviating the competition for computing resources?

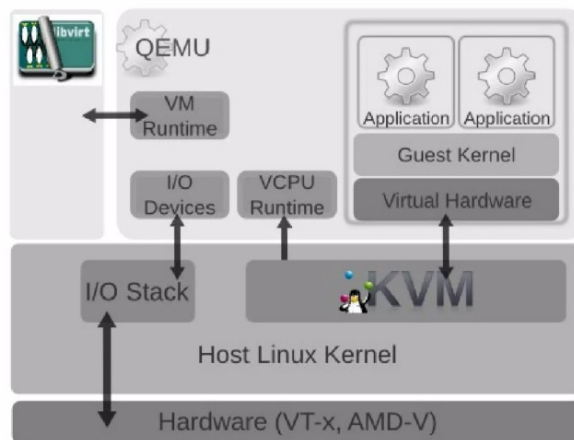
Solution

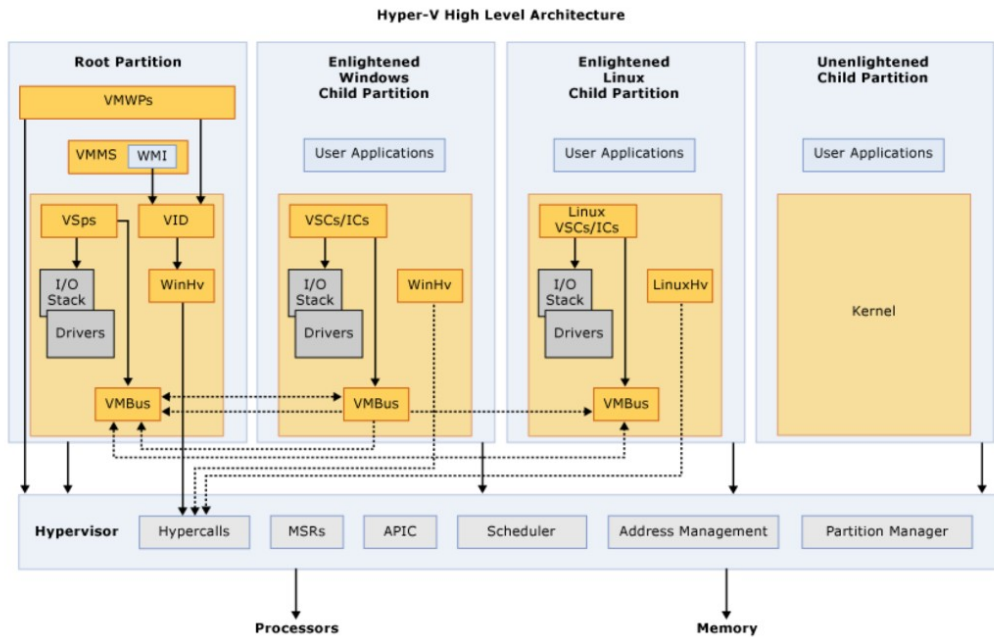
1. I will select KVM and Hyper-V

	KVM with QEMU	Hyper-V
About	This hypervisor is embedded with the Linux kernel and provides an API to handle virtual calls, That let us to run VMs with native performance inside the host OS allowing us to keep it among other processes and QEMU provides the management of a VM like virtual devices and status.	Is an Hypervisor embedded with the Windows OS. It provides an API which handles virtual calls. The VMs are managed by partitions where there's a parent partition where is saved the state of the other VMs and try to resolve virtual devices trough a VM bus.
CPUs	It has the capacity to create up to 512 vcpu using hardware threads and provide different features (mostly of security and restrictions) per architecture because many are supported (x86, ARM, PowerPC, s390 and MIPS). But it try to use the virtualisation extension of every architecture where provides the illusions of running on the most privileged mode but they are protected. For example	Also take advantage of the virtual instruction set of x86 (which is the only architecture supported). Any CPU trap is achieved by the API.

	for x86 it uses AMD-V and XT-V.	
Memory	It uses shadow page tables (a table conversion from VA->PA->MA) to track pages through VMs. However because it is to slow some architectures like x86 provides hardware speedup through extended/nested page tables to change quickly TBLs and flush caches.	Here the parent partition has the physical address access. So the read/write requests are send into a bus (VMBus). Then the parent partition allows memory access and even DMA features for great throughput capacities.
I/Os	To avoid high overhead I/O requests are diverged to run directly to the system calls of the guest OS. However it can be managed in 3 different ways. One is emulating devices where QEMU manages all the behaviour of the device. Next is virtio where the I/O request is handled by the Kernel and the last one leave the capacity of using the hardware device directly (but if you use it the Host cannot use the device)	It consist in creating virtual devices on the parent partition and managing them. Then the guest OS will use the Virtual Server Client where capture I/O request and send them through the VMBus and managed by the Virtualization Service Provider

KVM Architecture





- Yes, the use of multi threading can alleviate the demand because you can create an vcpu for each thread and now using cluster of many individuals machines to provide a pool of resources where a VM can run in total different machines

Exercise 6.11

We would like to execute the loop below as efficiently as possible. We have two different machines, a MIMD machine and a SIMD machine.

```
for (i=0; i<2000; i++)
    for (j=0; j<3000; j++)
        X_array[i][j] = Y_array[j][i] + 200;
```

- For a four-CPU MIMD machine, show the sequence of RISC-V instructions that you would execute on each CPU. What is the speed-up for this MIMD machine?
- For an eight-wide SIMD machine (i.e., eight parallel SIMD functional units), write an assembly program in using your own SIMD extensions to RISC-V to execute the loop. Compare the number of instructions executed on the SIMD machine to the MIMD machine.

Solution

- In theory we can get an speedup of 4x compared to SIMD. If we consider the cache problem of maintaining spatial locality then we need to subdivide the matrix using the columns (j). Then we use SIMD instructions to fetch packed data in rows (i) to avoid memory stalls. If this done the gain in performance comparing a SISD can be huge, about 32 times or even better.
- /*Pending*/

Exercise 6.12

A systolic array is an example of an MISD machine. A systolic array is a pipeline network or “wavefront” of data processing elements. Each of these elements does not need a program counter since execution is triggered by the arrival of data. Clocked systolic arrays compute in “lock-step” with each processor undertaking alternate compute and communication phases.

- Consider proposed implementations of a systolic array (you can find these on the Internet or in technical publications). Then attempt to program the loop provided in Exercise 6.11 using this MISD model. Discuss any difficulties you encounter.
- Discuss the similarities and differences between an MISD and SIMD machines. Answer this question in terms of data-level parallelism.

Solution

1. Well the main difficulty is that systolic arrays cores is that need to match the number of cores to the number of cells to compute to be able to get a result. You can do the trick of doing sub matrices to compute them and then packing them in the correct order
2. Both seems good to do matrix multiplications. However MISD are limited by the number of cores and SIMD are limited by the bandwidth of instructions

Exercise 6.13

Assume we want to execute the DAXPY loop shown on page 529 in RISC-V vector assembly on the NVIDIA 8800 GTX GPU described in this chapter. In this problem, we will assume that all math operations are performed on single precision floating-point numbers (we will rename the loop SAXPY). Assume that instructions take the following number of cycles to execute.

Loads	Stores	Add.S	Mult.S
5	2	3	4

1. Describe how you will constructs warps for the SAXPY loop to exploit the eight cores provided in a single multiprocessor.

Solution

/*Pending*/

Exercise 6.14

Download the CUDA Toolkit and SDK from <https://developer.nvidia.com/cuda-toolkit>. Make sure to use the “emurelease” (Emulation Mode) version of the code. (You will not need actual NVIDIA hardware for this assignment.) Build the example programs provided in the SDK, and confirm that they run on the emulator.

1. Using the “template” SDK sample as a starting point, write a CUDA program to perform the following vector operations:

A) $a - b$ (vector-vector subtraction)

B) $a \cdot b$ (vector dot product)

The dot product of two vectors $a = [a_1, a_2, \dots, a_n]$ and $b = [b_1, b_2, \dots, b_n]$ is defined as:

$$a \cdot b = \sum_{i=1}^n a_i \cdot b_i + a_2 \cdot b_2 + \dots + a_n \cdot b_n$$

Submit code for each program that demonstrates each operation and verifies the correctness of the results.

2. If you have GPU hardware available, complete a performance analysis on your program, examining the computation time for the GPU and a CPU version of your program for a range of vector sizes. Explain any results you see.

Solution

/*Pending*/

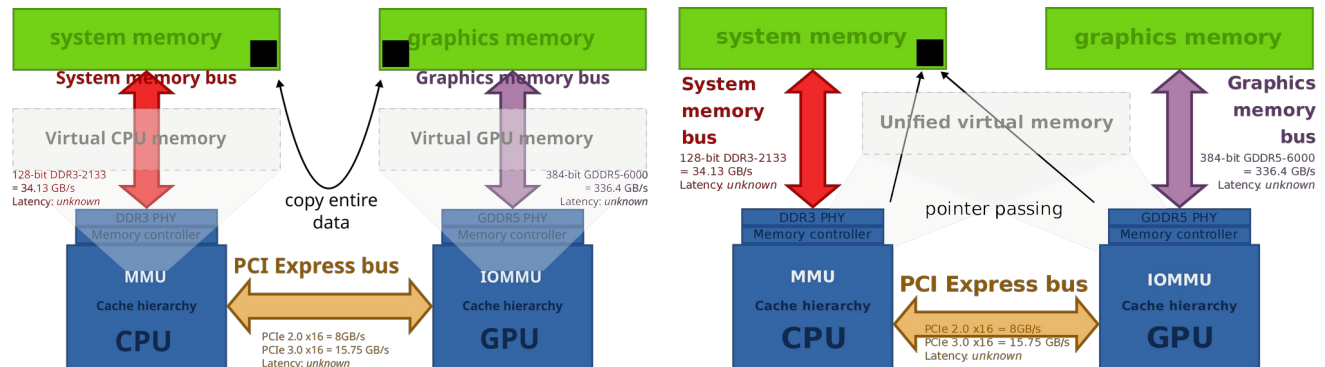
Exercise 6.15

AMD has recently announced integrating a graphics processing unit with their x86 cores into a single package (though with different clocks for each of the cores). This is an example of a heterogeneous multiprocessor system. One of the key design points is to allow for fast data communication between the CPU and the GPU. Before AMD’s Fusion architecture, communications were needed between discrete CPU and GPU chips. Presently, the plan is to use multiple (at least 16) PCI express channels to facilitate intercommunication.

1. Compare the bandwidth and latency associated with these two interconnect technologies.

Solution

1. The heterogeneous System Architecture (HSA) Is a system which provide the capability for a program to avoid coping data from virtual memory of CPU to virtual memory to GPU. Normally when a task is delegated to be computed by the GPU all data required data has to be found by the MMU translating the virtual address to then copying it into the Bus (PCIe) to be processed by IOMMU (an MMU for devices which doesn't share the CPU bus like GPUs) then the IOMMU allocates the data and copy it into GPU memory to then start the job. With HSA we can map altogether MMU and IOMMU to share virtual memory with main memory and GPU memory (Like an real address extension but inside an specific virtual address I suppose). So instead passing data just share memory pointers to the data. This provide a latency save (Because it don't have to allocate space nor translating virtual addresses) and saves bandwidth because the data isn't copied. However If the GPU start working with main memory then the computing power is dropped. I guess maybe there's a way to move this pages into the best memory system that is needed.



Exercise 6.16

Refer to Figure 6.15b, which shows an n-cube interconnect topology of order 3 that interconnects eight nodes. One attractive feature of an n-cube interconnection network topology is its ability to sustain broken links and still provide connectivity.

1. Develop an equation that computes how many links in the n-cube (where n is the order of the cube) can fail and we can still guarantee an unbroken link will exist to connect any node in the n-cube.
2. Compare the resiliency to failure of n-cube to a fully connected interconnection network. Plot a comparison of reliability as a function of the added number of links for the two topologies.

Solution

1. If you need to guaranty the availability of all nodes then the minimum number of broken links has to be the number of edges per node but one. So it increases in order of N. So for 2^N cube nodes the maximum number of broken links are N-1
2. A full interconnection network grows the number of links at rate of $N*(N-1)/2$ and because all the nodes are fully connected then you can rely on a single link path of N-1 edges.

Exercise 6.17

Benchmarking is a field of study that involves identifying representative workloads to run on specific computing platforms in order to be able to objectively compare performance of one system to another. In this exercise we will compare two classes of benchmarks: the Whetstone CPU benchmark and the PARSEC Benchmark suite. Select one program from PARSEC. All programs should be freely available on the Internet. Consider running multiple copies of Whetstone versus running the PARSEC Benchmark on any of the systems described in Section 6.11.

1. What is inherently different between these two classes of workload when run on these multi-core systems?
2. In terms of the Roofline Model, how dependent will the results you obtain when running these benchmarks be on the amount of sharing and synchronization present in the workload used?

Solution

/*Pending*/

Exercise 6.18

When performing computations on sparse matrices, latency in the memory hierarchy becomes much more of a factor. Sparse matrices lack the spatial locality in the datastream typically found in matrix operations. As a result, new matrix representations have been proposed.

One of the earliest sparse matrix representations is the Yale Sparse Matrix Format. It stores an initial sparse $m \times n$ matrix, M in row form using three one dimensional arrays. Let R be the number of nonzero entries in M . We construct an array A of length R that contains all nonzero entries of M (in left-to-right top to-bottom order). We also construct a second array IA of length $m+1$ (i.e., one entry per row, plus one). $IA(i)$ contains the index in A of the first nonzero element of row i . Row i of the original matrix extends from $A(IA(i))$ to $A(IA(i+1)-1)$. The third array, JA , contains the column index of each element of A , so it also is of length R .

1. Consider the sparse matrix X below and write C code that would store this code in Yale Sparse Matrix Format.

Row 1 [1, 2, 0, 0, 0, 0]

Row 2 [0, 0, 1, 1, 0, 0]

Row 3 [0, 0, 0, 0, 9, 0]

Row 4 [2, 0, 0, 0, 0, 2]

Row 5 [0, 0, 3, 3, 0, 7]

Row 6 [1, 3, 0, 0, 0, 1]

2. In terms of storage space, assuming that each element in matrix X is single-precision floating point, compute the amount of storage used to store the matrix above in Yale Sparse Matrix Format.
3. Perform matrix multiplication of matrix X by matrix Y shown below.

[2, 4, 1, 99, 7, 2]

Put this computation in a loop, and time its execution. Make sure to increase the number of times this loop is executed to get good resolution in your timing measurement. Compare the runtime of using a naïve representation of the matrix, and the Yale Sparse Matrix Format.

4. Can you find a more efficient sparse matrix representation (in terms of space and computational overhead)?

Solution

1. Code

```
69 sparsematrix_t* getSparseMatrix(matrix_t* ptrMatrix) {
70     int i, j, R=0;
71     for(i=0; i<ptrMatrix->rows; i++) {
72         for(j=0; j<ptrMatrix->cols; j++) {
73             if (*(ptrMatrix->ptrMatrix+ptrMatrix->cols*i+j) != 0) {
74                 R++;
75             }
76         }
77     }
78
79     sparsematrix_t* sparseMatrix = malloc(sizeof(sparsematrix_t));
80     if(!sparseMatrix) {
81         printf("Failed to allocate sparseMatrix structure\n");
82         exit(1);
83     }
84     sparseMatrix->sizeA = R;
85     sparseMatrix->sizeIA = ptrMatrix->rows+1;
86     sparseMatrix->sizeJA = R;
87     sparseMatrix->ptrA = malloc(sizeof(int)*sparseMatrix->sizeA);
88     sparseMatrix->ptrIA = malloc(sizeof(int)*sparseMatrix->sizeIA);
89     sparseMatrix->ptrJA = malloc(sizeof(int)*sparseMatrix->sizeJA);
90     if (!sparseMatrix->ptrA || !sparseMatrix->ptrIA || !sparseMatrix->ptrJA) {
91         printf("Failed to allocate sparseMatrix data\n");
```

```

92     exit(1);
93 }
94 R = 0;
95 for(i=0; i<ptrMatrix->rows; i++) {
96     *(sparseMatrix->ptrIA+i) = R;
97     for(j=0; j<ptrMatrix->cols; j++) {
98         if(*(ptrMatrix->ptrMatrix+ptrMatrix->cols*i+j) != 0) {
99             *(sparseMatrix->ptrA+R) = *(ptrMatrix->ptrMatrix+ptrMatrix-
>cols*i+j);
100             *(sparseMatrix->ptrJA+R) = j;
101             R++;
102         }
103     }
104 }
105
106 *(sparseMatrix->ptrIA+sparseMatrix->sizeIA-1) = R;
107
108 return sparseMatrix;
109 }

```

2. If the result of the matrices is the following

A [1, 2, 1, 1, 9, 2, 2, 3, 3, 7, 1, 3, 1]

IA [0, 2, 4, 5, 7, 10, 13]

JA [0, 1, 2, 3, 4, 0, 5, 2, 3, 5, 0, 1, 5]

For A we have 13 elements, IA has 7 and JA has 13. If A is filled with single precision floating point and the numbers in IA and JA are small integers then the Yale Sparse Matrix occupy $13*4+7*2+13*2=92$ Bytes

3. /*Pending*/

4. Probably are better forms but this mode is at least the best standard way to optimize a Sparse Matrix

Exercise 6.19

In future systems, we expect to see heterogeneous computing platforms constructed out of heterogeneous CPUs. We have begun to see some appear in the embedded processing market in systems that contain both floating-point DSPs and microcontroller CPUs in a multichip module package.

Assume that you have three classes of CPU:

CPU A—A moderate-speed multi-core CPU (with a floating-point unit) that can execute multiple instructions per cycle.

CPU B—A fast single-core integer CPU (i.e., no floating-point unit) that can execute a single instruction per cycle.

CPU C—A slow vector CPU (with floating-point capability) that can execute multiple copies of the same instruction per cycle.

Assume that our processors run at the following frequencies:

CPU A	CPU B	CPU C
1 GHz	3 GHz	250 MHz

CPU A can execute two instructions per cycle, CPU B can execute one instruction per cycle, and CPU C can execute eight instructions (through the same instruction) per cycle. Assume all operations can complete execution in a single cycle of latency without any hazards.

All three CPUs have the ability to perform integer arithmetic, though CPU B cannot perform floating point arithmetic. CPU A and B have an instruction set similar to a RISC-V processor. CPU C can only perform floating point add and subtract operations, as well as memory loads and stores. Assume all CPUs have access to shared memory and that synchronization has zero cost.

The task at hand is to compare two matrices X and Y that each contain 1024×1024 floating-point elements. The output should be a count of the number of indices where the value in X was larger or equal to the value in Y.

1. Describe how you would partition the problem on the three different CPUs to obtain the best performance.
2. What kind of instruction would you add to the vector CPU C to obtain better performance?

Solution

1. First we need to compare the amount of instructions that every core can do and which kind of instruction can do

$$IPS = \text{Clock rate} * IPC$$

$$IPS_A = 1 * 10^9 \text{ GHz} * 2 \text{ instr/cycle} = 2 * 10^9 \text{ instr/sec}$$

$$IPS_B = 3 * 10^9 \text{ GHz} * 1 \text{ instr/cycle} = 3 * 10^9 \text{ instr/sec}$$

$$IPS_C = 0.250 * 10^9 \text{ GHz} * 8 \text{ instr/cycle} = 2 * 10^9 \text{ instr/sec}$$

Now, because we have similar performance in all CPUs but different utilities then the theory tell us that core A is best for parallel algorithms (like matrix multiplication), B is best sequential algorithms (like control behaviours) and C for high bandwidth operations (like load and stores of spatial locality data).

Then for comparing all elements between X and Y we can use A for comparing sub matrices in parallel and C to load and store all data from X and Y. B isn't useful for this particular task.

2. If C could compare data this task could be achieved only using C and with better performance

Exercise 6.20

This question looks at the amount of queuing that is occurring in the system given a maximum transaction processing rate, and the latency observed on average for a transaction. The latency includes both the service time (which is computed by the maximum rate) and the queue time.

Assume a quad-core computer system can process database queries at a steady state maximum rate of rate requests per second. Also assume that each transaction takes, on average, lat ms to process. For each of the pairs in the table, answer the following questions:

Average Transaction Latency	Maximum transaction processing rate
1 ms	5000/sec
2 ms	5000/sec
1 ms	10,000/sec
2 ms	10,000/sec

For each of the pairs in the table, answer the following questions:

1. On average, how many requests are being processed at any given instant?
2. If we move to an eight-core system, ideally, what will happen to the system throughput (i.e., how many queries/second will the computer process)?
3. Discuss why we rarely obtain this kind of speed-up by simply increasing the number of cores.

Solution

1. The following table explain any parameter

Avg. Transaction Latency	Maximum Transaction Processing Rate	Avg. Number of Transactions per Core Every Millisecond
1 ms	5000/sec	$1 * 5000 / 4 = 1.25$
2 ms	5000/sec	$2 * 5000 / 4 = 2.5$
1 ms	10000/sec	$1 * 10000 / 4 = 2.5$
2 ms	10000/sec	$2 * 10000 / 4 = 5$

2. Apparently it should double it
3. Because Amdahl's law. There's a limit about the process that can be fork in parallel. For example in this scenery the queue time cannot be parallelized so even if we double the cores the gain will be less of 2x.